# JWT 게시판 구현 보고서

## 01 서비스 소개와 구현 범위

**학번 : 202412937 이름: 박원후 제출일: 2026.10.28**

**서비스 이름 : "다 찾아드립니다"**

**기술 조합과 버전 : Vue 3 / Spring Boot / SQLite / Java JDK 21.0.12.1 / Node.js 24.14.1**

**GitHub 저장소 URL : https://github.com/awefs-svg/Web-First-assignment.git**

**소스 코드 기준**

**- 브랜치 : main**

**- 커밋 ID(SHA) : 3c0a84337d57c5c14f73bb86a15243ea6edd001a**

**[서비스 소개와 선택 이유 작성]**

"다찾아드립니다"는 대학 캠퍼스에서 발생하는 분실물 정보를 학생들이 공유할 수 있도록 돕는 서비스입니다. 사용자는 로그인하지 않아도 분실물 제보를 조회할 수 있으며, 로그인한 사용자는 자신이 분실하거나 발견한 물건의 특징과 위치를 게시글로 등록할 수 있습니다. 분실물의 습득 장소나 보관 위치에 대한 정보가 여러 곳에 흩어져 필요한 정보를 찾기 어렵다는 문제를 해결하고자 이 주제를 선택했습니다. Vue와 Spring Boot를 이용해 게시글 조회 및 JWT 기반 인증 기능을 구성하고, SQLite를 통해 데이터를 저장하며 작성자만 자신의 제보를 수정하거나 삭제할 수 있도록 구현했습니다.

| 기능 | 완료 여부 | 관련 파일 | 비고 또는 제한 |
|---|---|---|---|
| 공개 목록과 상세 | 완료 | BoardController.java , App.vue, api.js | 로그인 없이 게시글 목록 및 상세 조회 가능 |
| 로그인과 JWT 발급 | 완료 | AuthService.java, BoardController.java , api.js | 로그인 성공 시 JWT 발금 및 사용자 정보 반환 |
| 인증과 내 정보 조회 | 완료 | AuthSevice.java, BoardController.java , api.js | JWT 검증 후 인증된 사용자 정보 조회 |
| 본인 글 작성·수정·삭제 | 완료 | AuthSevice.java, BoardController.java , PostStore.java, App.vue, api.js | 로그인 후 작성 가능, 수정 삭제는 작성자 본인만 가능 |
| 만료·로그아웃·오류 표시 | 완료 | AuthSevice.java, App.vue, api.js | 만료 및 인증 오류 메시지 표시, 로그아웃 시 메모리의 토큰 초기화 |

| 기능 | 완료 여부 | 관련 파일 | 비고 또는 제한 |
| --- | --- | --- | --- |
| DB 데이터 유지 | 완료 | PostStore.java 등 DB 관련 파일 | SQLite에 게시글 저장, 새로고침이나 로그아웃 후에도 데이터 유지 |

# 구조와 변경 내역

(1) 브라우저 (Vue 프론트엔드)
: 사용자는 Vue로 구현된 브라우저 화면에서 로그인과 게시글 조회,작성,수정,삭제의 기능을 이용할 수 있습니다. 로그인 시 이메일과 비밀번호를 입력하면 api.js의 login()함수가 API서버로 POST /api/auth/login 요청을 전송합니다. 로그인에 성공하게 되면 서버에서 반환한 JWT와 사용자 정보를 JavaScript 메모리의 session변수에 저장합니다. 이후 게시글 요청 시 authorizationHeaders() 함수가 저장된 JWT를 Authorization: Bearer 헤더에 포함하여 API 서버로 전송합니다.

(2) API 서버 (Spring Boot 백엔드)
: Spring Boot 서버는 브라우저에서 전달된 HTTP 요청을 처리합니다. 만약 로그인 요청 시 사용자 정보를 확인하고 JWT를 발급하며 게시글 수정 요청 시 AuthService.java의 currentUserId() 함수로 JWT를 검증해 사용자 ID를 추출합니다. 이후 assertOwner() 함수에서 인증된 사용자 ID와 게시글 작성자 ID를 비교하여 수정 권한을 검사합니다. 작성자 본인인 경우에만 수정 요청을 처리하며, 다른 사용자의 게시글을 수정하려할 경우에는 HTTP 403 Forbidden과 JSON 오류 코드인 FORBIDDEN을 반환합니다.

(3) SQLite (데이터베이스)
: SQLite는 사용자 정보와 게시글 데이터를 저장하는 데이터베이스입니다. API 서버는 로그인 요청 시 저장된 사용자 정보를 조회하며 게시글에 대한 요청에 따라 데이터베이스의 게시글 데이터를 처리합니다. 수정 권한 검사를 통과한 요청에 대해서만 게시글 변경 내용을 저장하며, 저장된 게시글 데이터는 브라우저의 로그인 상태와 별개로 유지됩니다.

| 변경한 파일과 함수 | 제공 코드의 원래 역할 | 내가 변경한 내용 | 변경 이유 |
|---|---|---|---|
| AuthService.java / issueToken(), currentUserId() , assertOwner() | JWT 인증과 작성자 권한 검사를 위한 기본 구조 제공 | JWT 처리에 필요한 라이브러리 import 및 생성자 초기화 설정을 추가, JWT 발급 및 검증, 사용자 ID 추출 및 게시글 작성자 ID 비교를 통한 권한 검사 로직을 구현 | 사용자 인증과 타인 게시글 수정 및 삭제 방지 |
| api.js / login(), authorizationHeaders() , canEdit() | API 요청 및 메모리 세션 관리 구조 제공 & 게시글 수정,삭제 버튼 표시 여부를 결정하는 함수 제공 | 로그인 요청, JWT 메모리 저장, Bearer 헤더 생성 기능 구현, 로그인 사용자 ID와 작성자 ID를 비교하여 수정 및 삭제 버튼의 표시 여부를 결정하도록 변경 | 브라우저에서 로그인하고 인증 정보를 서버에 전달하기 위해 & 본인 게시글에만 수정 및 삭제 버튼을 표시하기 위해 |
| App.vue / save(), run() 및 화면 | 게시글 작성 화면 및 로그인 화면 구성, 오류 표시 구조 제공 | 로그인 하지 않은 사용자의 작성 방지, 오류 코드별 메시지 처리, 서비스 안내 문구 변경 | 사용자에게 인증 상태와 오류 원인을 명확하게 안내하기 위해 |

< 변경 사항의 작성 출처 >

1. AI 초안 후 본인 수정

- AuthService.java : issueToken(), currentUserId()

- api.js : authorizationHeaders()

2. 본인 작성 및 수정

- AuthService.java : assertOwner(), JWT 관련 import 및 생성자 설쩡

- api.js : login(), canEdit()

- App.vue : save(), run() 및 화면 구성 변경

3. 참고 자료

- Java 및 JavaScript 문법과 라이브러리 사용 방법은 인터넷 (네이버, 구글) 자료를 참고했으며, 실제 코드 구현과 수정은 직접 수행

## 로컬 실행 기록

1. 운영체제와 버전

- 운영체제 : Windows 11

- 개발 도구 : IntelliJ IDEA

- 백엔드 : Java / Spring Boot

- 데이터베이트 : SQLite

- 프론트엔드 : Vue.js

2. 새 폴더 설치 및 환경설정

(1) 새 폴더 설치 : 제공된 Spring Boot 백엔드 프로젝트와 Vue 프론트엔드 프로젝트를 새 폴더에 준비한 다음 백엔드 프로젝트를 IntelliJ IDEA에서 열어 Java 버전과 실행 환경을 확인했습니다. 프론트엔드는 PowerShell에서 프로젝트 폴더로 이동해 필요한 의존성 패키지를 설치했습니다.

(2) 환경설정 : JWT 인증에 사용할 서버 비밀 키는 README.md에 명시되어있던 명령어로 생성했습니다.

| **실행 명령어 : node -e "console.log(require('node:crypto').randomBytes(32).toString('hex'))"** |
|---|

생성한 64자리, 16진수 키를 IntelliJ 실행 구성의 “JWT_SECRET” 환경변수에 등록했습니다. 또한 제공된 “application.properties”에 “app.jwt.secret=${JWT_SECRET:REPLACE_WITH_RANDOM_64_HEX_CHARACTERS}” 설정을 통해 Spring Boot실행 시 등록한 환경변수를 참조하도록 구성했습니다. 환경변수를 설정한 이후에는 별도의 PowerShell 환경변수 설정 없이 IntelliJ IDEA의 Run 버튼을 통해 백엔드 서버를 실행하도록 구성했습니다.

3. 프론트엔드 설치 및 빌드

: 제공된 Vue 프론트엔드 프로젝트 폴더에서 npm을 이용해 필요한 패키지를 설치하고, 프로젝트 초기 설정을 진행한 후 로컬 개발 서버를 실행했습니다.

| **실행 명령어 & 과정** |
|---|
| **(1) npm ci : 프로젝트에 필요한 의존성 패키지 설치**<br>**(2) npm run build : 프론트엔드 빌드를 수행**<br>**(3) npm run preivew : 빌드된 프론트엔드를 로컬 미리보기 서버로 실행** |

- 실행 결과 : 빌드된 프론트엔드를 로컬 미리보기 서버로 실행해 Local : http://127.0.0.1:4173/ 의 주소가 표시되었습니다. 이 주소를 Microsoft Edge에서 접속해 화면이 정상적으로 표시되는 것을 확인했습니다.

4. 백엔드 빌드 및 실행

:Spring Boot 백엔드 프로젝트를 Maven으로 빌드한 결과 터미널에서 BUILD SUCCESS가 출력되어 빌드가 정상적으로 완료되었음을 확인했습니다. 일반적인 개발 과정에서는 IntelliJ IDEA의 Run 기능으로 백엔드 서버를 실행했으며 별도로 생성된 JAR 파일을 이용한 실행도 확인했습니다.

| **Java 실행 명령어 : java -jar target/jwt-board-1.0.0.jar** |
|---|

실행 결과 Spring Boot 로그에서 Tomcat started on port 8000가 출력된 것을 기반으로 백엔드 서버가 8000번 포트에서 정상적으로 실행되었음을 확인했습니다. JAR 파일을 직접 실행할 때에는 JWT인증에 필요한 JWT_SECRET 환경변수가 해당 실행 프로세스에도 전달되어야 합니다.

5. SQLite 데이터베이스

: 게시글 데이터 저장에는 SQLite를 사용했습니다. application.properties에서 app.db.file=${DB_FILE:data/board.sqlite}로 데이터베이스 경로를 설정했으며, 별도의 DB_FILE 환경변수가 없다면 기본적으로 백엔드 프로젝트의 data/board.sqlite 파일을 사용합니다. 서버 실행 시 DatabaseConfig.java에서 데이터베이스 파일의 상위 폴더를 생성하고 SQLite에 연결하며, schema.sql의 CREATE TABLE IF NOT EXISTS 구문을 통해 게시글 테이블이 존재하지 않으면 자동으로 생성되도록 구성되어 있습니다. 데이터베이스를 초기화하려면 먼저 Spring Boot 서버를 종료한 뒤 data/board.sqlite 파일을 삭제하고 서버를 다시 실행하면 됩니다. 이 경우 기존 게시글 데이터는 삭제됩니다.

6. 4173번 포트 접속 및 API 연결 확인

: 프론트엔드 미리보기 서버를 실행한 후 Microsoft Edge에서 http://127.0.0.1:4173/에 접속했습니다. 이후 개발자 도구의 Network 탭에서 게시글 목록 조회 요청을 통해 API 연결을 확인했습니다.

| **요청 URL : http://127.0.0.1:4173/api/posts**<br>**요청 메서드 : GET**<br>**실제 응답 : HTTP 200 OK** |
|---|

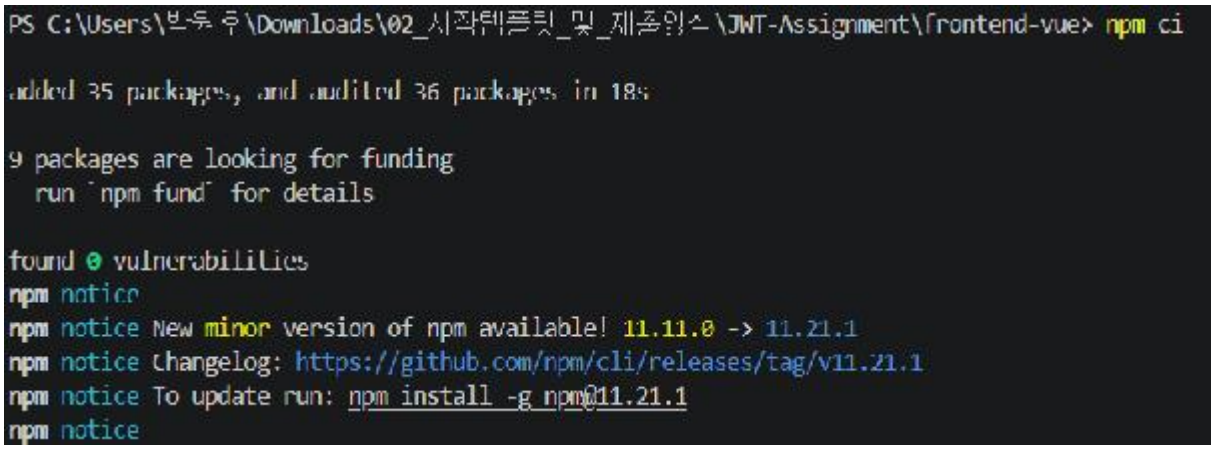
이를 통해 프론트엔드 화면이 정상적으로 실행되는 것과 프론트엔드에서 게시글 조회 API로 요청을 전송해 정상 응답을 받은 것을 확인했습니다.

7. 서버 재시작 후 데이터 유지 확인

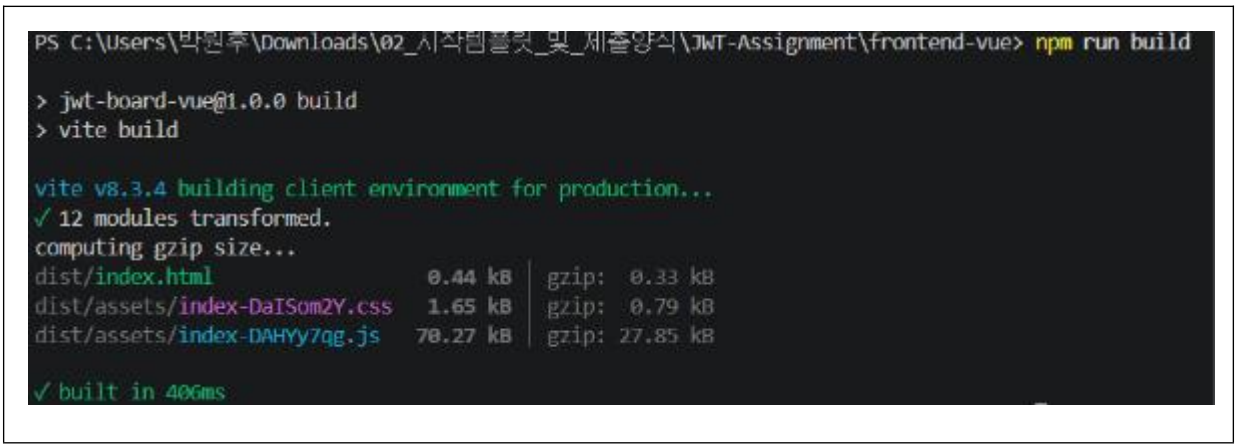
: SQLite 데이터가 유지되는지 확인하기 위해 로그인 후 게시글을 등록하고, 백엔드 서버를 종료한 다음 다시 실행했습니다. 서버 재시작 전에 등록한 게시글의 제목, 내용 및 작성자 정보가 유지되는 것을 확인했습니다. 이를 통해 백엔드 서버를 재시작하더라도 SQLite에 저장된 게시글 데이터가 유지되는 것을 확인할 수 있었습니다.

8. 실행 및 검증 증거 위치

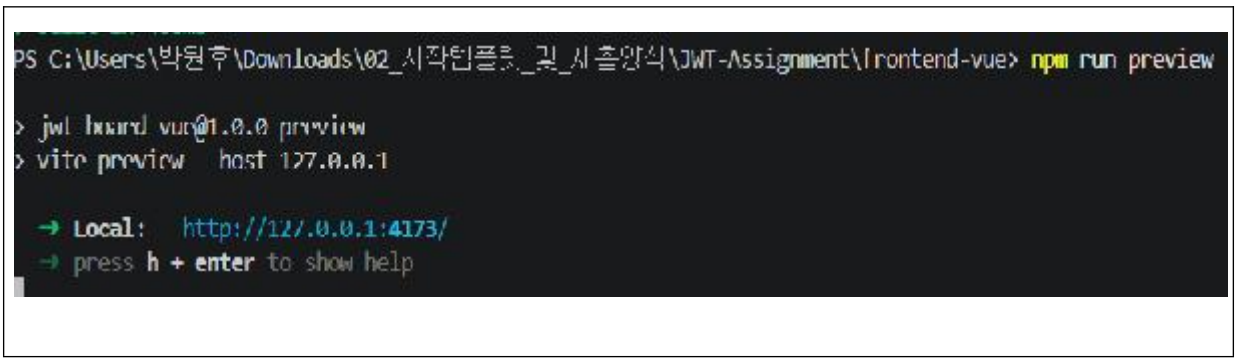
1 ) 프론트엔드 의존성 설치



2) 프론트엔드 빌드




3) 프론트엔드 실행




4) 백엔드 빌드




5) 백엔드 실행




6) 4173번 접속 및 API 연결 : 테스트 증거 D. [4173번 연결] 참조

7) 서버 재시작 후 데이터 유지 : 테스트 증거 D. [T15] 참조

# 핵심 코드 설명

## 3 1 로그인과 JWT 발급

파일 / 함수:

(1) BoardController.java / login() : 로그인 요청을 받아 입력값을 검사하고, 계정 검증 결과에 따라 JWT를 포함한 응답 또는 오류를 반환합니다.

(2) Users.java / verify() : 기존에 등록된 계정의 이메일과 비밀번호를 검증합니다.

(3) AuthService.java / issueToken() : 로그인에 성공한 사용자 정보를 바탕으로 HS256 방식의 JWT를 발급합니다.

(4) api.js / login() : Vue 로그인 화면에서 입력받은 이메일과 비밀번호를 JSON 형식으로 서버에 전송하고, 로그인에 성공하면 응답으로 받은 JWT와 사용자 정보를 메모리에 저장합니다. 로그인 실패할 경우에는 오류를 전달하여 화면에 오류 메시지가 표시되도록 합니다.

(5) App.vue / 로그인 처리 함수 및 화면 : 사용자가 로그인 버튼을 누르면 api.js의 login()함수를 호출합니다. 로그인에 성공하면 사용자 정보를 화면에 표시하고, 실패하면 오류 메시지를 표시하여 로그인 상태로 전환하지 않도록 합니다.

### 코드 발췌

(1) 로그인 검증 & 응답 ( BoardController.java / 제공된 코드 )

```
@PostMapping("/auth/login")
  public Object login(@RequestBody Map<String, Object> body) {
    var input = fields(body, Map.of("email", 254, "password", 128));
    var user = users.verify(input.get("email"), input.get("password"));
    if (user == null) throw new ApiError(401, "INVALID_CREDENTIALS", "이메일 또는 비밀번호를
확인하세요.");
    return Map.of("accessToken", auth.issueToken(user), "expiresIn", ttl, "user", user);
  }
```

=> 해당 코드에서는 fields()가 입력값을 검증하고, users.verify()가 계정과 비밀번호를 확인합니다. user == null 이면 401 오류가 발생하고, 정상 계정이면 JWT 발급 후 응답을 반환합니다.

(2) JWT 발급 ( AuthService.java / BE-01 )

```
public String issueToken(Users.PublicUser user) {
    // Users.PublicUser : 사용자 정보를 담는 자료형
    // user : 그 정보를 저장하는 변수

    Instant now = Instant.now(); // 현재 시각 가져오기
    Instant expiresAt = now.plusSeconds(ttl); // JWT 만료 시각 계산하기
    // 현재 시각에 TTL만큼 초를 더해 만료 시각을 계산하는 코드

    return Jwts.builder().subject(String.valueOf(user.id())).issuer(issuer)
            .audience().add(audience).and().issuedAt(Date.from(now))
            .expiration(Date.from(expiresAt))
            .signWith(key, Jwts.SIG.HS256).compact();
```

```
}
```

=> 해당 코드에서는 사용자 ID와 필수 클레임을 설정하고, TTL에 따라 만료 시각을 계산한 뒤 HS256으로 서명한 JWT 문자열을 반환합니다.

(3) 프론트엔드 로그인 요청 및 메모리 저장 ( api.js / FE-01 )

```
export async function login(email, password) {
  // 로그인 요청 보내기
  const data = await request("/auth/login", {method: "POST", body:JSON.stringify
    ({email : email, password : password,}),});
    // request("/auth/login",~) : 기존 request() 함수가 /api를 붙여서 /api/auth/login으로 요청
    // method : "POST" 서버에 로그인 정보를 전송
    // JSON.stringify(~) : 이메일과 비밀번호를 JSON 문자열로 변환
    // await : 서버가 응답할 때까지 기다림
    // data : 서버에서 반환한 로그인 결과를 저장

  //로그인 정보 저장하기
    session = {
      accessToken : data.accessToken, user: data.user,
    };
    return data.user;
}
```

=> 해당 코드에서는 로그인 화면에서 전달받은 이메일과 비밀번호를 JSON 형식으로 변환하여 /api/auth/login에 POST 요청을 보냅니다. 로그인에 성공하면 응답으로 받은 JWT와 사용자 정보를 JavaScript 메모리의 session 변수에 저장하고, 사용자 정보를 반환합니다. localStorage와 sessionStorage에는 저장하지 않습니다.

### 동작과 응답 코드 설명

(1) 입력 및 계정 검증
: 사용자가 Vue 로그인 화면에서 이메일과 비밀번호를 입력하면 api.js의 login() 함수가 해당 정보를 JSON 형식으로 변환하여 /api/auth/login에 POST 요청을 전송합니다. BoardController.java에 있는 login() 함수는 fields()를 호출하여 입력 형식을 검사하고, Users.java의 verify() 함수로 등록된 계정과 비밀번호가 서로 일치하는지 확인합니다.

(2) JWT 발급 처리
: 입력값 검증과 계정 인증에 성공하면 구현한 AuthService.java의 issueToken() 함수가 호출됩니다. issueToken() 함수는 사용자 ID를 문자열로 변환하여 JWT의 sub에 저장하고, 발급자(iss)와 사용 대상(aud)을 설정합니다. 또한 현재 시각을 발급 시각(iat)으로 기록하고, 현재 시각에 설정된 TTL을 더하여 만료 시각(exp)을 계산합니다. 서버 비밀 키를 사용해 HS256 방식으로 JWT에 서명하하고, 비밀번호와 비밀번호 해시는 JWT에 포함하지 않습니다.

(3) 정상 응답 및 성공 코드
: 로그인에 성공하면 login() 함수의 return Map.of()에서 발급된 JWT인 accessToken, 유효 시간인 expiresIn, 사용자 정보인 user를 반환합니다. user에는 id, name, email만 포함되며, 정상 응답에서는 별도의 JSON 오류 코드가 없습니다. 프론트엔드의 api.js는 응답으로 받은 accessToken과 user를 JavaScript 메모리의 session 변수에 저장하고, App.vue에는 로그인한 사용자 이름을 화면에 표시합니다. JWT는 Local Storage와 Session Storage에는 저장하지 않으므로 새로고침하면 로그인 상태가 초기화됩니다.

(4) 오류 응답 및 실패 조건
: 이메일 혹은 비밀번호의 입력형식이 잘못된 경우에는 fields()의 입력 검증 과정에서 400 Bad Request, JSON 오류 코드 "BAD_INPUT"이 반환됩니다. 등록되지 않은 이메일이거나 비밀번호가 일치하지 않으면 verify()가 null 값을 반환합니다. 이때 login()의 if(user == null) 조건문에서 ApiError(401, "INVALID_CREDENTIALS", "이메일 또는 비밀번호를 확인하세요.");가 발생하여 401 Unauthorized과 JSON

오류 코드는 "INVALID_CREDENTIALS"를 반환합니다. 해당 경우에는 JWT는 발급되지 않습니다.
프론트엔드는 서버에서 전달받은 오류 메시지를 로그인 화면에 표시하며, 로그인 성공 상태로 전환하지 않습니다. 실제 테스트 해 본 결과 잘못된 비밀번호 입력 시 "이메일 또는 비밀번호를 확인하세요"의 오류 메시지가 표시되는 것을 확인했습니다.

## 3 2 브라우저 토큰 저장과 전송

파일 / 함수:

(1) BoardController.java / login() : 로그인에 성공한 사용자에게 JWT와 사용자 정보를 반환합니다.

(2) api.js/login() : 로그인 성공 응답에서 JWT와 사용자 정보를 받아 JavaScript 메모리의 session 변수에 저장합니다.

(3) api.js / authorizationHeaders() : 메모리에 저장된 JWT가 존재하면 Authorization : Bearer 형식의 헤더를 생성합니다.

(4) api.js / request() : authorizationHeaders()가 반환한 인증 헤더를 HTTP 요청에 포함하여 서버로 전송합니다.

(5) api.js / clearSession() : 메모리에 저장된 JWT와 사용자 정보를 초기화하는 함수입니다.

(6) App.vue / logout() : 로그아웃 버튼 클릭 시 clearSession()을 호출하여 로그인 정보를 삭제하고, Vue 화면의 사용자 정보와 안내 메시지를 갱신합니다.

### 코드 발췌

(1) Authorization 헤더 생성 (api.js / FE-02)

```
function authorizationHeaders() {
  // TODO FE-02: 메모리에 보관한 토큰을 Bearer 헤더로 보내세요.
  if(!session.accessToken){
      return {};
  }
  return { Authorization : `Bearer ${session.accessToken}`};

}
```

=> 해당 코드는 JavaScript 메모리의 session.accessToken에 JWT가 저장되어 있는지 확인합니다. 토큰이 존재하면 Authorization: Bearer 형식의 인증 헤더를 생성하고, 토큰이 없으면 빈 객체를 반환하여 Bearer undefined와 같은 잘못된 헤더가 생성되지 않도록 구현했습니다.

(2) HTTP 요청에 인증 헤더 포함 (api.js / request() , 기존 제공된 코드 )

```
export async function request(path, options = {}) {
  const response = await fetch(`/api${path}`, {
    ...options,
    headers: {
      "Content-Type": "application/json",
      ...authorizationHeaders(),
      ...options.headers,
    },
  });
```

=> 해당 코드는 authorizationHeaders()가 반환한 인증 헤더를 HTTP 요청에 포함하여 서버로 전송하므로 로그인 후 게시글 등록, 수정, 삭제 및 내 정보 조회 요청에 JWT가 전달됩니다.

(3) 로그인 정보 초기화 (api.js / clearSession(), 기존 제공된 코드 )

```
export function clearSession() {
  session = { accessToken: "", user: null };
}
```

=> 해당 코드는 JavaScript 메모리에 저장된 JWT를 빈 문자열로 변경하고 사용자 정보를 null로 초기화 합니다. 로그아웃할 때 호출되며, 이후에는 인증 헤더에 JWT가 포함되지 않습니다.

(4) 로그아웃 및 화면 로그인 상태 초기화 ( App.vue / logout(), 기존 코드 활용 FE-04 )

```
function logout() {
  clearSession();
  user.value = null;
  message.value = "이 탭의 로그인 상태를 해제했습니다.";
}
```

=> 해당 코드는 사용자가 로그아웃 버튼을 클릭했을 때 실행됩니다. 먼저 clearSession()을 호출해 JavaScript 메모리에 저장된 JWT와 사용자 정보를 삭제합니다. 그 다음에 user.value = null 을 통해 Vue 화면의 로그인 사용자 정보를 초기화 및 message.value 를 변경하여 로그아웃 안내 메시지를 표시하도록 합니다. 이를 통해 로그아웃 버튼을 클릭하면 화면에 표시되는 로그인 상태도 즉시 해제 되도록 구성되어있습니다.

### 동작과 응답 코드 설명

(1) 로그인 응답 및 토큰 저장

: 사용자가 Vue 로그인 화면에서 이메일과 비밀번호를 입력하면 서버의 /api/auth/login으로 POST 요청을 전송합니다. 로그인에 성공하면 HTTP 200 OK와 함께 accessToken, expiresln, user를 JSON 형식으로 반환합니다. 정상 응답에는 별도의 JSON 오류 코드가 없습니다. 프론트엔드의 api.js에서는 반환된 accessToken과 user를 JavaScript 메모리의 session 변수에 저장합니다. JWT는 Local Storage와 Session Storage에 저장하지 않으므로 페이지를 새로고침하면 메모리에 저장된 로그인 정보가 초기화됩니다.

(2) Authorization 헤더 생성 및 전송

: 로그인 이후에 인증이 필요한 API 요청이 발생하면 FE-02에서 구현한 authorizationHeaders() 함수가 실행됩니다. 해당 함수는 session.accessToken에 JWT가 존재하는지 확인하며, 토큰이 있는 경우에는 Authorization: Bearer 형식의 헤더를 생성합니다. 생성된 헤더는 기존에 제공된 코드인 request() 함수에서 HTTP 요청에 포함되어 서버로 전송됩니다. 반대로 토큰이 없는 경우에는 빈 객체를 반환하여 Bearer undefined와 같은 잘못된 인증 헤더가 생성되지 않도록 구현했습니다. 또한 기존에 제공된 코드에서는 게시글 목록과 상세 조회를 공개 API로 구성하여 로그인을 하지 않은 상태에서도 조회할 수 있도록 구성되었습니다.

(3) 정상 응답 및 성공 코드

: 정상적인 JWT를 포함하여 GET/api/me를 요청하면 서버에서 인증된 사용자의 정보를 반환하며, HTTP 상태 코드는 200 OK입니다. 정상 응답에는 별도의 JSON 오류 코드가 없습니다. 로그인 상태에서 게시글 등록, 수정, 삭제 요청에도 동일한 Authorization 헤더가 포함되도록 구현했습니다. 게시글 등록에 성공하면 HTTP 201 Created를, 수정에 성공하면 HTTP 200 OK, 삭제에 성공하면 HTTP 204 No Content를 반환합니다.

(4) 오류 응답 및 실패 조건

: 인증 헤더가 없거나 Bearer 토큰이 전달되지 않은 경우 HTTP 401 Unauthorized와 JSON 오류 코드 TOKEN_REQUIRED 가 반환됩니다. JWT의 서명이나 필수 클레임이 올바르지 않은 경우에는 HTTP 401 Unauthorized와 JSON 오류 코드 INVALID_TOKEN이 반환됩니다. JWT의 유효 시간이 만료된 경우에는 HTTP 401 Unauthorized와 JSON 오류 코드 TOKEN_EXPIRED가 반환됩니다. 이와 같이 인증 실패에 대한 서버 검증 및 프론트엔드 오류 처리는 " 서버 JWT 검증 " 에서 추가적으로 설명했습니다.

(5) 토큰 초기화 및 로그아웃 (FE-04)

: 기존에 제공된 코드인 api.js의 clearSession() 함수는 JavaScript a메모리의 accessToken을 빈 문자열로 변경하고 user를 null 값으로 초기화하도록 구성되었습니다. 이 함수가 실행되면 이후 authorizationHeaders()는 빈 객체를 반환하므로 Authorization 헤더에 JWT가 포함되지 않습니다. 사용자가 로그아웃 버튼을 클릭하면 App.vue의 logout() 함수가 실행되면서 clearSession()을 호출합니다. 그 다음에 user.value = null을 통해 Vue 화면의 로그인 사용자 정보를 초기화하며 "이 탭의 로그인 상태를 해제했습니다."라는 안내 메시지를 표시하도록 합니다.

**웹서비스설계 03 계속**

# 서버 인증과 작성자 권한

## 3 3 서버 JWT 검증

파일 / 함수:

(2) AuthsService.java / currentUserId() : HTTP 요청의 Authorization 헤더에서 Bearer 토큰을 추출하고, JWT의 서명과 만료 여부 및 필수 클레임을 검증한 뒤 인증된 사용자 ID를 반환합니다.

(2) api.js / request() : 서버에서 반환된 HTTP 상태 코드를 확인하고 HTTP 401 인증 오류가 발생하면 메모리에 저장된 JWT와 사용자 정보를 초기화합니다. 또한 HTTP 상태 코드와 JSON 오류 코드를 App.vue로 전달합니다.

(3) App.vue / run() : 서버에서 반환한 HTTP 상태 코드와 JSON 오류 코드에 따라 오류 메시지를 구분하여 표시하고, 인증 실패 시 화면의 로그인 상태를 갱신합니다.

### 코드 발췌

(1) JWT 추출 및 검증 (AuthService.java / BE-02 )

```
String header = request.getHeader("Authorization");
// 토큰 유무 확인
if(header ==null || header.isBlank()){
  throw new ApiError(401, "TOKEN_REQUIRED","토큰이 필요합니다.");
}
// Baerer 형식 확인
// regionMatches : 문자열의 처음 7글자가 Bearer인지 비교하는 메서드
if(!header.regionMatches(true,0,"Bearer ",0,7) || header.substring(7).isBlank()){
  throw new ApiError(401, "TOKEN_REQUIRED", "Bearer 토큰을 보내세요.");
}
// JWT만 추출
String token = header.substring(7).trim(); //trim() : 문자열 양끝 공백 제거
try{
  var jwt = Jwts.parser().verifyWith(key).build().parseSignedClaims(token);
  // parser() : JWT를 읽고 검증하기 위한 parser 생성
  // verifyWith(key) : BE-01에서 사용한 비밀 키 -> 검증 키로 지정
  // build() : 설정된 JWT parser 생성
  // parseSignedClaims(token) : JWT 서명을 검증 & 클레임을 읽기, 만료된 토큰 등은 예외 발생

  // Claims : JWT 내부에 저장된 클레임 정보를 담는 자료형
  // getPayload() : 검증된 JWT에 payload를 가져오는 메서드
  Claims claims = jwt.getPayload();
```

=> 해당 코드는 HTTP 요청의 Authorization 헤더에서 Bearer 토큰을 추출하고, 서버의 비밀 키를 이용해 JWT의 서명과 만료 여부를 검증합니다. 먼저 인증 헤더가 없거나 Bearer 형식이 올바르지 않으면 HTTP

401과 JSON 오류 코드 TOKEN_REQUIRED를 발생시킵니다. 이후 parseSignedClaims()를 이용해 JWT의 서명을 검증하고 Claims 객체를 통해 토큰에 포함된 정보를 가져옵니다. 이 과정에서 만료된 토큰이나 유효하지 않은 토큰은 예외가 발생합니다.

(2) JWT 알고리즘 및 필수 클레임 검사 ( AuthService.java -> current UserId() / BE-02)

```
    if (!"HS256".equals(jwt.getHeader().getAlgorithm())
            ||!issuer.equals(claims.getIssuer())
            || claims.getAudience() == null
            || !claims.getAudience().contains(audience)
            || claims.getIssuedAt() == null || claims.getExpiration() == null
            || claims.getSubject()  == null
            || !claims.getSubject().matches("[1-9][0-9]*")) // 사용자 ID가 0이 아닌 양의 정수 형태인지
검사하는 정규표현식
    {
      throw new ApiError(401,"INVALID_TOKEN", "토큰 정보가 올바르지 않습니다.");
    }
```

=> 해당 코드는 JWT의 알고리즘이 HS256인지 확인하고, 발급자(iss), 사용 대상(aud), 발급 시각(iat), 만료 시각(exp), 사용자 ID(sub) 등의 필수 정보를 검사합니다. sub은 0이 아닌 양의 정수 형태인지 검사하는 정규표현식으로 확인합니다. 필수 클레임이 누락되거나 값이 올바르지 않으면 HTTP 401 Unauthorized와 JSON 오류 코드 INVALID_TOKEN을 반환하도록 구현했습니다.

(3) 인증 오류 처리 ( api.js -> request() / FE-03 )

```
  const data = response.status === 204 ? null : await response.json();
   if (!response.ok) {
     if (response.status === 401 ){ // 401 -> 메모리의 로그인 정보를 초기화
       clearSession();
     }
   // TODO FE-03: 401이면 메모리의 로그인 상태도 해제하세요.
   const error = new Error(
     data.message || data.code || `HTTP ${response.status}`,
   );
   error.status = response.status;
   error.code = data.code;
   throw error;
 }
```

=> 해당 코드는 서버의 HTTP 응답이 정상인지 확인하고, 오류가 발생한 경우 HTTP 상태 코드에 따라 처리하도록 구현했습니다. 특히 HTTP 401 Unauthorized가 반환되면 clearSession()을 호출하여 JavaScript 메모리에 저장된 JWT와 사용자 정보를 초기화합니다. 이후 서버가 반환한 HTTP 상태 코드와 JSON 오류 코드를 오류 객체에 저장하여 App.vue로 전달합니다. 정상 응답인 경우에는 서버에서 반환된 데이터를 그대로 반환합니다.

(4) HTTP 응답 코드별 오류 처리 ( App.vue -> run() / FE-03)

```
catch (e) {
   if (e.status === 401) {
     if (e.code === "INVALID_CREDENTIALS") {
       message.value = "이메일 또는 비밀번호가 올바르지 않습니다.";
     } else if (e.code === "TOKEN_REQUIRED") {
       message.value = "로그인이 필요합니다.";
     } else if (e.code === "INVALID_TOKEN") {
       message.value = "유효하지 않은 토큰입니다. 다시 로그인하세요.";
     } else if (e.code === "TOKEN_EXPIRED") {
       message.value = "로그인 시간이 만료되었습니다. 다시 로그인하세요.";
```

```
    } else {
      message.value = "인증에 실패했습니다. 다시 로그인하세요.";
    }
  } else if (e.status === 400) {
    message.value = e.message || "입력값을 확인하세요.";
  } else if (e.status === 403) {
    message.value = "해당 게시글을 수정하거나 삭제할 권한이 없습니다.";
  } else if (e.status === 404) {
    message.value = "게시글을 찾을 수 없습니다.";
    selected.value = null;
    await refresh().catch(() => {});
  } else {
    message.value = e.message || "요청 중 오류가 발생했습니다.";
  }
```

=> 해당 코드는 api.js에서 전달받은 HTTP 상태 코드와 JSON 오류 코드에 따라 서로 다른 오류 메시지를 표시하도록 구현했습니다. HTTP 401에서는 INVALID_CREDENTIALS, TOKEN_REQUIRED, INVALID_TOKEN, TOKEN_EXPIRED를 구분하여 안내하고, HTTP 400에서는 입력 오류 메시지를 표시합니다. HTTP 403에서는 권한 부족 메시지를 표시하지만 로그인 상태를 유지하며, HTTP 404에서는 선택된 게시글을 초기화하고 목록을 다시 조회합니다. 또한 run() 함수의 finally에서 메모리에 저장된 사용자 정보를 화면에 반영하여 인증 오류 발생 시 '로그아웃 상태'로 변경되도록 합니다.

### 동작과 응답 코드 설명

(1) Authorization 헤더 및 JWT 추출
: 사용자가 로그인한 상태에서 인증이 필요한 API를 요청하면 프론트엔드는 Authorization 헤더에 Bearer 토큰을 포함하여 서버로 전송합니다. 서버의 AuthService.java에 구현된 currentUserId() 함수는 request.getHeader("Authorization")를 통해 인증 헤더를 읽습니다. 헤더가 없거나 비어 있으면 HTTP 401 Unauthorized와 JSON 오류 코드 TOKEN_REQUIRED를 반환합니다. 이후 Bearer 형식인지 확인하고 substring(7).trim()을 이용해 JWT 문자열만 추출하도록 합니다.

(2) JWT 서명, 알고리즘, 만료 및 필수 클레임 검증
: 추출한 JWT는 JJWT 라이브러리의 Jwts.parser().verifyWith(key).build().parseSignedClaims(token)을 이용하여 서명을 검증합니다. 이 과정에서 서버가 지정한 검증 키와 토큰의 서명이 일치하는지 확인하며, 만료된 토큰은 예외로 처리됩니다. 그 다음에 JWT 헤더의 알고리즘이 HS256인지 확인하고, Claims 객체를 통해 발급자(iss), 사용 대상(aud), 발급 시각(iat), 만료 시각(exp), 사용자 ID(sub)를 검사합니다. 또한 Claims.getIssuedAt().after(new Date())를 통해 JWT의 발급 시각이 현재 시각보다 미래인지 확인합니다. 서명이나 필수 클레임이 올바르지 않은 경우에는 인증 오류로 처리합니다.

(3) 사용자 ID 추출 및 전달
: JWT의 서명과 필수 클레임 검증을 통과하면 sub에 저장된 사용자 ID를 Long.parseLong()으로 숫자로 변환합니다. 이후 users.find(userId)를 통해 해당 사용자가 실제로 등록되어 있는지 확인합니다. 모든 검증을 통과하면 인증된 사용자 ID를 반환하며, 해당 ID는 내 정보 조회와 게시글 등록, 수정, 삭제 등 인증이 필요한 기능에 사용됩니다. 이를 통해 클라이언트가 임의로 지정한 사용자 ID가 아니라 서버에서 검증한 사용자 ID를 사용하도록 합니다.

(4) 정상 응답 및 성공 코드
: 정상적인 JWT로 GET /api/me를 요청하면 서버는 인증된 사용자 정보를 반환하며 HTTP 상태 코드는 200 OK입니다. 게시글 등록에 성공하면 HTTP 201 Created, 본인 게시글 수정에 성공하면 HTTP 200 OK, 삭제에 성공하면 HTTP 204 No Content를 반환합니다. 정상 응답에는 별도의 JSON 오류 코드가 없습니다.

(5) 인증 실패 조건 및 오류 응답
: 인증 헤더가 없거나 Bearer 토큰이 전달되지 않은 경우 HTTP 401 Unauthorized와 JSON 오류 코드 TOKEN_REQUIRED가 반환됩니다. JWT의 서명이나 필수 클레임이 올바르지 않은 경우에는 HTTP 401과

INVALID_TOKEN이 반환됩니다. JWT의 유효 시간이 만료된 경우에는 HTTP 401과 TOKEN_EXPIRED가 반환됩니다. 만료된 JWT로 인해 ExpiredJwtException이 발생하면 TOKEN_EXPIRED로 처리하고, 그 외에 JWT 검증 과정에서 JwtException 또는 IllegalArgumentException이 발생했을 경우에는 INVALID_TOKEN으로 처리하도록 구현했습니다. 또한 로그인 과정에서 이메일 또는 비밀번호가 올바르지 않으면 HTTP 401과 INVALID_CREDENTIALS가 반환됩니다.

(6) 프론트엔드의 HTTP 오류 처리
:서버에서 HTTP 401 응답을 반환하면 api.js의 request() 함수는 clearSession()을 호출하여 메모리에 저장된 JWT와 사용자 정보를 초기화합니다. 그 다음에 HTTP 상태 코드와 JSON 오류 코드를 App.vue의 run() 함수에 전달합니다. run() 함수에서는 오류 코드에 따라 로그인 실패, 토큰 누락, 유효하지 않은 토큰, 토큰 만료 등의 안내 메시지를 구분하여 표시합니다. 인증 실패 시 화면의 로그인 상태도 초기화됩니다. 반면에 HTTP 400은 잘못된 입력에 대한 오류 메시지를 표시하며, HTTP 403은 권한 부족을 안내하되 로그인 상태를 유지합니다. HTTP 404는 존재하지 않는 게시글에 대한 오류를 표시하고 선택된 게시글을 초기화한 뒤 목록을 갱신하도록 구현했습니다.

## 3 4 작성자 권한 검사

파일 / 함수:

(1) AuthService.java / assertOwner() : 현재 로그인한 사용자 ID와 게시글 작성자 ID를 비교하여 수정 및 삭제 권한을 검사합니다.

(2) BoardController.java / create() : JWT 검증을 통해 확인된 사용자 ID를 게시글 작성자로 저장합니다.

(3) BoardController.java / update(),delete() : 게시글 수정 및 삭제 전에 작성자 권한 검사를 수행합니다.

(4) PostStore.java / uupdate() : 게시글 수정 시 기존 작성자 ID와 작성 시각을 유지하고 제목, 내용 및 수정 시각을 변경합니다.

(5) api.js / canEdit() : 로그인 사용자 ID와 게시글 작성자 ID를 비교하여 수정 및 삭제 버튼의 표시 여부를 결정합니다.

(6) App.vue / save() : 로그인을 하지 않은 사용자가 게시글 등록을 시도하게 되면 로그인 안내 메시지를 표시하고 요청을 중단합니다.

### 코드 발췌

(1) 작성자 권한 검사 ( AuthService. java / assertOwner() - BE-03)

```
  public void assertOwner(long userId, long authorId) {
    if (userId != authorId){
      throw new ApiError(403, "FORBIDDEN","본인이 작성한 게시글만 수정하거나 삭제할 수
있습니다.");
    }
  }
```

=> 해당 코드는 현재 사용자 ID와 게시글 작성자 ID를 비교하며 두 값이 다르면 ApiError를 통해 HTTP 403과 FORBIDDEN 오류를 발생시키도록 구현했습니다.

(2) 게시글 등록 시 작성자 ID 지정 (BoardController.java / create() - BE-03)

```
long user =  auth.currentUserId(req);
var p = store.create(input.get(”title”), input. get(”content”),user);
```

=> 해당 코드는 JWT 검증을 통해 확인한 사용자 ID를 store.create()에 전달하여 게시글 작성자로 저장하도록 구현했습니다.

(3) 게시글 수정 및 삭제 (BoardController.java)

```
var p = existing(pid);
auth.assertOwner(user,p.authorId());
```
=> 해당 코드는 저장된 게시글의 작성자 ID와 현재 사용자 ID를 비교한 후에 권한이 확인된 경우에만 수정 및 삭제 작업을 진행하도록 구현했습니다.

(4) 로그인 사용자와 게시글 작성자 비교 (api.js / canEdit() - FE-04)
```
export function canEdit(post) {
  if (!session.user || !post){
    return false;
  }
  return String(session.user.id) === String(post.authorId); // 시작 코드에서는 CRUD를 체험할 수 있도록 모두 표시합니다.
}
```
=> 로그인 정보 혹은 게시글 정보가 없을 경우에 false를 반환합니다. 두 정보가 존재하면 String()으로 ID의 자료형을 통일한 뒤 비교하도록 구현했습니다.

(5) 로그인하지 않은 사용자 게시글 작성 방지 (App.vue / save() - FE-04)
```
function logout() {
  clearSession();
  user.value = null;
  message.value = "이 탭의 로그인 상태를 해제했습니다.";
}
```
=> 기존에 제공된 save()함수에서 로그인 여부를 확인하는 조건문을 추가했습니다. 로그인 정보가 없으면 안내 메시지를 표시하고 return으로 게시글 등록 요청을 중단하도록 합니다.

(6) 최종 서비스 안내 문구 변경 (App.vue / FE-04)
```
<header>
      <span class="eyebrow">CAMPUS LOST & FOUND</span>
      <h1>다 찾아드립니다!</h1>
      <p class="muted">
        캠퍼스에서 잃어버리거나 발견한 물건의 정보를 함께 공유하는 대학생 분실물 게시판입니다.
      </p>
    </header>
    <p class="banner">
      분실물 제보는 누구나 조회 가능!<br />
        로그인 후 잃어버리거나 발견한 물건의 특징과 위치를 등록해 주세요.<br />
        제보 수정 및 삭제는 작성자 본인만 가능합니다.
    </p>
```
=> 기존에 인증 미완성 데모 배너를 "다찾아드립니다" 분실물 공유 서비스의 이용 안내 문구로 변경했습니다.

## 동작과 응답 코드 설명

(1) 작성자 ID 비교 및 권한 검사
: 게시글을 수정하거나 삭제할 때는 JWT 검증을 통해 확인된 현재 로그인 사용자 ID와 데이터베이스에 저장된 게시글 작성자 ID를 비교합니다. BE-03에서 구현한 assertOwner() 함수는 두 ID가 일치하면 작업을 허용하고, 일치하지 않으면 HTTP 403 Forbidden과 JSON 오류 코드 FORBIDDEN을 반환하도록 구현했습니다. 만약 권한 검사에 실패하면 게시글 수정 및 삭제가 진행되지 않습니다. 프론트엔드에서는 FE-04로 구현한 canEdit() 함수를 통해 로그인 사용자 ID와 게시글 작성자 ID를 비교합니다. 기존 App.vue의

v-if 조건과 연결되어 작성자 본인에게만 수정, 삭제 버튼이 표시되도록 구성되어 있습니다. 다만 실제 권한은 서버에서 다시 검사합니다.

(2) 작성자 위조 방지
: 게시글 등록 시에는 auth.currentUserId(req)를 통해 JWT에서 검증된 사용자 ID를 가져와 store.create()에 전달하도록 구현했습니다. 클라이언트가 임의로 지정한 작성자 ID를 사용하지 않으므로 다른 사용자의 ID로 게시글을 등록하는 작성자 위조를 방지할 수 있습니다.

(3) 정상 응답 및 성공 코드
: 정상적으로 인증된 사용자가 게시글을 등록하면 HTTP 201 Created, 본인의 게시글을 수정하면 HTTP 200 OK, 삭제하면 HTTP 204 No Content를 반환합니다. 정상 응답일 경우에는 별도의 JSON 오류 코드가 없습니다.

(4) 오류 응답 및 실패 조건
: HTTP 401 Unauthorized는 JWT가 없거나 유효하지 않아 사용자를 인증할 수 없는 경우입니다. 토큰이 없으면 TOKEN_REQUIRED, 유효하지 않으면 INVALID_TOKEN, 만료되면 TOKEN_EXPIRED가 반환됩니다. 반면 HTTP 403 Forbidden은 JWT 인증에는 성공했지만 다른 사용자의 게시글을 수정하거나 삭제하려는 경우이며, JSON 오류 코드 FORBIDDEN을 반환하도록 구현했습니다.

# 테스트 결과

실행 환경: 운영체제 - Windows 11 / 프론트엔드 - Vue3 / 백엔드 - Spring Boot / Java - Open JDK 21.0.12.1 / Node.js 24.14.1 / 데이터 베이스 - SQLite / 브라우저 - Microsoft Edge

로컬 배포 확인:

1. 새 폴더에서 설치 및 실행

(1) Spring Boot 백엔드 프로젝트를 IntelliJ IDEA에서 열고 Java 버전 및 실행 환경을 확인했습니다.

(2) IntelliJ IDEA의 Run Configuration에 JWT_SECRET 환경변수를 등록하고, Run 버튼을 통해 Spring Boot 백엔드 서버를 실행했습니다.

(3) 별도의 터미널에서 Vue 프론트엔드 프로젝트의 의존성 설치

node -v

npm ci

2. 프론트엔드 빌드 결과

: Vue 프로젝트에서 명령어를 실행해 배포용 파일 생성

```
npm run build
npm run preview
```

=> 로컬 배포 주소 : http://127.0.0.1:4173

3. 백엔드 실행 및 HTTP 응답 확인

(1) 백엔드 실행 결과

: 실행 콘솔에서 [ Spring Boot 버전 : 3.5.16 / Java 버전 : 21.0.12.1

/ 웹 서버 : Apache Tomcat 10.1.55 / 서버 포트 : 8000 / 애플리케이션 실행 시간 : 5.543초 ] 가 나온 것을 기반으로 애플리케이션이 정상적으로 시작 되는 것을 확인

(2) HTTP 응답 확인

: 백엔드 서버의 실행 상태를 확인하기 위해 POSTMAN을 이용해 /api/health API에 GET 요청 전송하여 HTTP 상태 코드 및 응답 데이터를 확인

- 요청 방식 : GET / 요청 URL : http://127.0.0.1:8000/api/health

- HTTP 상태 코드 : 200 OK

- 응답 본문

```
{
    "status": "ok",
    "stack": "spring"
}
```

4. 4173번 접속 및 API 연결 증거

: 브라우저에서 http://127.0.0.1:4173에 접속하여 게시판 화면이 표시되는지 확인, 이후 게시글 목록 조회 등의 기능을 실행하고, 개발자 도구의 Network 탭에서 /api 요청이 Spring Boot 서버로 정상 전달되는지 확인

[새 폴더에서 설치한 절차 / 프론트 빌드 결과 / 백엔드 실행 명령 / 4173번 접속 및 API 연결 증거]

테스트 일시 및 코드 버전: 2026.10.27 / main 브랜치 / 커밋 ID(SHA) : 3c0a84337d57c5c14f73bb86a15243ea6edd001a

토큰 TTL: 일반 테스트 90초(기본 값) / 만료 테스트 90초(기본값, 설정 변경 없음)

| ID | 수행 절차와 사용자 | 예상 결과 | 실제 HTTP와 code | 판정 | 증거 |
|---|---|---|---|---|---|
| T01 | 비로그인 사용자, 게시글 목록 및 상세 조회 | 목록 조회 HTTP 200, 상세 조회 HTTP 200 | 목록 조회 : HTTP 200, 해당없음 (응답 본문 : []) / 상세 조회 : HTTP 200 | PASS | 테스트 증거 E. [ T01 - 1 ] & [ T01 - 2 ] |
| T02 | Alice 계정으로 로그인 | 로그인 HTTP 200, JWT 및 공개 사용자 정보 반환, 화면 로그인 상태 표시 | 로그인 : HTTP 200, 해당 없음 / 화면 : Alice님 로그인 중 (JWT 및 사용자 정보 확인 전) | PASS | 테스트 증거 A. [T02] |
| T03 | 잘못된 비밀번호 및 존재하지 않는 이메일로 각각 로그인 | 두 경우 모두 HTTP 401, INVALID_CREDENTIALS, JWT 미발급 | 잘못된 비밀번호 : HTTP 401, INVALID_CREDENTIALS / 없는 이메일 : HTTP 401, INVALID_CREDENTIALS (JWT 미발급, 로그아웃 상태 유지) | PASS | 테스트 증거 E. [ T03 - 1 ] & [ T03 - 2 ] |
| T04 | 토큰 없이 /me와 글 등록 요청 | 내 정보 조회 및 게시글 등록 요청 모두 401 / TOKEN_REQUIRED | 내 정보 조회 : HTTP 401, TOKEN_REQUIRED / 게시글 등록 : HTTP 401, TOKEN_REQUIRED | PASS | 테스트 증거 E. [ T04 - 1 ] & [ T04 - 2 ] |
| T05 | Alice 토큰으로 글 작성 후 /me 조회 | 게시글 작성 HTTP 201 및 authorId 1, 내 정보 조회 HTTP 200 및 Alice 정보 반환 | 게시글 작성 : HTTP 201 , 해당 없음 (authorId 1 확인) / 내 정보 조회 : HTTP 200, 해당 없음 | PASS | 테스트 증거 A. [ T05 - 1 ] & [ T05 - 2 ] |

| ID | 수행 절차와 사용자 | 예상 결과 | 실제 HTTP와 code | 판정 | 증거 |
|---|---|---|---|---|---|
| T06 | Alice 토큰으로 본인 글 수정,삭제 | 수정 HTTP 200, 삭제 HTTP 204, 수정 및 삭제 결과 DB 반영 | 본인 글 수정 : HTTP 200, 해당 없음 / 삭제 : HTTP 204, 해당 없음 | PASS | 테스트 증거 E. [ T06 - 1 ] & [ T06 - 2 ] |
| T07 | Alice 글을 Bob의 정상 토큰으로 수정 | 수정 요청 HTTP 403 / FORBIDDEN, 기존 게시글 원문 유지 | 타인 게시글의 수정 버튼 미표시 / Bob의 정상 JWT로 Alice 게시글 수정 요청 : HTTP 403 , FORBIDDEN | PASS | 테스트 증거 B. [T07] |
| T08 | Alice 글을 Bob의 정상 토큰으로 삭제 | 삭제 요청 HTTP 403 / FORBIDDEN, 기존 게시글 유지 | 타인 게시글의 수정 버튼 미표시 / Bob의 정상 JWT로 Alice 게시글 삭제 요청 : HTTP 403 , FORBIDDEN | PASS | 테스트 증거 B. [T08] |
| T09 | 유효 토큰의 서명 부분 첫 글자를 다른 문자로 바꿔 /me 및 변경 요청 | HTTP 401 / INVALID_TOKEN, 데이터 변경 없음 | GET /api/me : HTTP 401, INVALID_TOKEN / POST /api/posts : HTTP 401, INVALID_TOKEN, PUT, DELETE 및 데이터 유지 여부 | PASS | 테스트 증거 C. [ T09 - 1 ] & [ T09 - 2 ] |
| T10 | 짧은 TTL로 발급하고 실제 만료 뒤 /me 및 변경 요청 | HTTP 401 / TOKEN_EXPIRED, 화면 로그인 해제와 안내 | GET /api/me : HTTP 401, TOKEN_EXPIRED / POST /api/posts : HTTP 401, TOKEN_EXPIRED, 화면 로그인 해제 및 만료 안내 확인 | PASS | 테스트 증거 C. [ T10 - 1 ] & [ T10 - 2 ] |
| T11 | 로그인 후 로그아웃, 다음 /me 요청 | 헤더에서 토큰이 빠지고 HTTP 401 / TOKEN_REQUIRED | GET /api/me : HTTP 401, Authorization 헤더 없음, TOKEN_REQUIRED | PASS | 테스트 증거 E. [ T11 ] |
| T12 | Bob 로그인 상태로 글 등록 요청에 authorId:1 또는 userId:1 추가 | HTTP 400 / BAD_INPUT, 위조 글 미생성 | 작성자 위조 요청 : HTTP 400, BAD_INPUT / 허용되지 않은 JSON 필드로 요청 거부, 위조글 미생성 확인 | PASS | 테스트 증거 E. [ T12 ] |

| ID | 수행 절차와 사용자 | 예상 결과 | 실제 HTTP와 code | 판정 | 증거 |
|---|---|---|---|---|---|
| T13 | 본인 토큰으로 빈 제목,공백 본문,길이 초과 입력을 각각 전송 | HTTP 400 / BAD_REQUEST 및 BAD_INPUT 반환 게시글 생성 방지 | 빈 제목 : 브라우저 필수 입력 검사로 등록 차단 ( HTTP 요청 미발생 ) / 공백 본문 : 400, BAD_INPUT 응답 확인 / 길이 초과 : 실제 100자 이하로 제한되어 등록된 것임을 명확히 표시 | 부분확인 | 테스트 증거 E.<br>[ T13 - 1 ] & [ T13 - 2 ] & [ T13 - 3 ] |
| T14 | 없는 양의 정수 ID의 조회,수정,삭제 및 잘못된 ID 조회 | 없는 글의 조회,수정,삭제, 요청 HTTP 404 / NOT_FOUND, 잘못된 ID HTTP 400 / BAD_INPUT | 없는 게시글 조회 : 404, NOT_FOUND / 없는 게시글 수정 : 404, NOT_FOUND / 없는 게시글 삭제 : 404, NOT_FOUND 응답 확인 / 잘못된 ID 조회 : 400, BAD_INPUT | PASS | 테스트 증거 E.<br>[ T14 - 1 ] & [ T14 - 2 ] & [ T14 - 3 ] & [ T14 - 4 ] |
| T15 | 글을 만든 뒤 서버 종료,재실행 | 기존 글과 작성자, 내용 유지 | 서버 재실행 후 게시글 목록 조회 : HTTP 200 OK ( 제목, 내용 작성자 정보 모두 동일하게 유지 ) | PASS | 테스트 증거 D.<br>[ T15 ] |
| T16 | 로그인 후 페이지 새로고침 | 공개 글 유지, 메모리 로그인 상태 해제 | 로그인 : 200 OK , 페이지 새로고침 후 : 로그아웃 상태 표시 확인 / 공개 게시글 : 기존 게시글 유지 및 200 OK | PASS | 테스트 증거 D.<br>[ T16 - 1 ] & [ T16 - 2 ] |
| T17 | 토큰 없는 수정,삭제 요청과 Bob 계정의 정상 작성,본인 수정,삭제 | 무토큰은 401, Bob 본인 작업은 성공 | 토큰 없는 수정 : HTTP 401 , TOKEN_REQUIRED 응답 확인 / 토큰 없는 삭제 : HTTP 401 , TOKEN_REQUIRED 응답 확인 / Bob 본인 글 작성 : HTTP 201 CREATED / Bob 본인 글 수정 : 200 OK / Bob 본인 글 삭제 : HTTP 204 No Content | PASS | 테스트 증거 E.<br>[ T17 - 1 ] & [ T17 - 2 ] & [ T17 - 3 ] & [ T17 - 4 ] & [ T17 - 5 ] |

# HTTP 응답 확인 기록

| 확인 항목 | 기대 응답 | 실제 HTTP와 code | 증거 위치 |
|---|---|---|---|
| 정상 조회·로그인·수정 | **200 OK** | HTTP 200 OK | 테스트 증거 A. [T02] |
| 새 글 등록 | **201 및 Location** | HTTP 201 CREATED | 테스트 증거 E. [ T17 - 3 ] |
| 본인 글 삭제 | **204 및 빈 본문** | HTTP 204 NO CONTENT | 테스트 증거 E. [ T17 - 5 ] |
| 입력 오류·추가 작성자 필드 | **400 BAD_INPUT** | 400 BAD REQUEST / BAD_INPUT ( 추가 작성자 필드 요청 거부 ) | 테스트 증거 E. [ T12 ] |
| 잘못된 계정 | **401 INVALID_CREDENTIALS** | HTTP 401 Unauthorized / INVALID_CREDENTIALS | 테스트 증거 E. [ T03 - 1 ] & [ T03 - 2 ] |
| 토큰 없음·헤더 형식 오류 | **401 TOKEN_REQUIRED** | HTTP 401 Unauthorized / TOKEN_REQUIRED | 테스트 증거 E. [ T04 - 1 ] & [ T04 - 2 ] |
| 변조·클레임 오류 | **401 INVALID_TOKEN** | HTTP 401 Unauthorized / INVALID_TOKEN | 테스트 증거 C. [ T09 - 1 ] & [ T09 - 2 ] |
| 정상 서명의 만료 토큰 | **401 TOKEN_EXPIRED** | HTTP 401 Unauthorized / TOKEN_EXPIRED | 테스트 증거 C. [ T10 - 1 ] & [ T10 - 2 ] |
| 다른 작성자의 변경 | **403 FORBIDDEN** | HTTP 403 Forbidden / FORBIDDEN | 테스트 증거 B. - [T07] & [ T08] |
| 없는 글 | **404 NOT_FOUND** | HTTP 404 NOT FOUND / NOT_FOUND | [ 사진 첨부 ] 정상 토큰으로 없는 글 수정 |

(1) 보호 API 401의 WWW-Authenticate: Bearer 확인 결과와 증거

: JWT 없이 게시글 삭제 요청을 전송했을 때 Network의 헤더에서 HTTP 41 Unauthorized 응답과 WWW-Quthenticate: Bearer 헤더를 확인했습니다. 또한 Response에서 JSON 오류 코드 TOKEN_REQUIRED가 반환되는 것을 확인했습니다. 이를 통해 인증 토큰이 없는 요청에 대해 서버가 접근을 거부하고 Bearer 인증이 필요하다는 것이 응답 헤더에서 나타나있는 것을 확인했습니다.

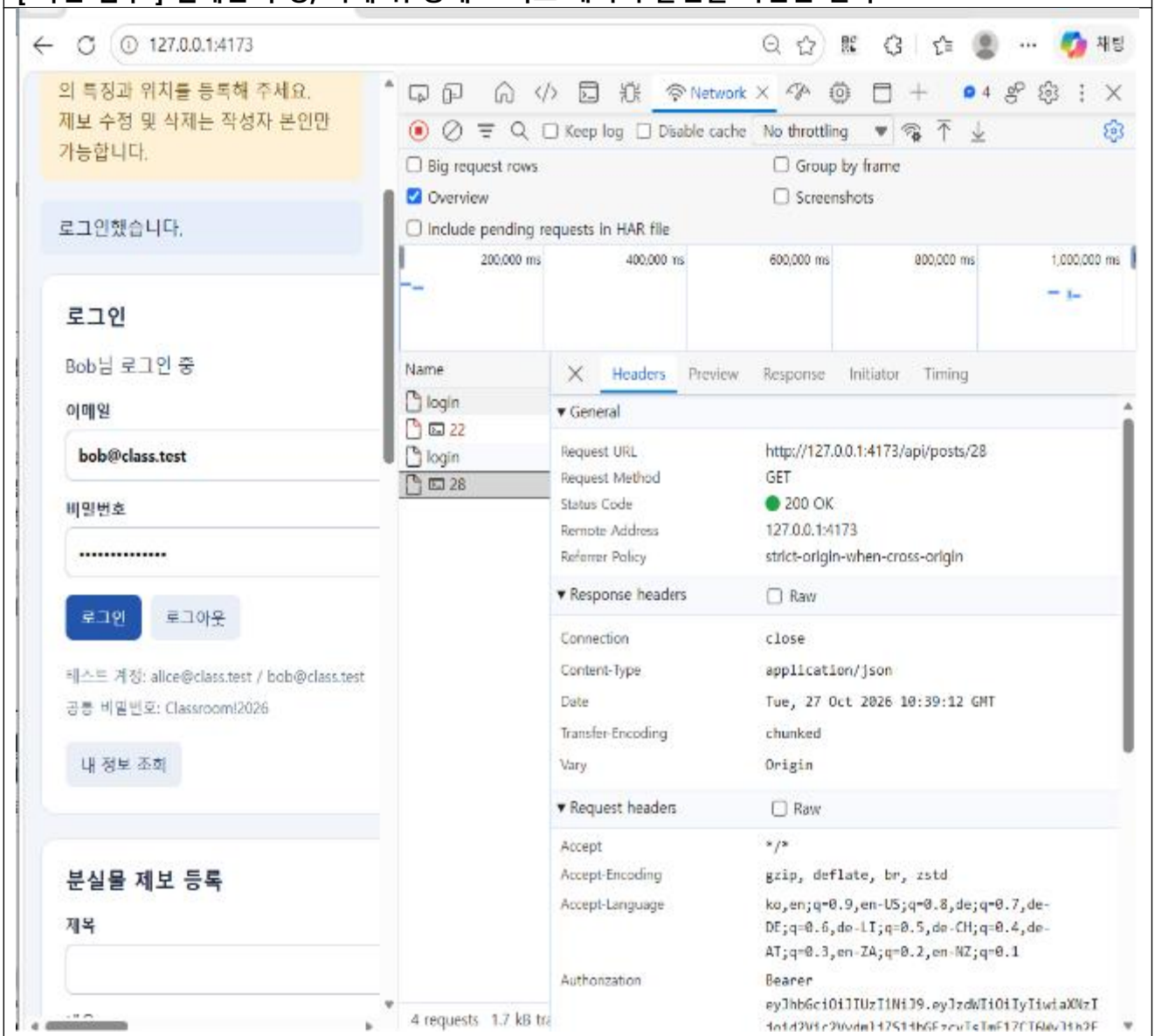
(2) HTTP 401과 HTTP 403에서 화면이 각각 어떻게 달라지는지 설명

: HTTP 401은 인증 토큰이 없거나 유효하지 않은 경우에 발생하는 오류로, 토큰 없이 게시글을 수정 및 삭제 요청을 전송했을 때 HTTP 401과 TOKEN_REQUIRED가 반환되는 것을 확인했습니다. 로그아웃 상태에서는 게시글 수정 및 삭제 기능을 사용할 수 없었습니다. HTTP 403은 로그인한 사용자에게 해당 게시글을 변경할 권한이 없는 경우에 발생하는 오류입니다. 먼저 하나의 계정(Bob)을 로그인한 상태에서 다른 계정(Alice)가 작성한 게시글의 수정, 삭제 버튼이 표시되지 않는 것을 확인한 이후 로그인 한 계정에서의 정상 JWT로 직접 수정 및 삭제 요청을 전송한 결과 두 요청 모두 HTTP 403과 FORBIDDEN이 반환되는 것 또한 확인해보았습니다. 이를 통해 화면의 권한 제한뿐만 아니라 서버에서도 다른 작성자의 게시글 변경 요청을 차단하는 것을 알 수 있습니다.

(3) 실패한 수정, 삭제 뒤 공개 조회로 데이터 불변을 확인한 결과

: 테스트 T17에서 JWT 없이 게시글 수정 및 삭제 요청을 전송한 결과 두 요청 모두 HTTP 401 Unauthorized와 JSON 오류 코드 TOKEN_REQUIRED가 반환되는 것을 확인했습니다. 이를 통해 인증되지 않은 사용자의 게시글 변경 요청이 서버에서 거부되는 것을 확인했습니다. 그 이후 공개 조회(GET)을 전송한 결과 HTTP 200 OK가 반환되었으며 원래 작성자가 작성한 게시글이 그대로 존재하는 것을 확인했습니다. 이를 통해 권한이 없는 사용자의 삭제 요청이 실제 데이터에 반영되지 않았음을 확인했습니다.

**[ 사진 첨부 ] 실패한 수정, 삭제 뒤 공개 조회로 데이터 불변을 확인한 결과**

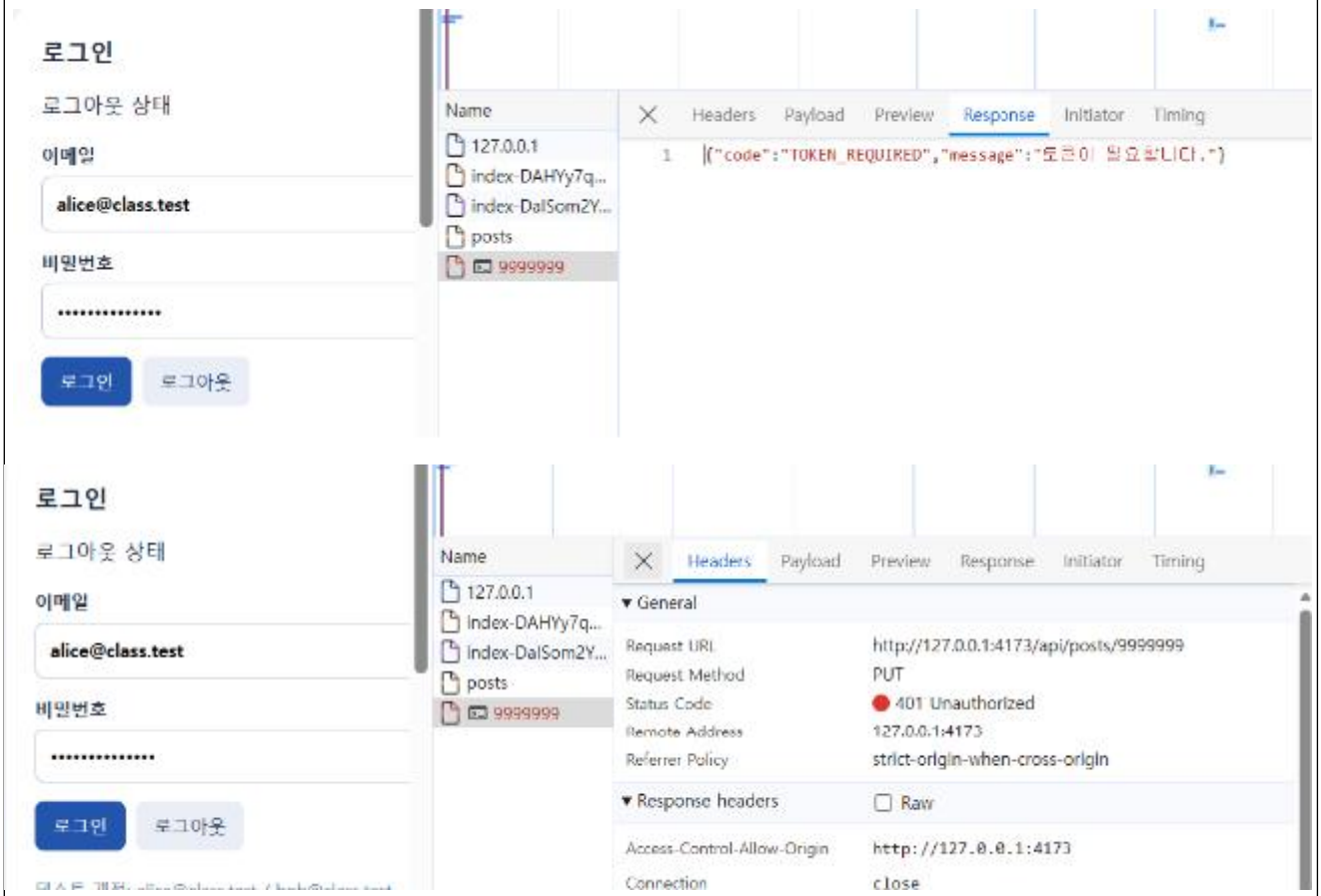



(4) 토큰 없이 없는 글 수정 HTTP 401 / 정상 토큰으로 없는 글 수정 HTTP 404의 실제 결과

: 테스트 T14에서 Alice 계정의 정상 JWT를 사용해 존재하지 않는 게시글 ID에 수정요청을 전송한 결과, HTTP 404 NOT FOUND와 JSON 오류 코드가 NOT_FOUND로 반환되는 것을 확인했습니다. 추가로 JWT

없이 존재하지 않는 게시글 ID(9999999)에 수정 요청을 전송한 결과를 확인해봤을 때 Network에서 HTTP 401 Unauthorized 응답을 확인했습니다. Pesponse에서는 JSON 오류 코드가 TOKEN_REQUIRED가 반환되는 것 또한 확인했습니다. 이를 통해 인증 토큰이 없는 요청은 게시글의 존재 여부와 관계없이 HTTP 401로 거부되며, 정상적으로 인증된 요청에서는 게시글의 존재 여부를 검사하여 존재하지 않는 경우 HTTP 404를 반환하는 것을 확인했습니다.

**[ 사진 첨부 ] 정상 토큰으로 없는 글 수정**

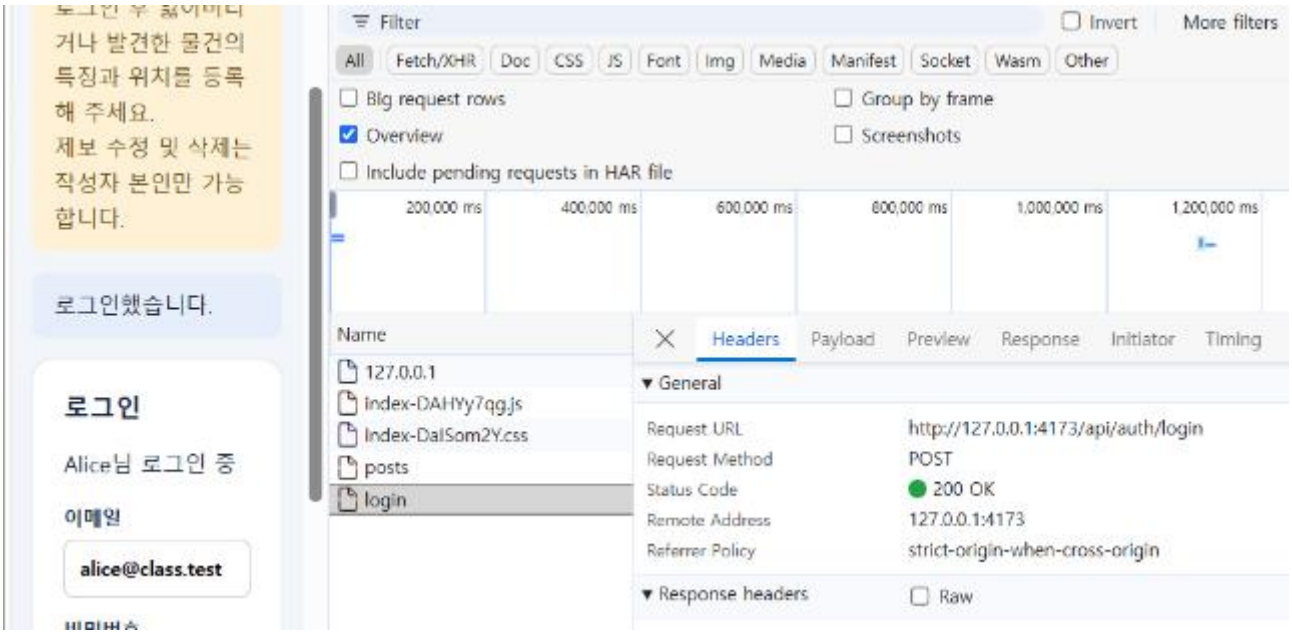


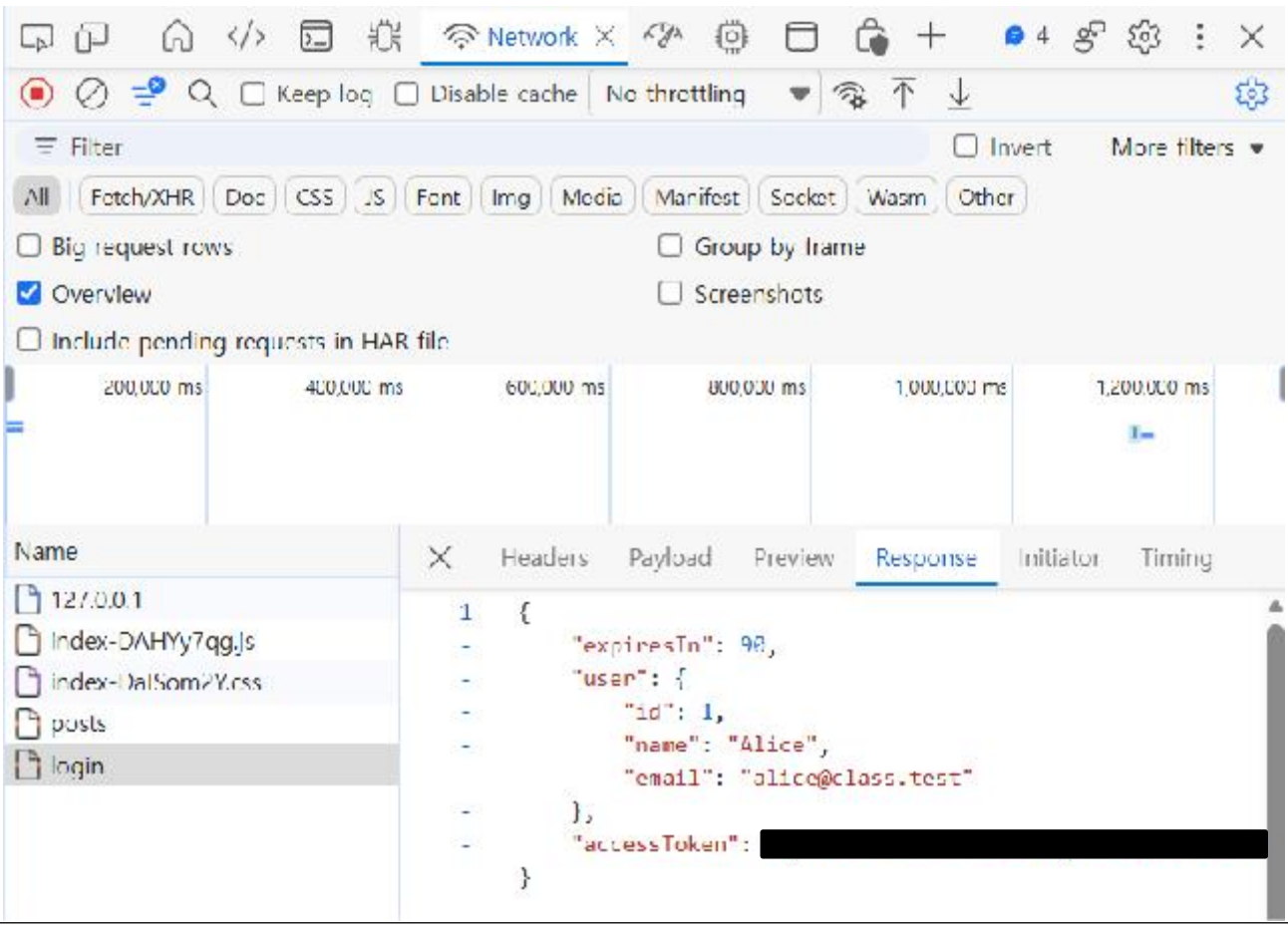



# 테스트 증거

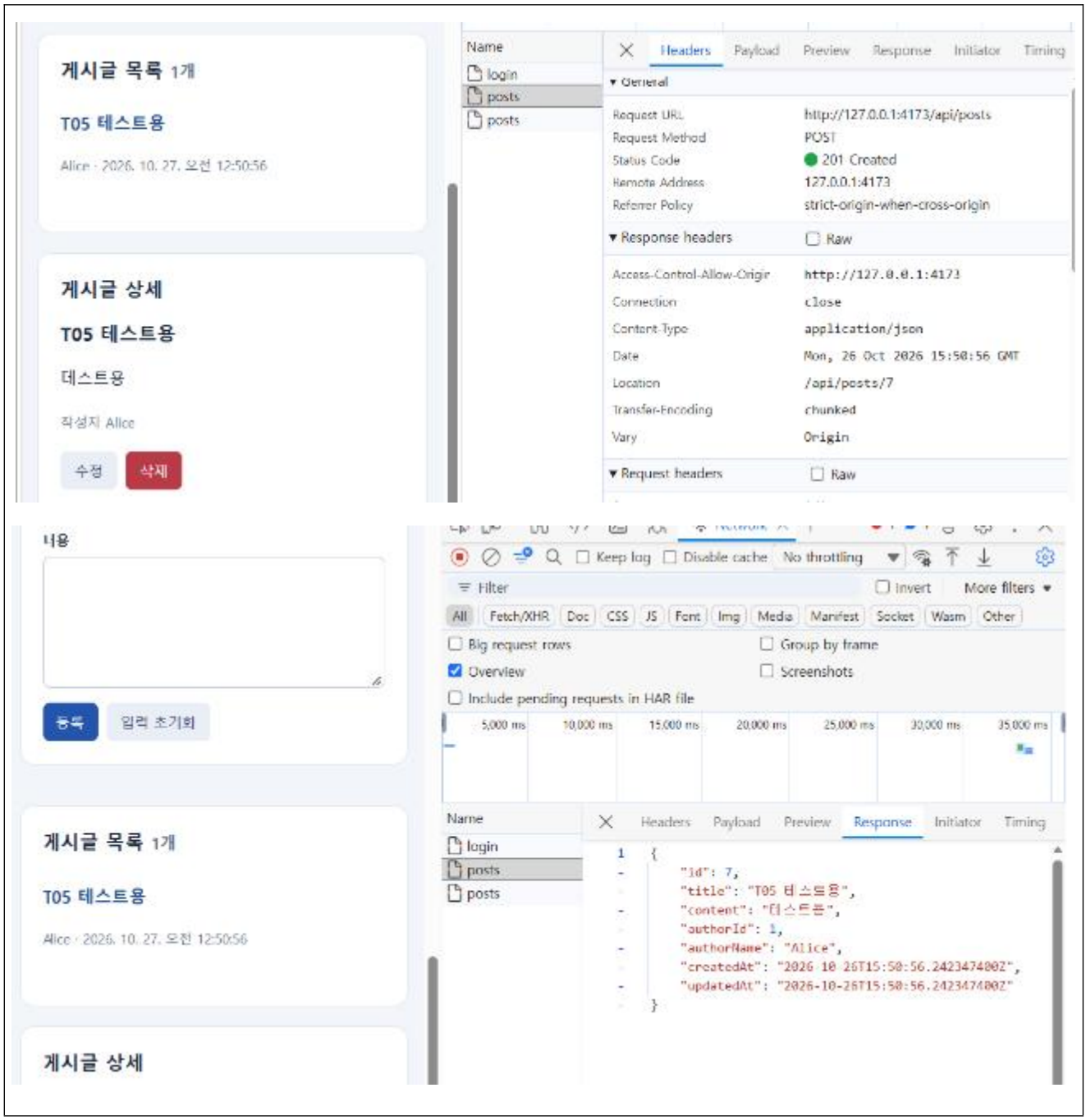
## A. 로그인과 토큰 전송

**[ T02 ] Alice 계정으로 로그인 ( 헤더 및 응답 )**

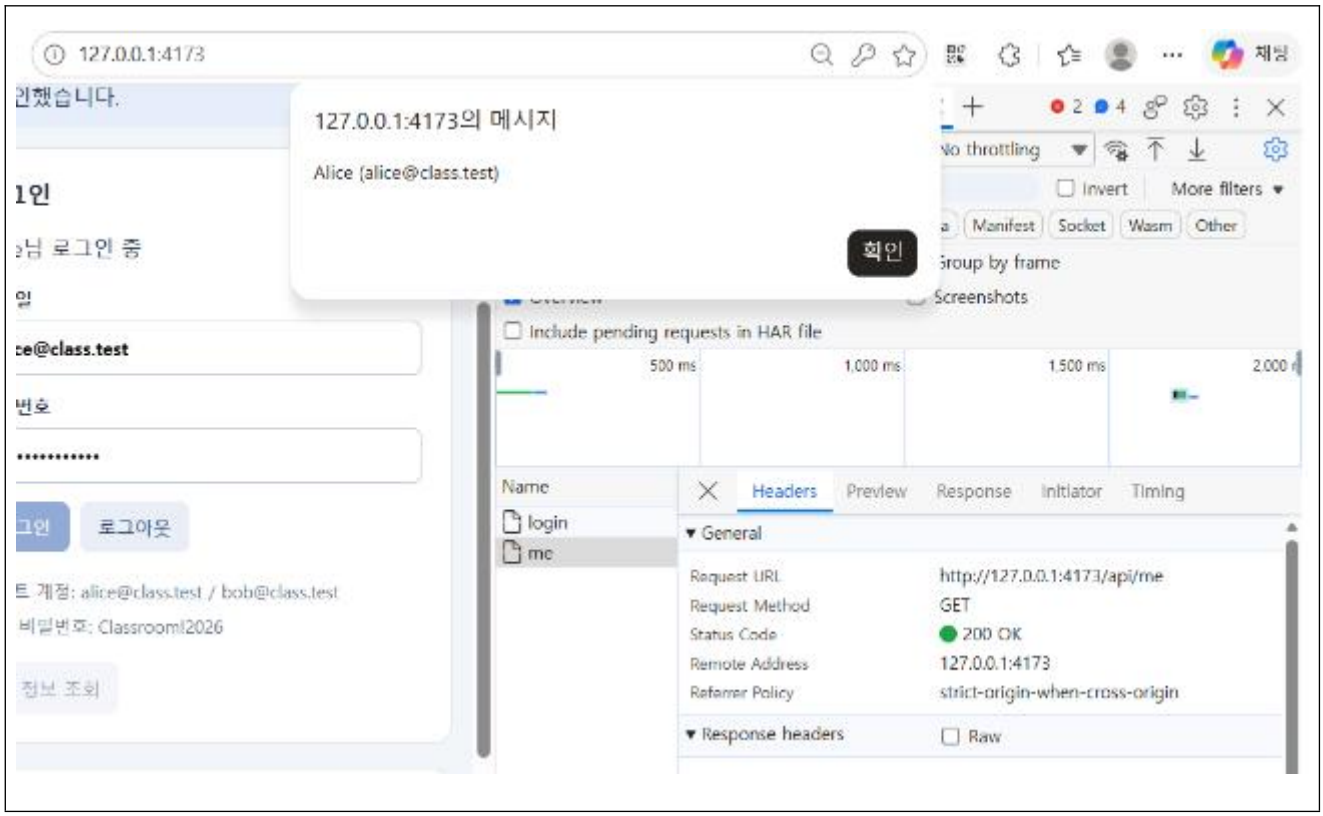



**[ T05 - 1 ] Alice 토큰으로 글 작성 ( 응답 및 헤더 )**

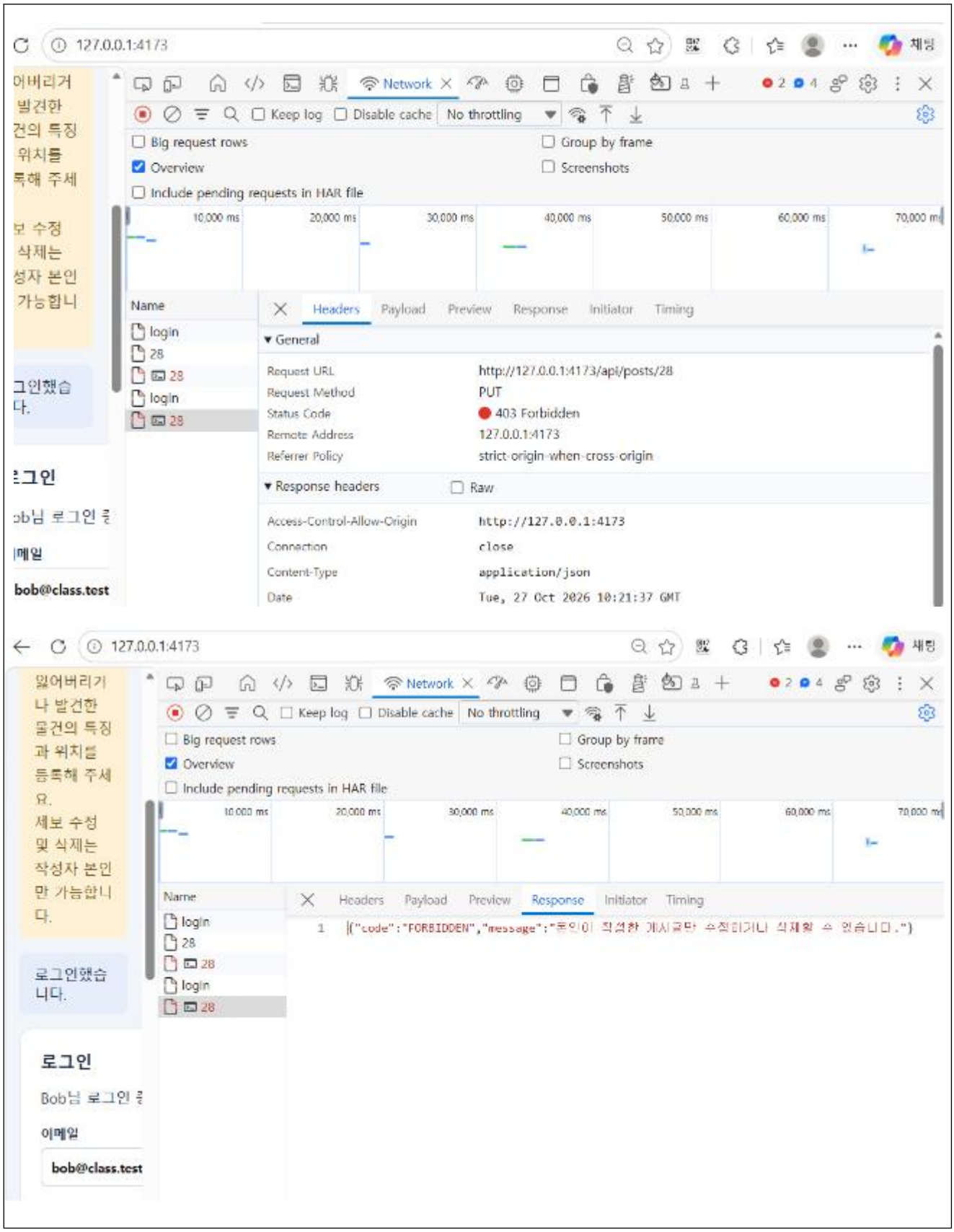



**[ T05 - 2 ] Alice 토큰으로 /me 조회**

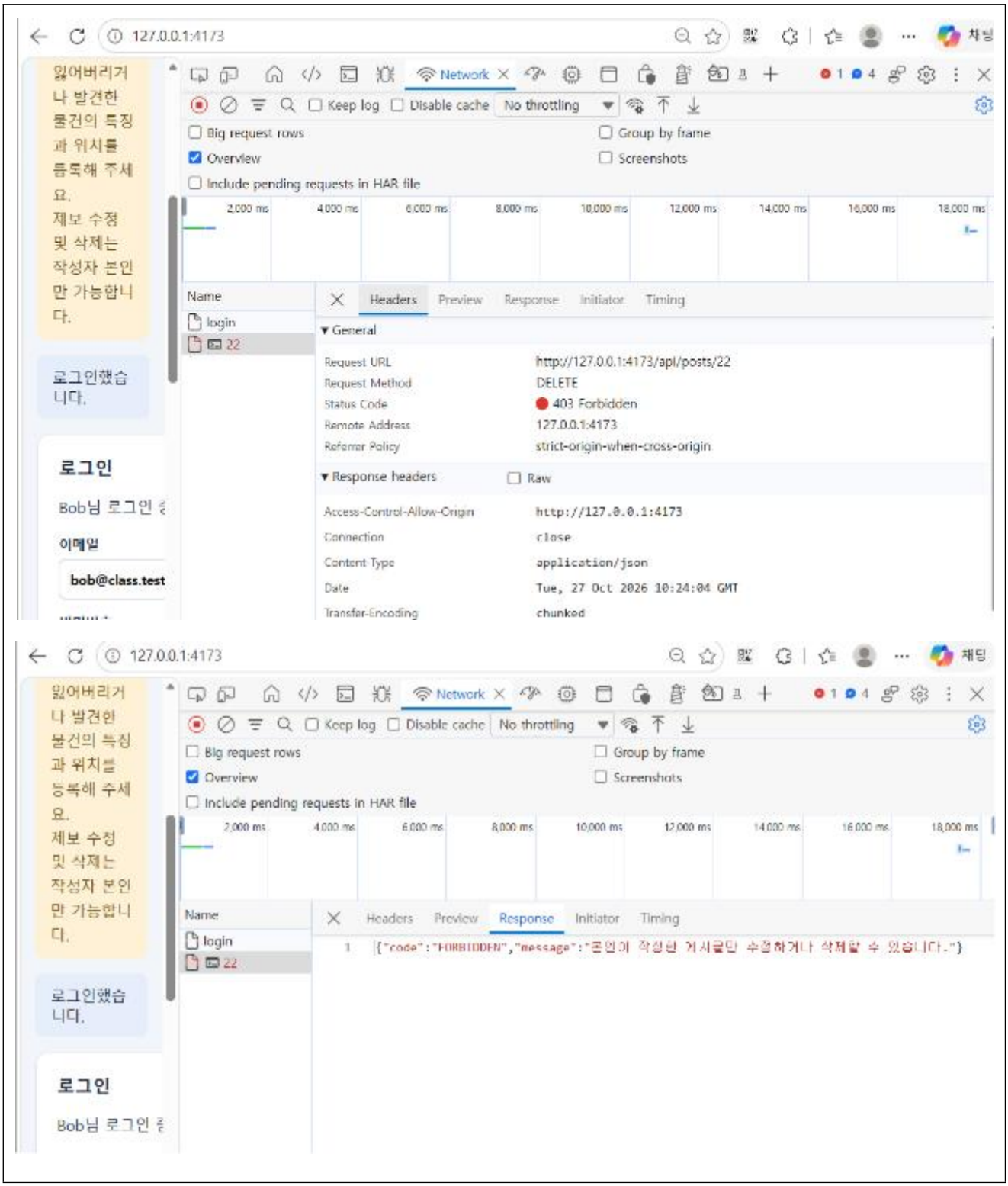



## B. 다른 작성자의 수정과 삭제

**[ T07 ] Alice 글을 Bob의 정상 토큰으로 수정 ( 헤더 및 응답 )**

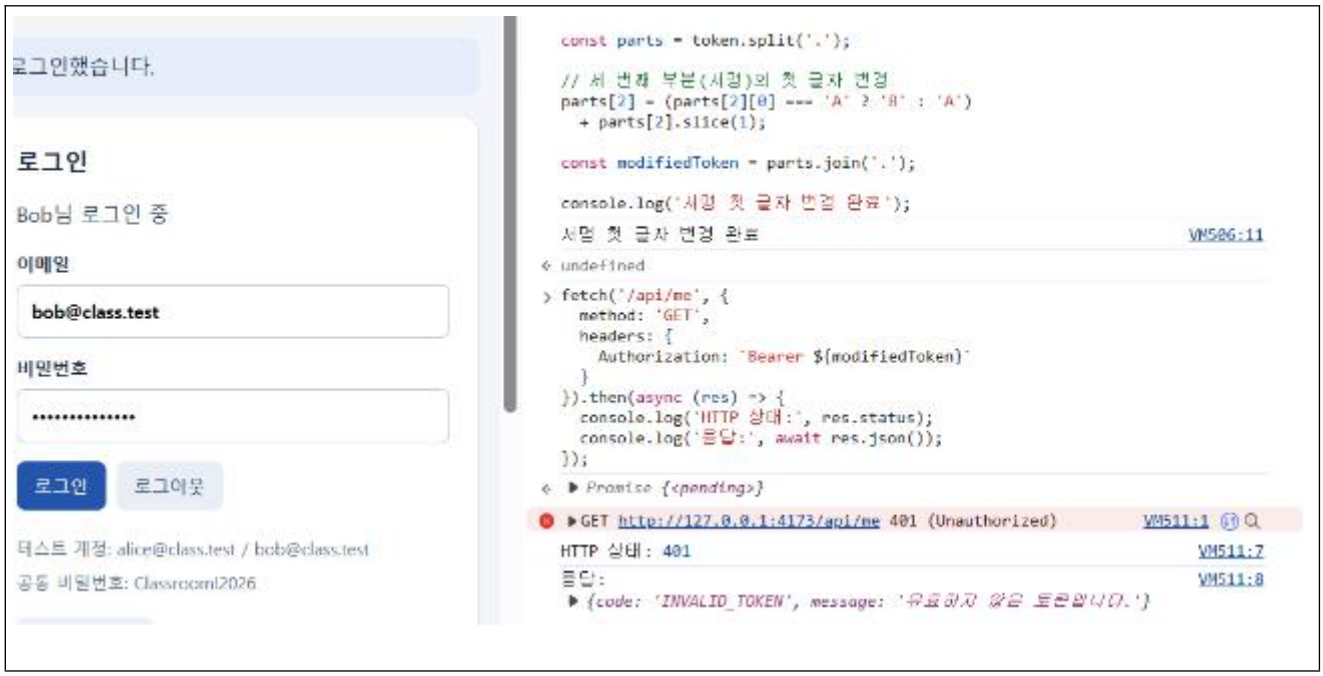



**[ T08 ] Alice 글을 Bob의 정상 토큰으로 삭제 ( 헤더 및 응답 )**




## C. 토큰 변조와 만료

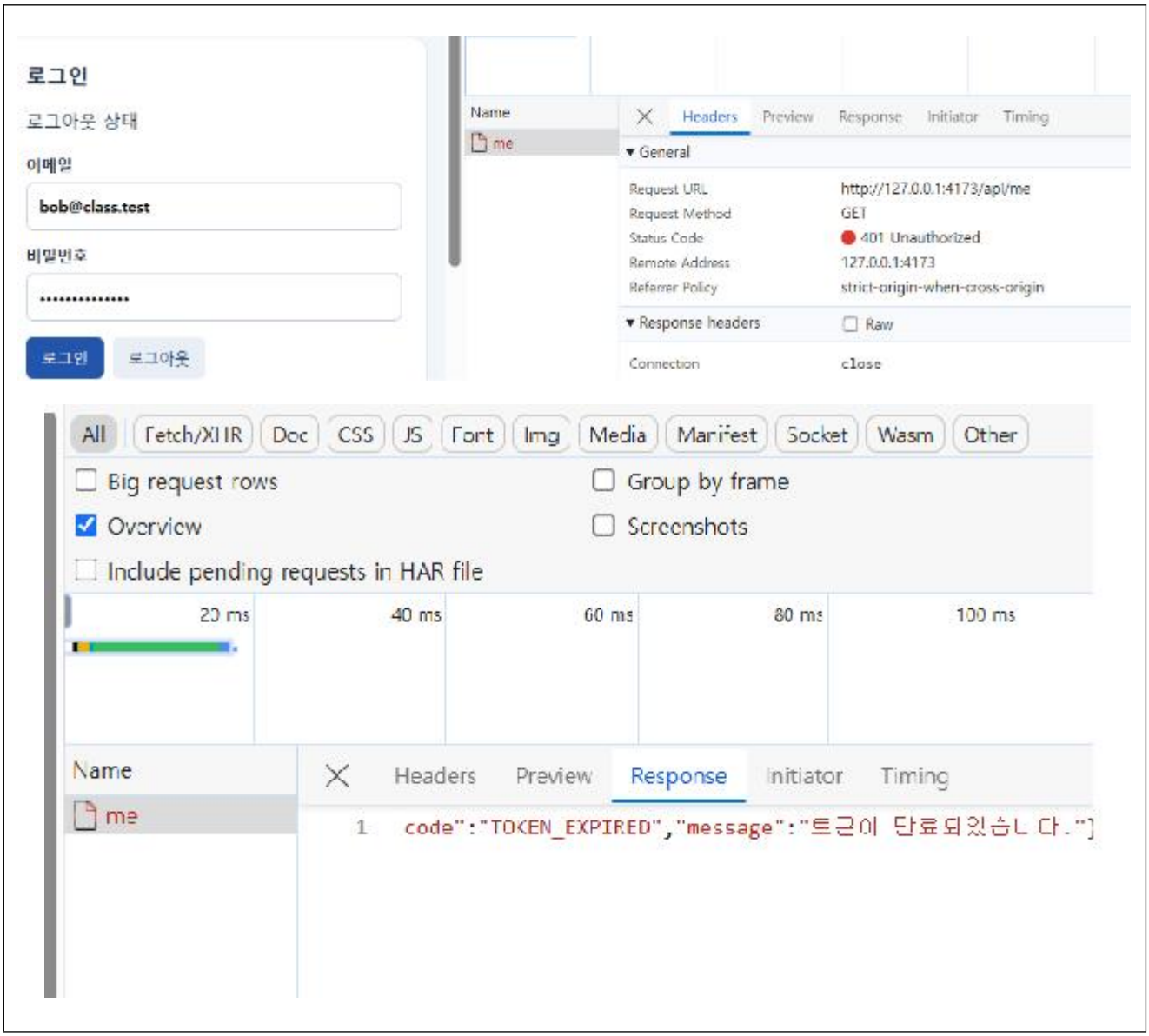
**[ T09 - 1 ] 유효 토큰의 서명 부분 첫 글자를 다른 문자로 변경**



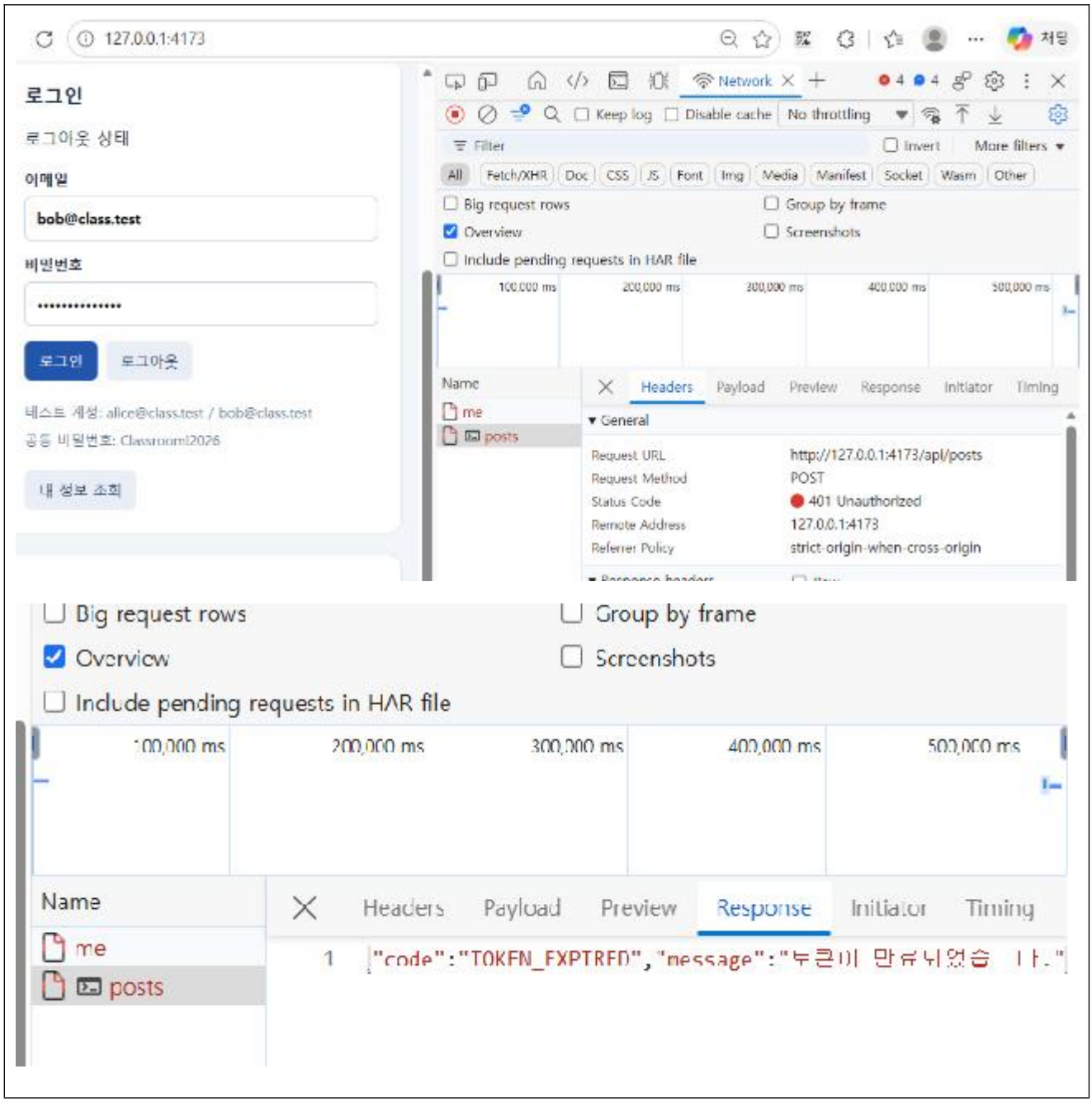

**[ T09 - 2 ] 유효 토큰 변경 후 /me 및 변경 요청**




**[ T10 - 1 ] 만료된 JWT로 /me 조회 ( 헤더 및 응답 )**

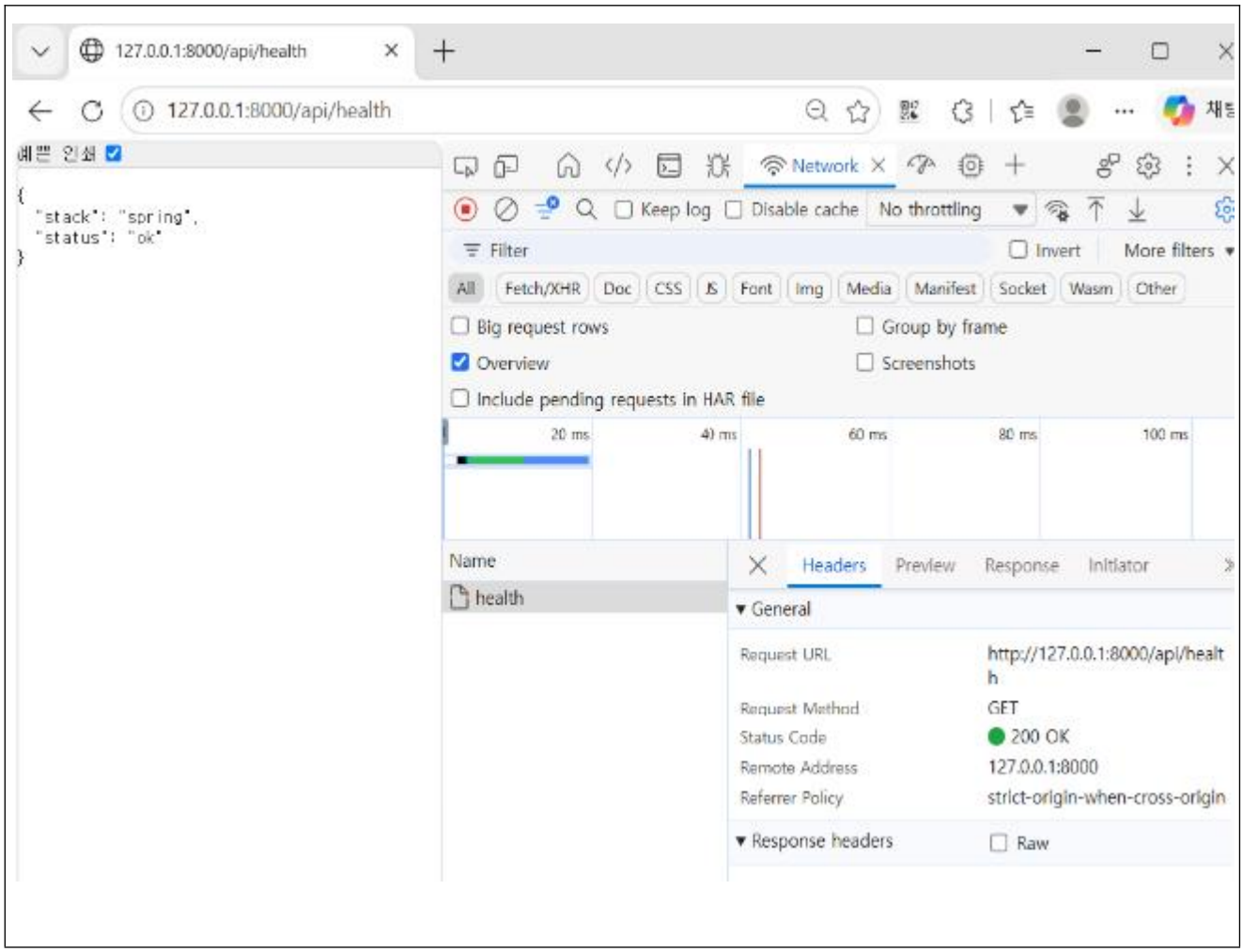



**[ T10 - 2 ] 만료된 JWT로 변경 요청 ( 헤더 및 응답 )**




## D. 자동 검사와 로컬 배포 증거

**4173번 API 연결**




**빌드 성공 로그 ( 프론트엔드 빌드 & 백엔드 패키징 )**

```
PS C:\Users\락은흐\Downloads\02_시작템플릿_및_제출양식\JWT Assignment\frontend vue> npm run build

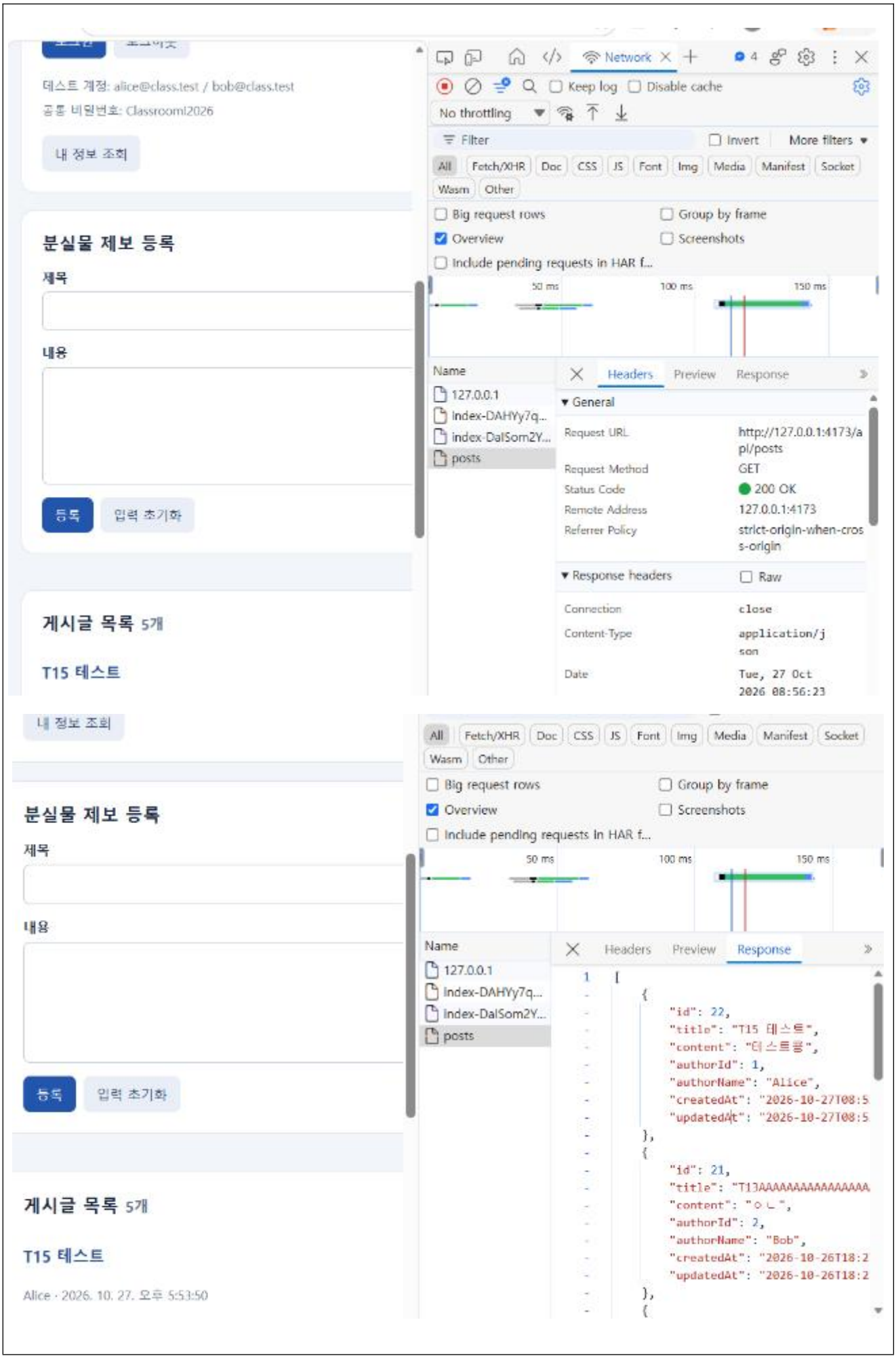
> jwt-board-vue@1.0.0 build
> vite build

vite v8.3.4 building client environment for production...
✓ 12 modules transformed.
computing gzip size...
dist/index.html                  0.44 kB | gzip:  0.33 kB
dist/assets/index-DaISom2Y.css   1.65 kB | gzip:  0.79 kB
dist/assets/index-DAHVy7qg.js   70.27 kB | gzip: 27.85 kB

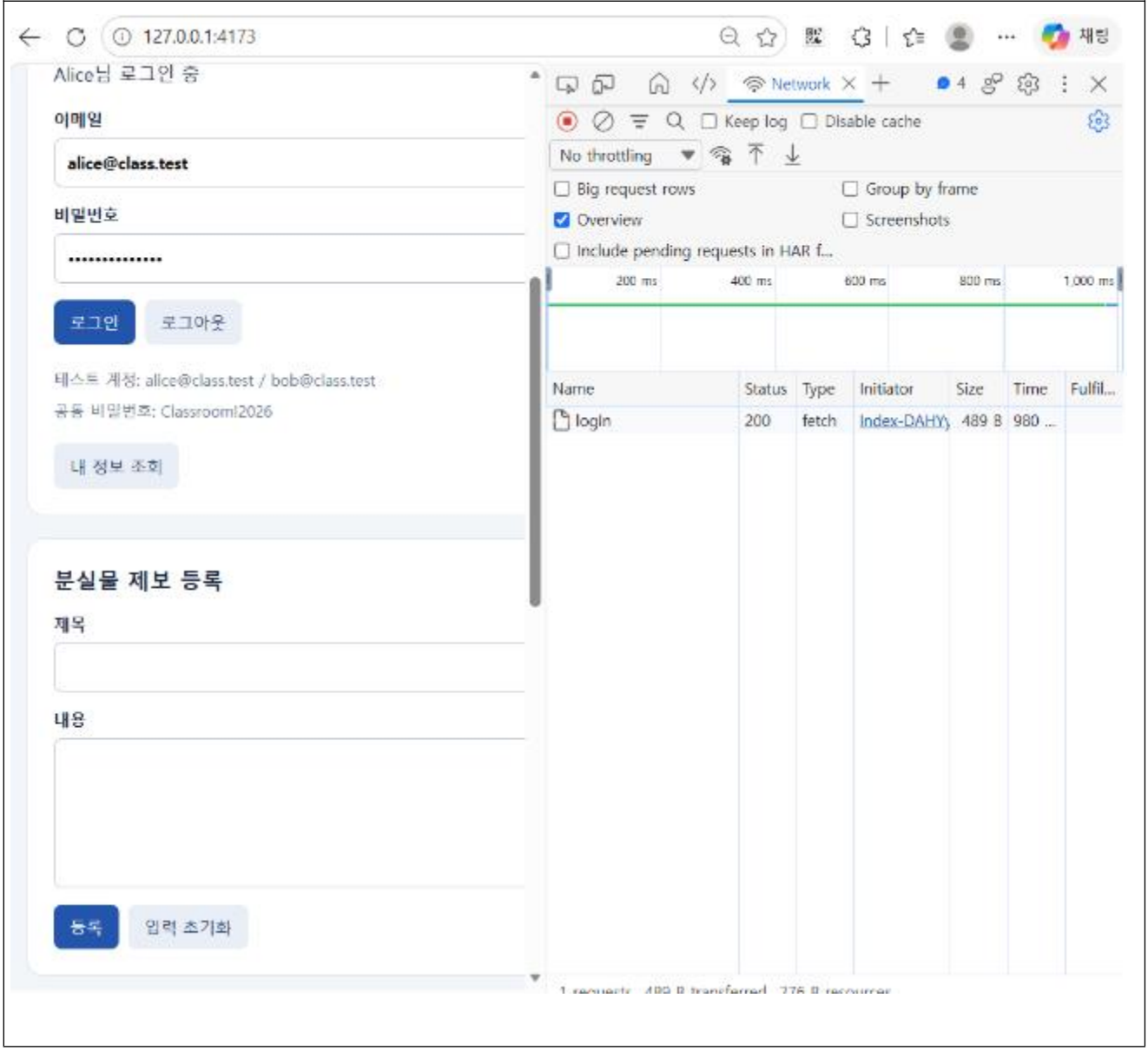
✓ built in 406ms
```

```
[INFO] Replacing main artifact C:\Users\박은흐\Downloads\02_시작템플릿_및_제출양식\JWT-Assignment\back
end-spring\target\jwt-board-1.0.0.jar with repackaged archive, adding nested dependencies in BOOT-INF/
.
[INFO] The original artifact has been renamed to C:\Users\락은흐\Downloads\02_시작템플릿_및_제출양식\J
WT-Assignment\backend-spring\target\jwt-board-1.0.0.jar.original
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  33.627 s
[INFO] Finished at: 2026-10-26T23:21:59+09:00
[INFO] ------------------------------------------------------------------------
```

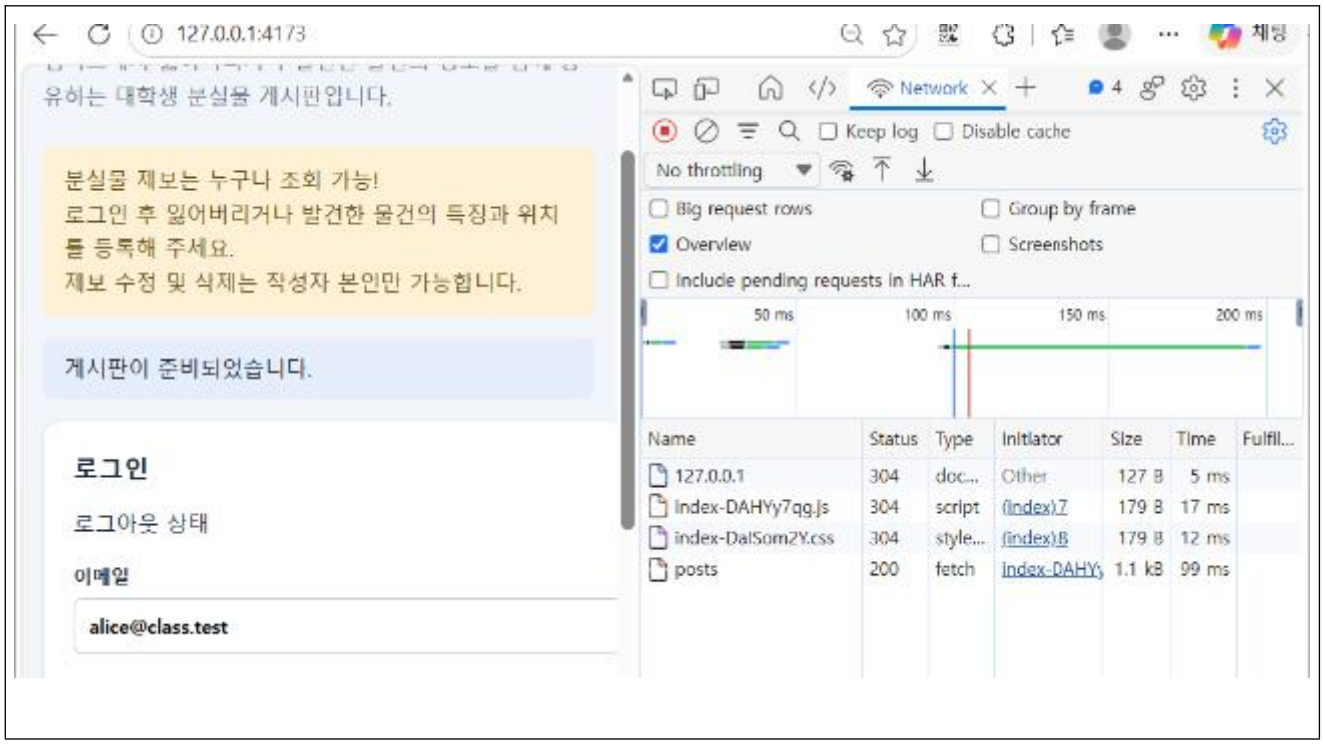
**[ T15 ] 글을 만든 뒤 서버 종료, 재실행 ( 헤더 및 응답 )**


데스트 계정: alice@class.test / bob@class.test
공통 비밀번호: Classroom!2026
내 정보 조회
분실물 제보 등록
제목
내용
등록
입력 초기화
게시글 목록 5개
T15 테스트
Network
Keep log
Disable cache
No throttling
Filter
Invert
More filters
All
Fetch/XHR
Doc
CSS
JS
Font
Img
Media
Manifest
Socket
Wasm
Other
Big request rows
Group by frame
Overview
Screenshots
Include pending requests in HAR f...
50 ms
100 ms
150 ms
Name
127.0.0.1
Index-DAHYy7q...
index-DalSom2Y...
posts
Headers
Preview
Response
General
Request URL
http://127.0.0.1:4173/api/posts
Request Method
GET
Status Code
200 OK
Remote Address
127.0.0.1:4173
Referrer Policy
strict-origin-when-cross-origin
Response headers
Raw
Connection
close
Content-Type
application/json
Date
Tue, 27 Oct 2026 08:56:23
"id": 22,
"title": "T15 테스트",
"content": "데스트용",
"authorId": 1,
"authorName": "Alice",
"createdAt": "2026-10-27T08:5
"updatedAt": "2026-10-27T08:5
"id": 21,
"title": "T13AAAAAAAAAAAAAAAA
"content": "ㅇㄴ",
"authorId": 2,
"authorName": "Bob",
"createdAt": "2026-10-26T18:2
"updatedAt": "2026-10-26T18:2
Alice · 2026. 10. 27. 오후 5:53:50

**[ T16 - 1 ] 로그인 후 페이지 새로고침 ( 새로고침 하기 전 - 로그인 상태 )**

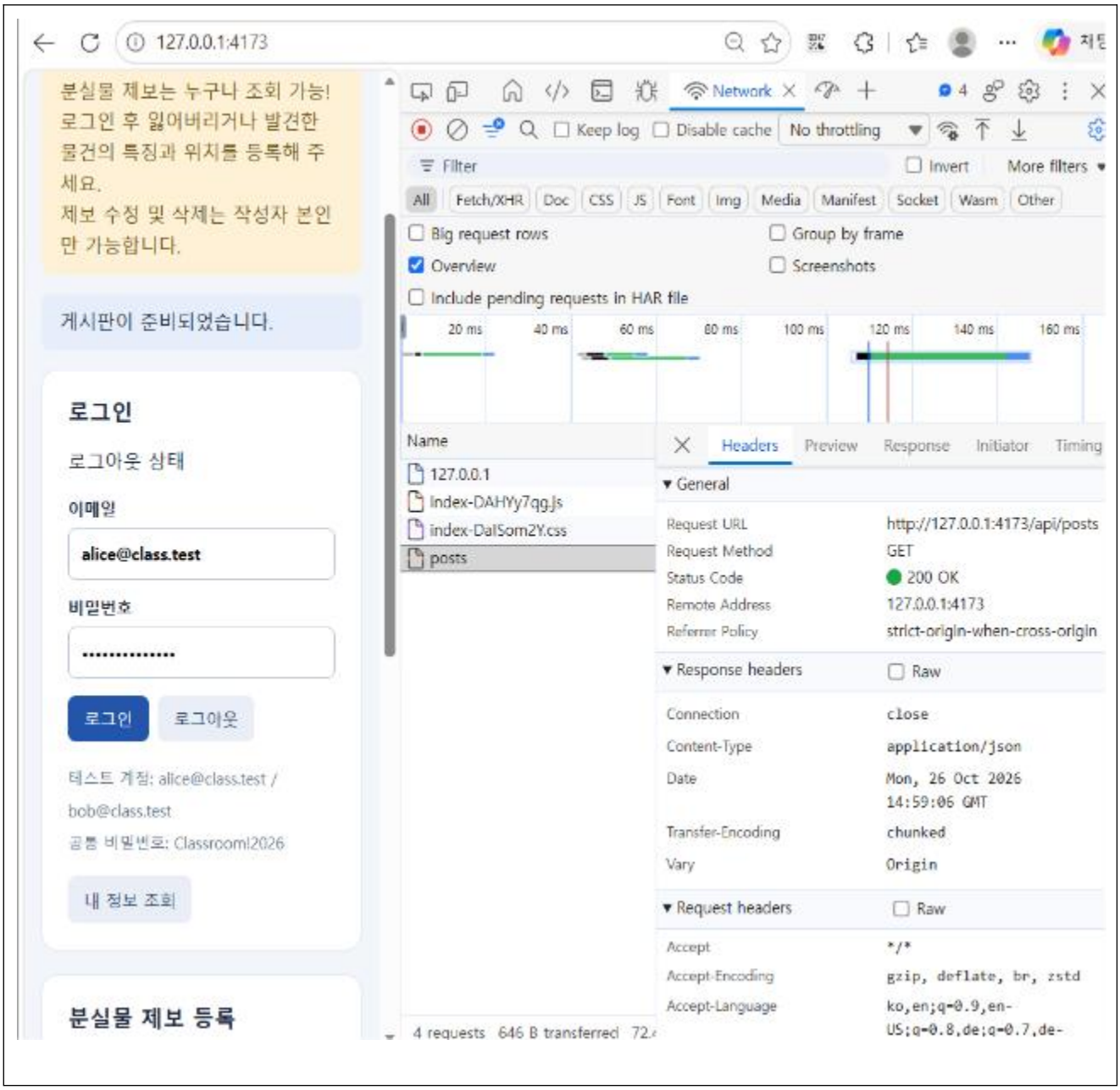



**[ T16 - 2 ] 로그인 후 페이지 새로고침 ( 새로고침 후 - 로그아웃 상태 )**

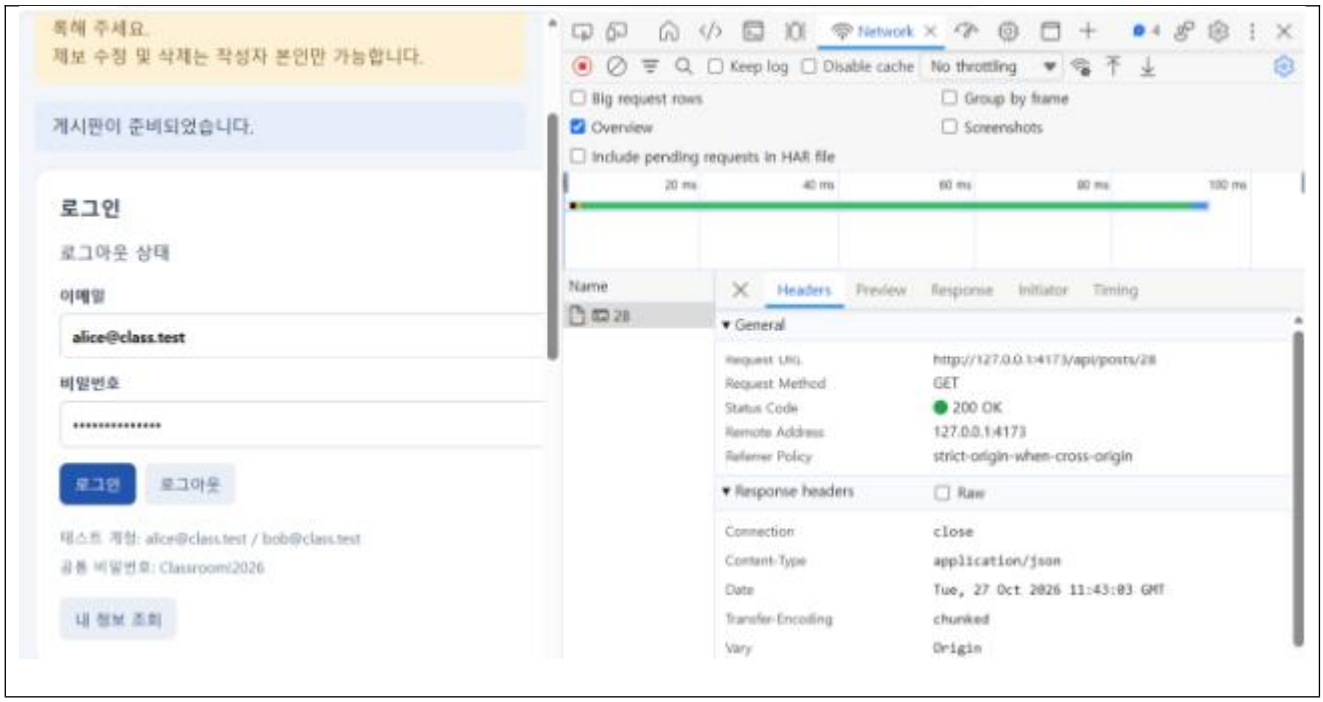



## E. 추가 테스트 증거

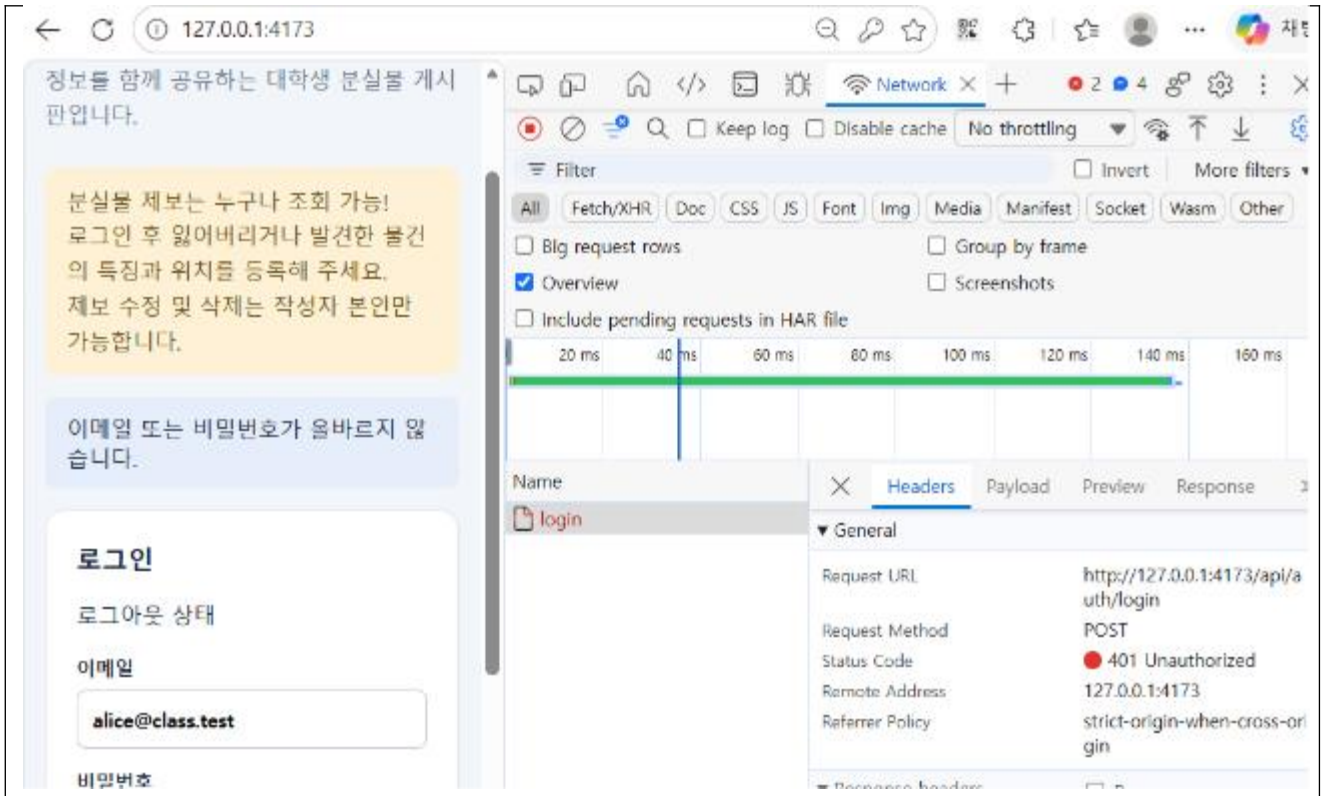
| [ T01 - 1 ] 토큰 없이 목록 조회 |
|---|

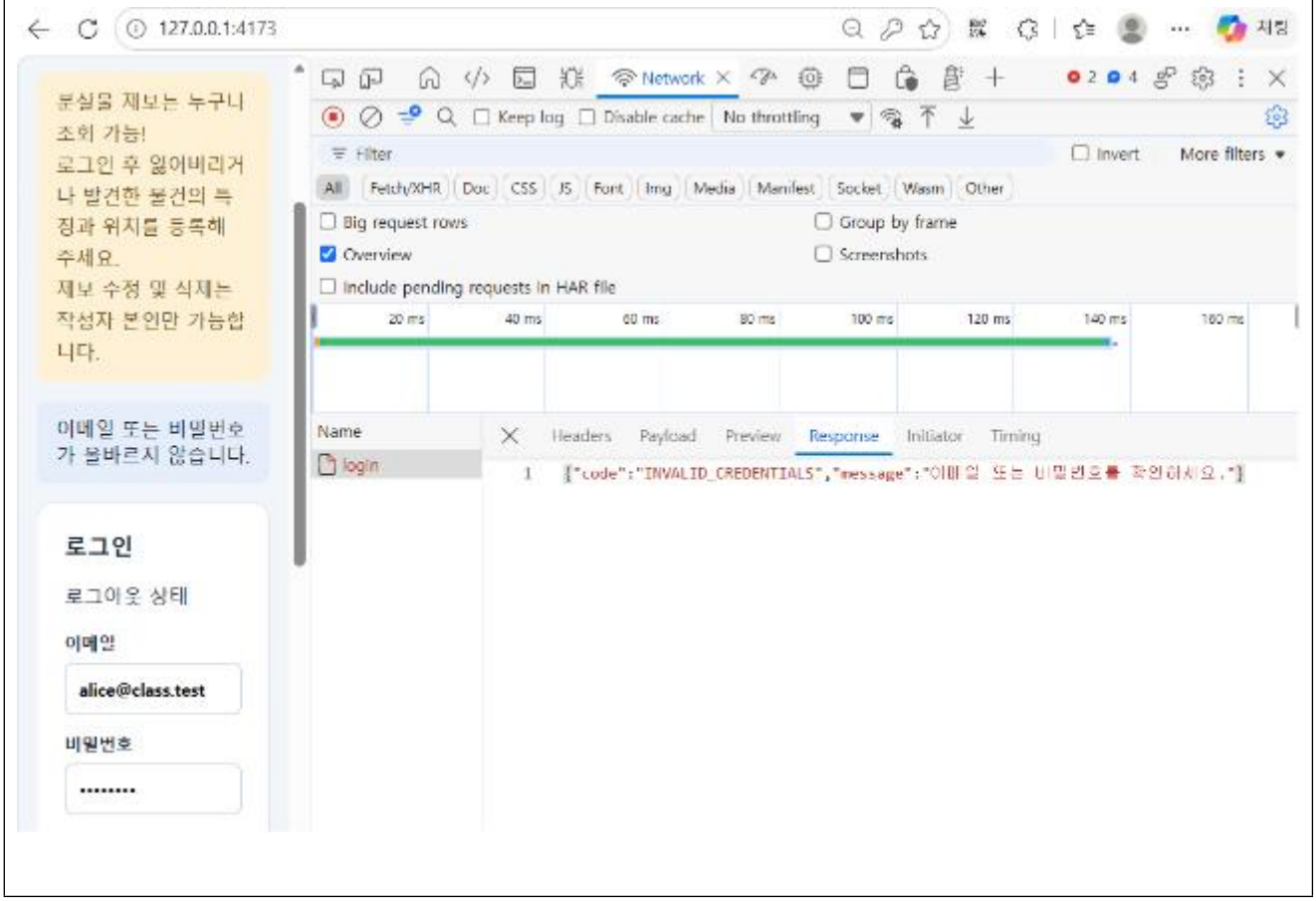



**[ T01 - 2 ] 비로그인 상세 목록 조회**

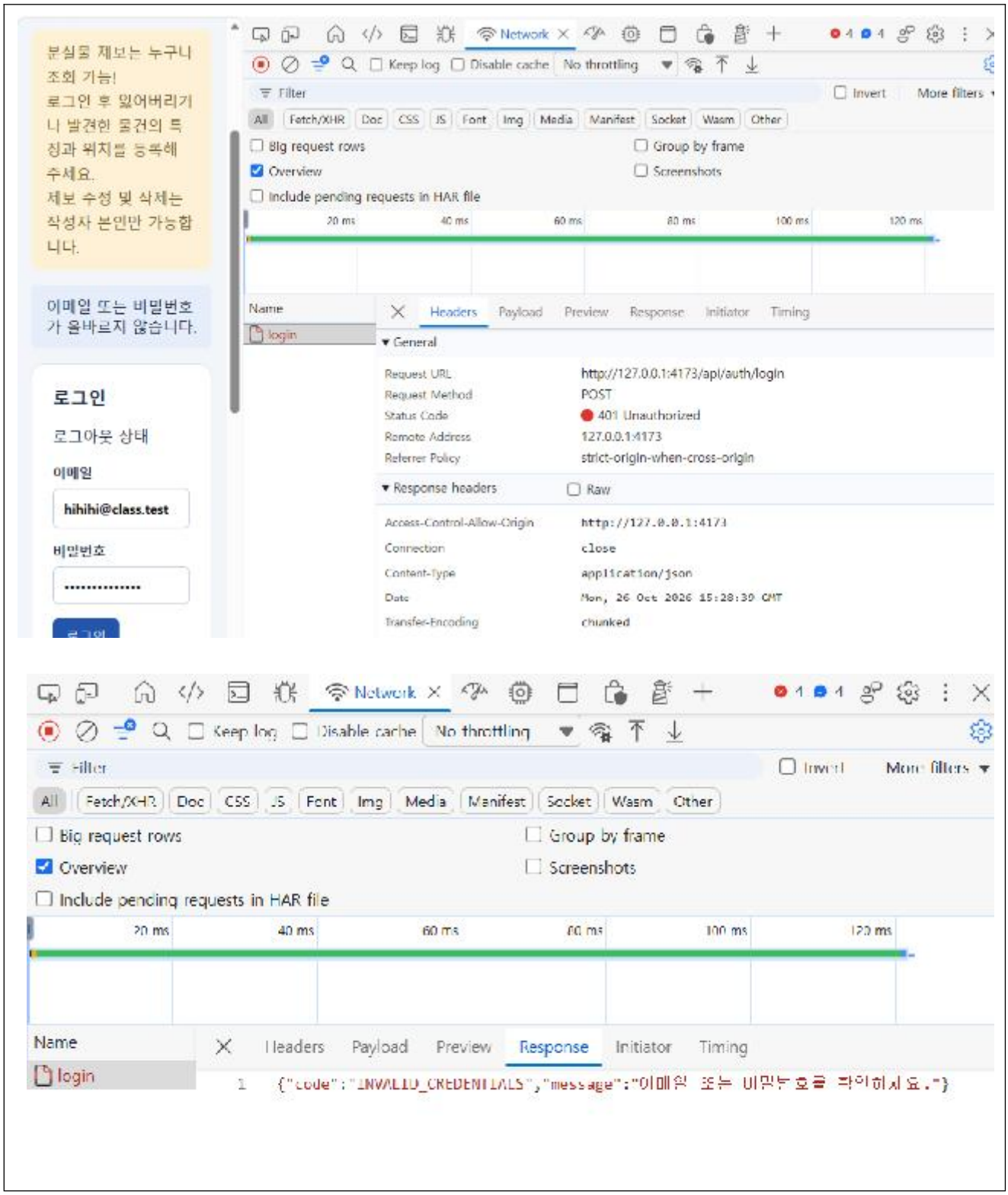



**[ T03 - 1 ] 잘못된 비밀번호**

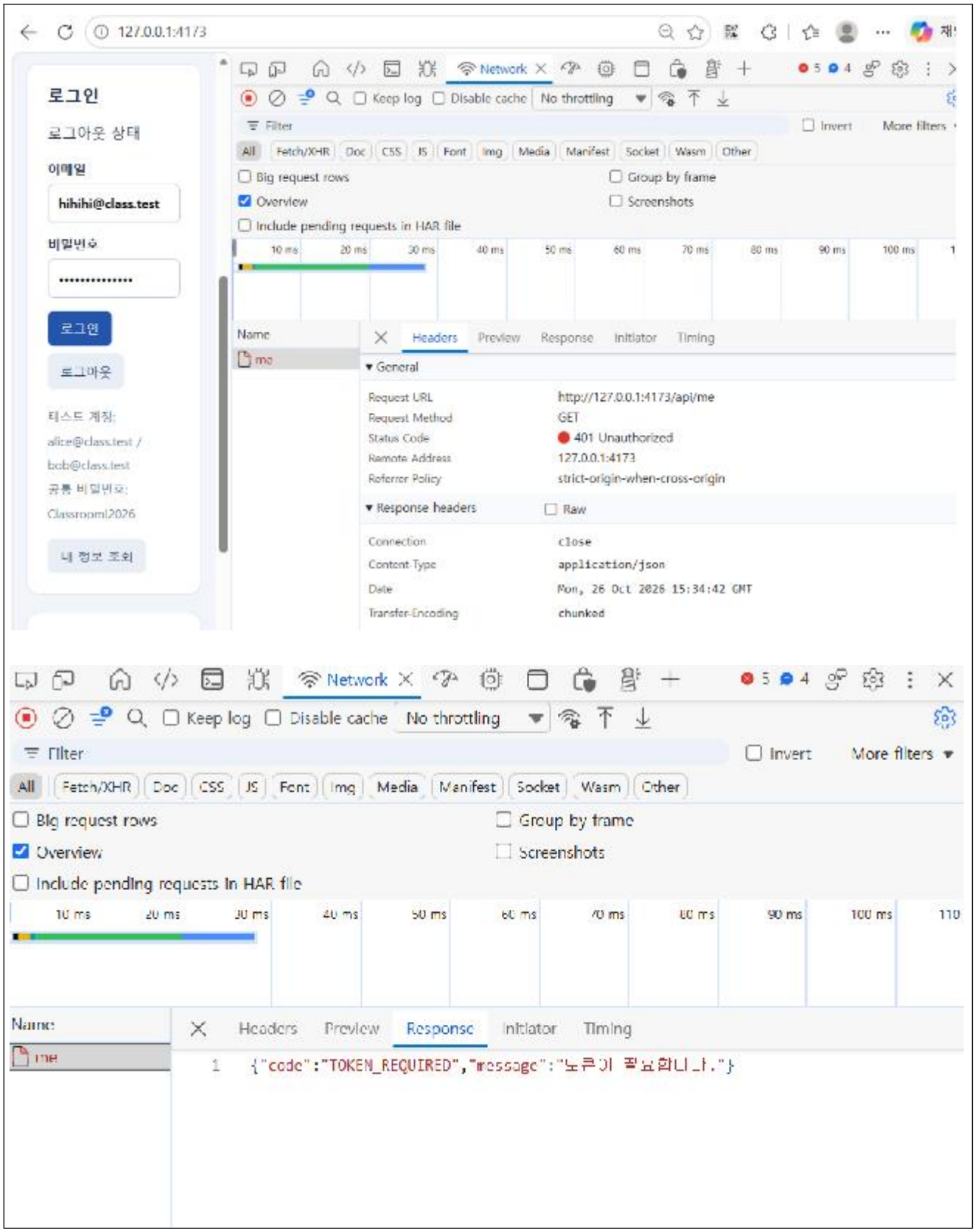


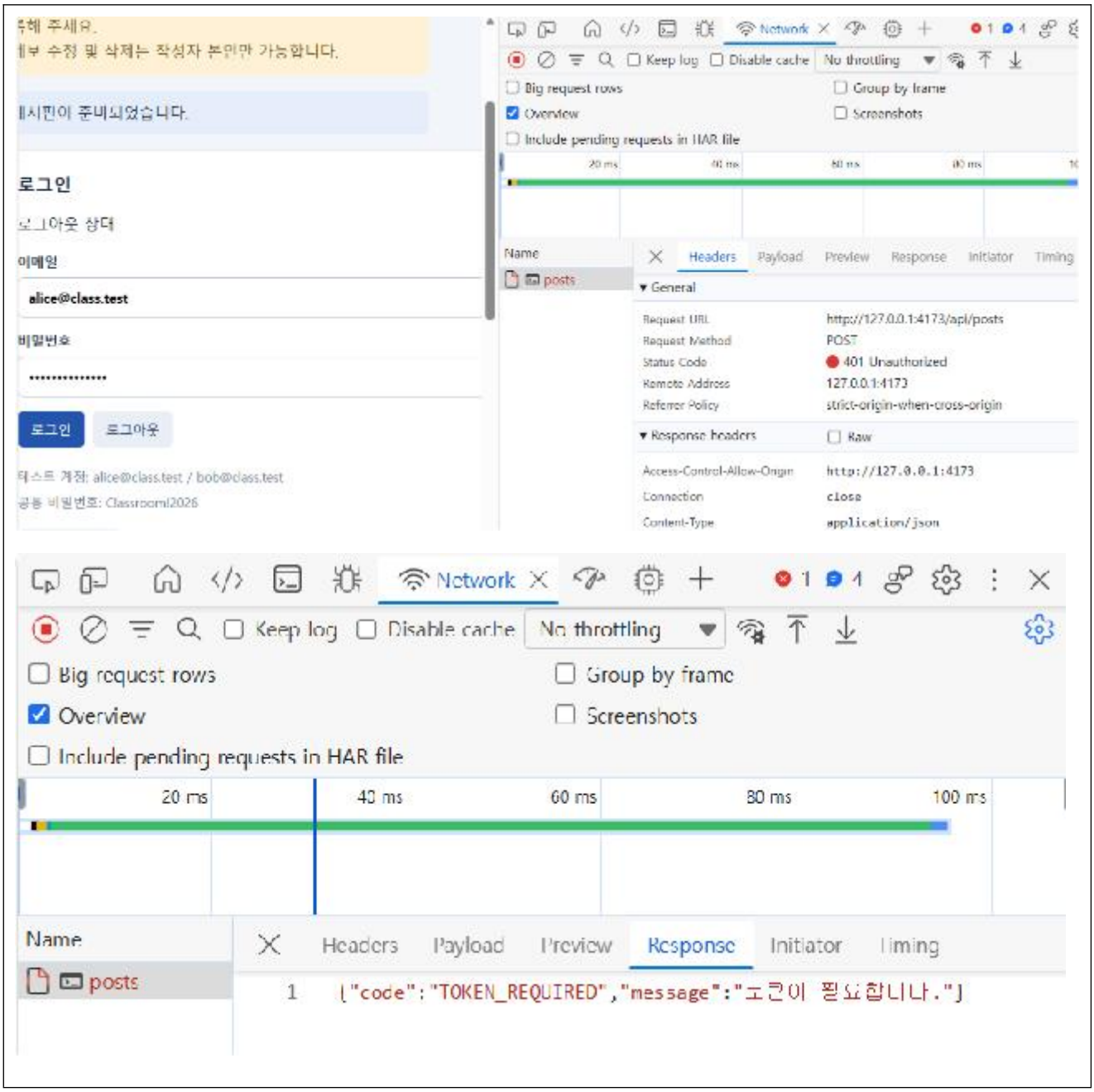

**[ T03 - 2 ] 없는 이메일로 각각 로그인**



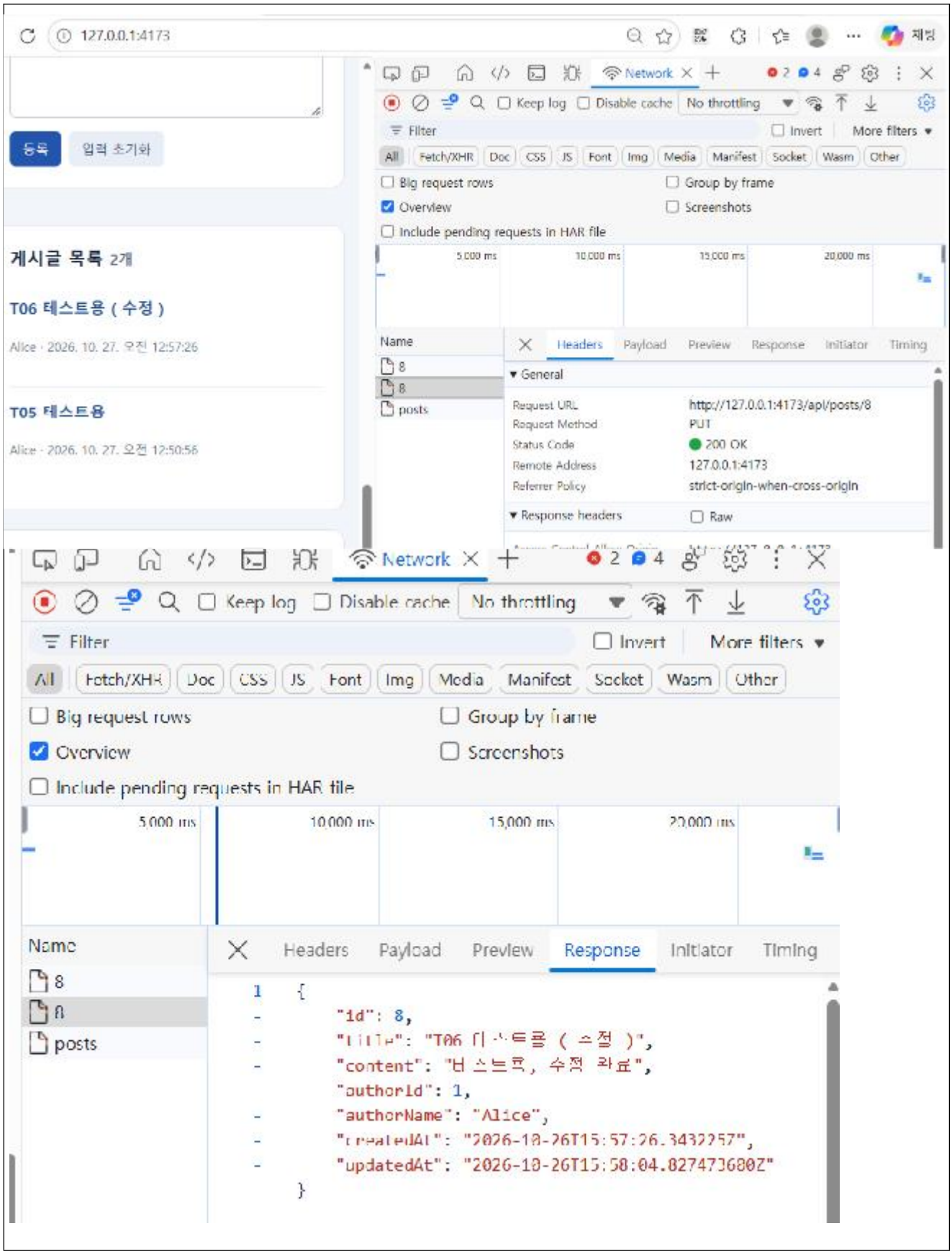

**[ T04 - 1 ] 토큰 없이 /me 요청 ( 헤더 및 응답 )**



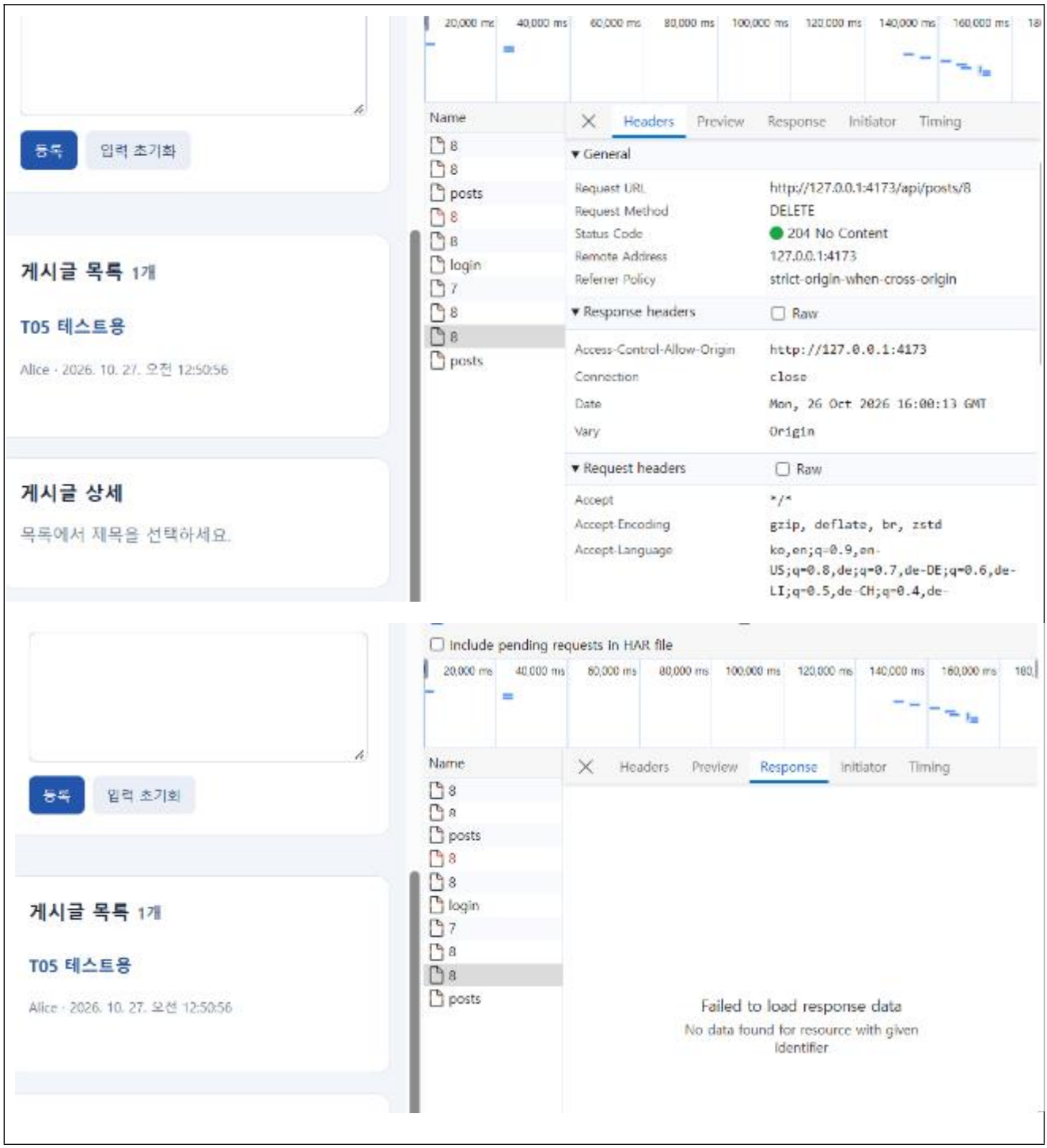

**[ T04 - 2 ] 토큰 없이 글 등록 요청 ( 헤더 및 응답 )**



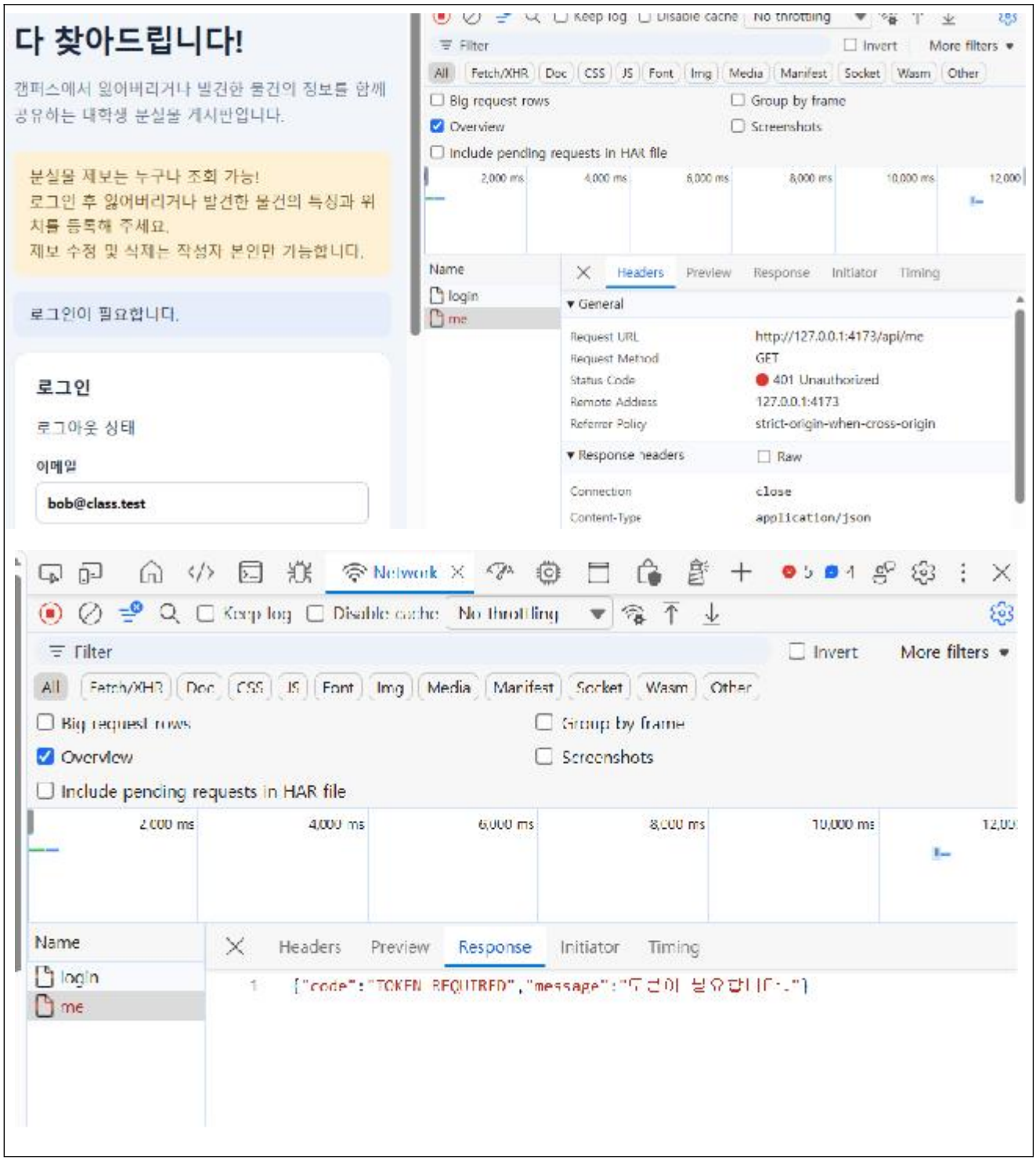

**[ T06 - 1 ] 작성자 본인의 게시글 수정 ( 헤더 및 응답 )**



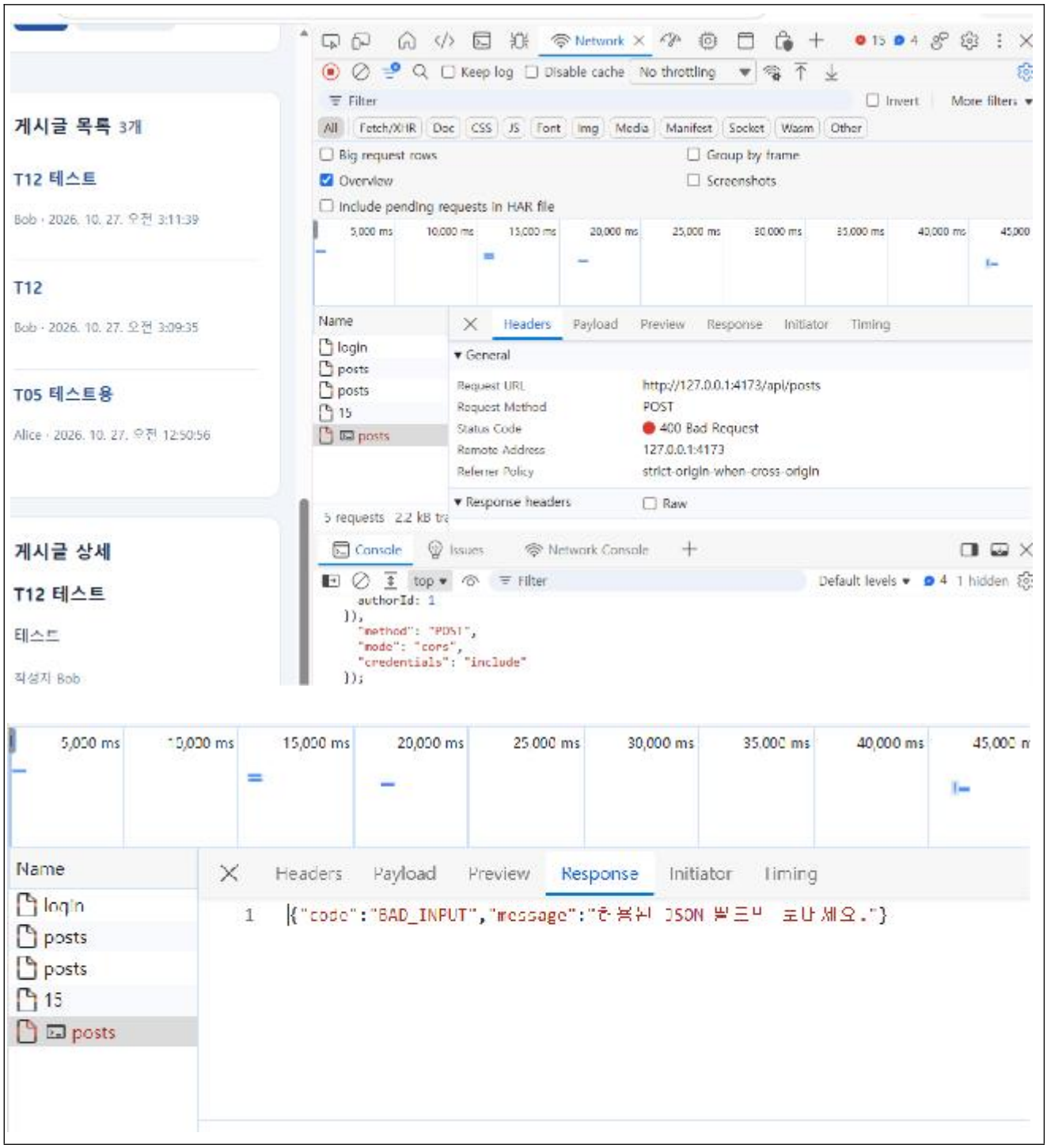

**[ T06 - 2 ] 작성자 본인의 게시글 삭제 ( 헤더 및 응답 )**




**[ T11 ] 로그인 후 로그아웃, 다음 /me 요청 ( 헤더 및 응답 )**



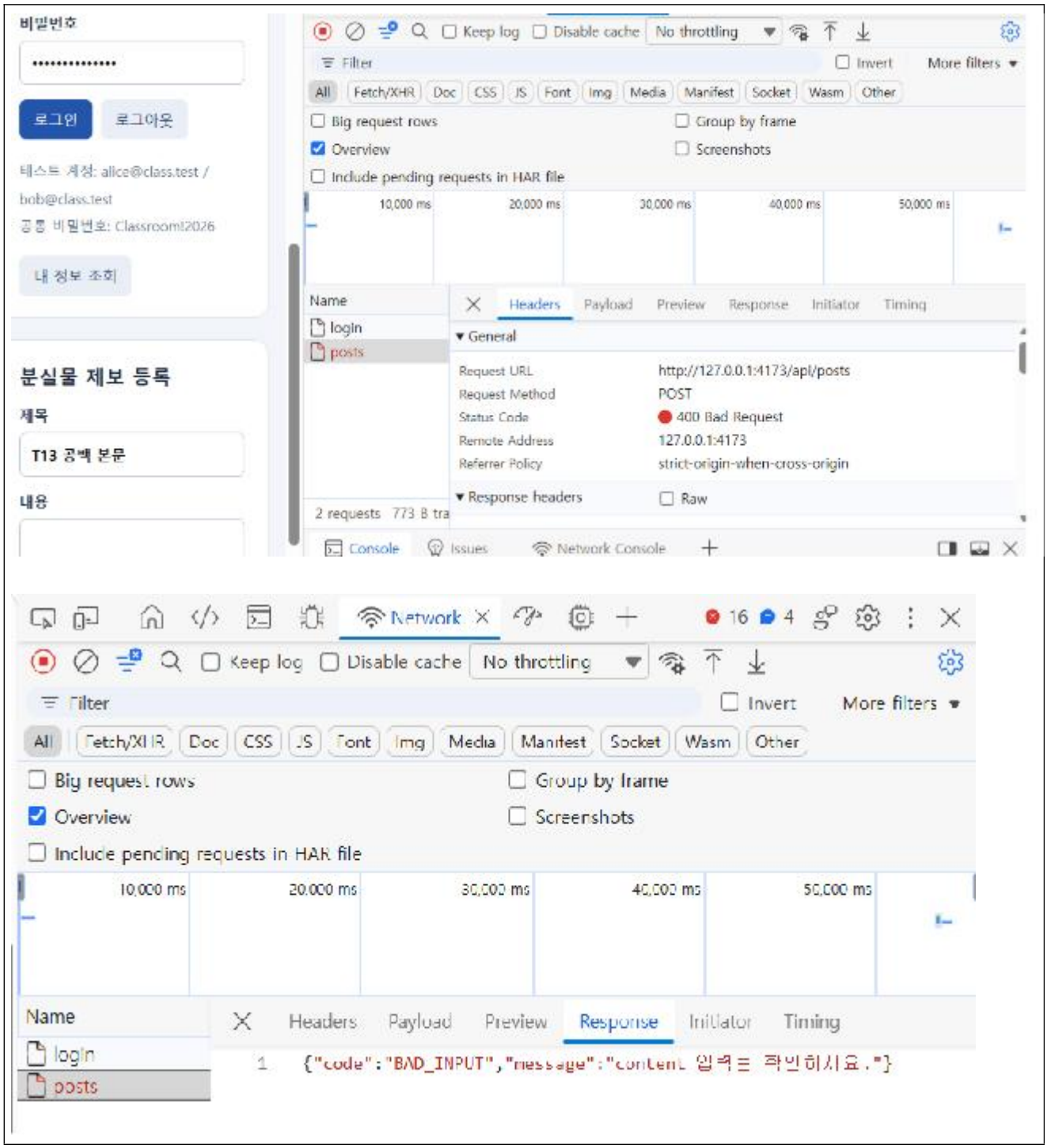

**[ T12 ] Bob 로그인 상태로 글 등록 요청에 authorId:1 또는 userId:1 추가 ( 헤더 및 응답 )**



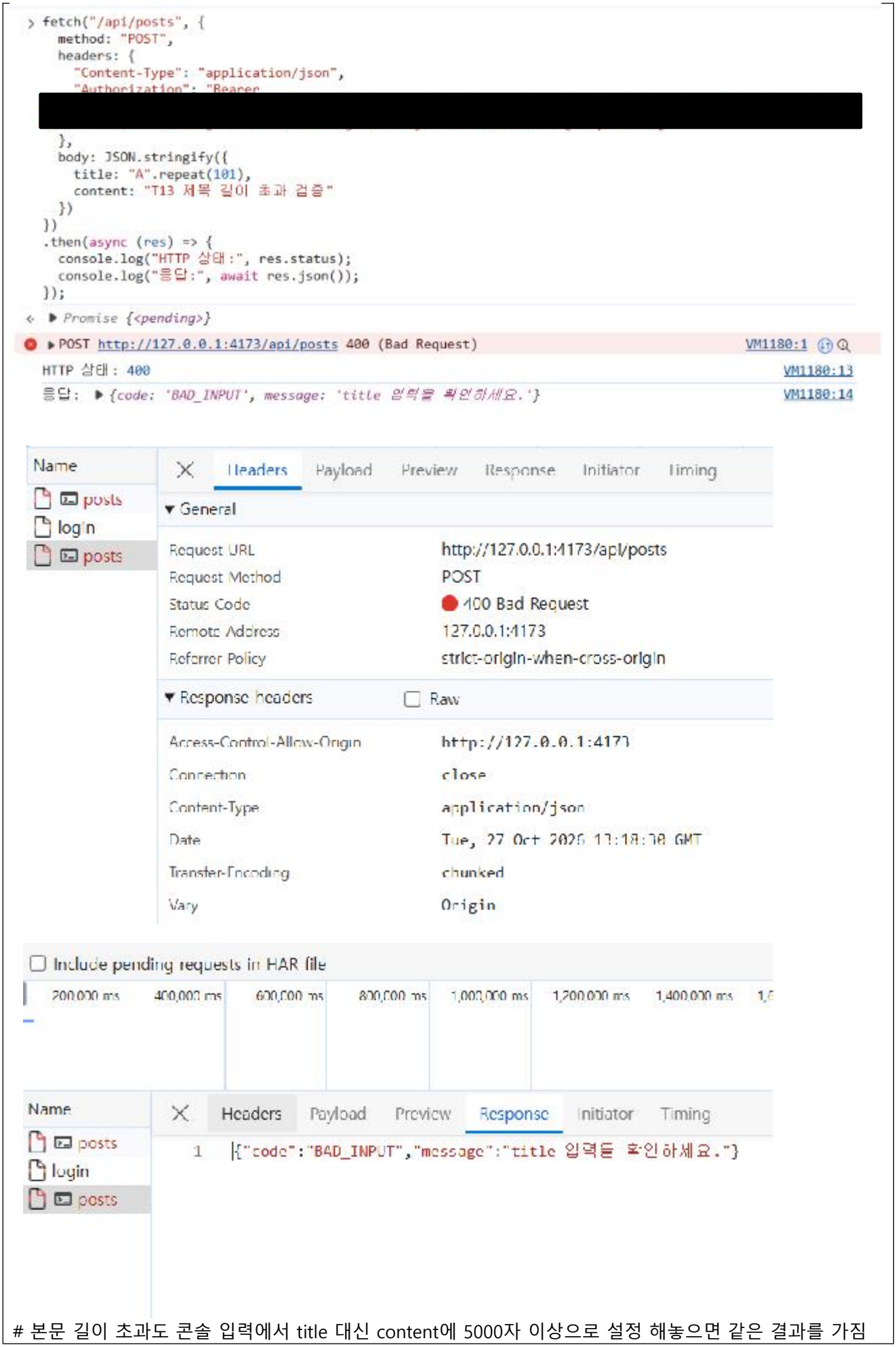

**[ T13 -1 ] 본인 토큰으로 빈 제목으로 전송**




**[ T13 -2 ] 본인 토큰으로 공백 본문 전송**

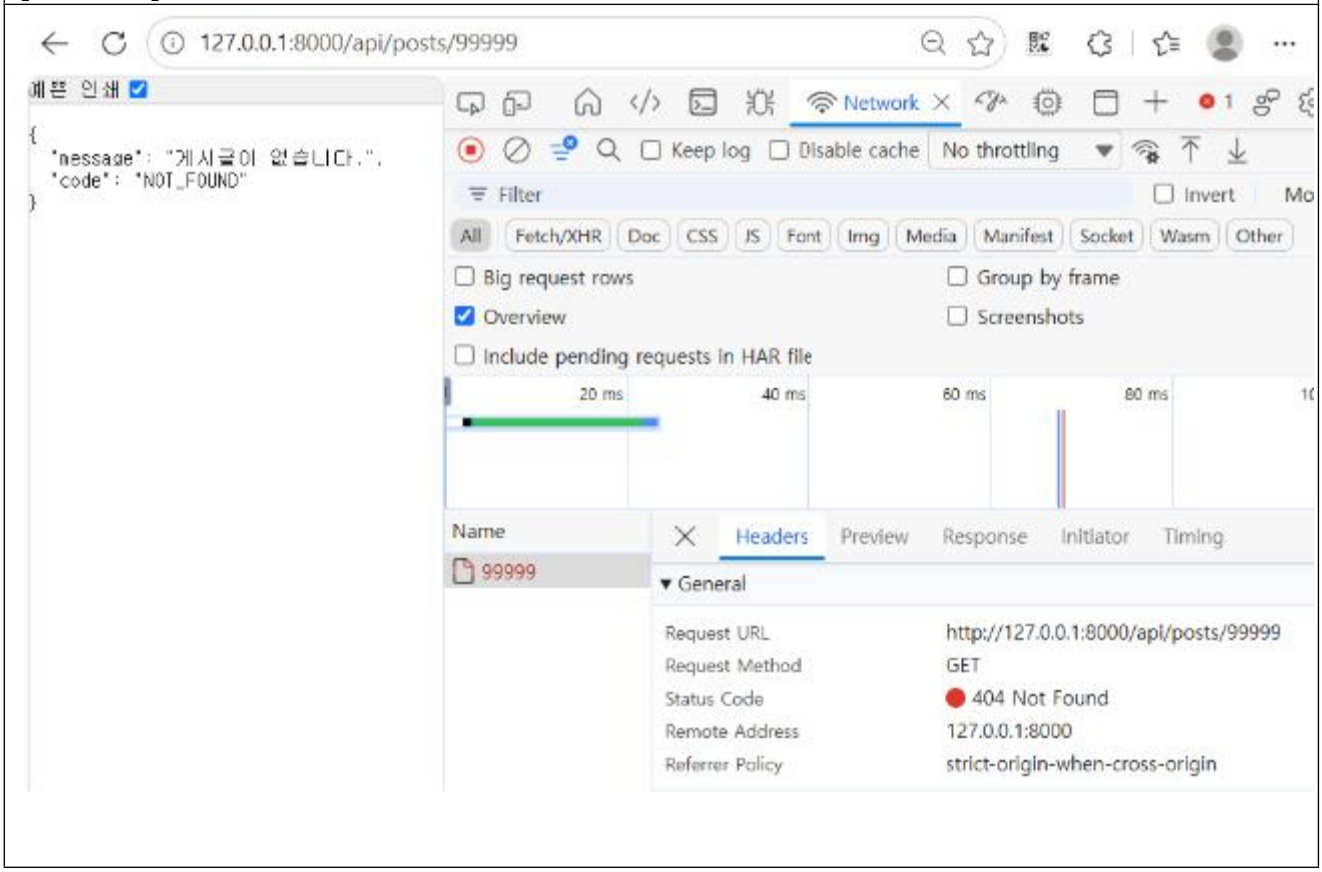



**[ T13 -3 ] 본인 토큰으로 (제목) 길이 초과**

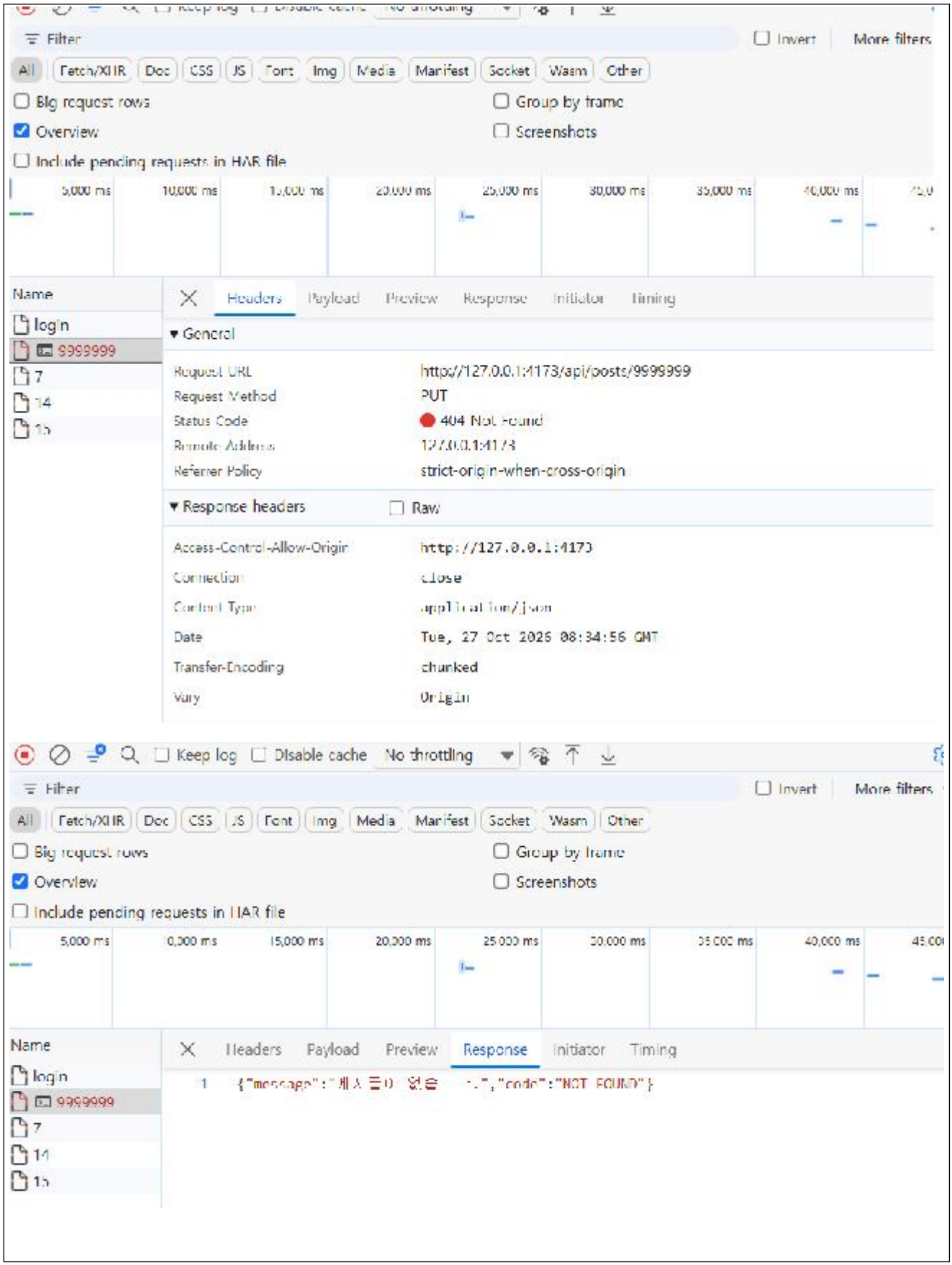



# 본문 길이 초과도 콘솔 입력에서 title 대신 content에 5000자 이상으로 설정 해놓으면 같은 결과를 가짐

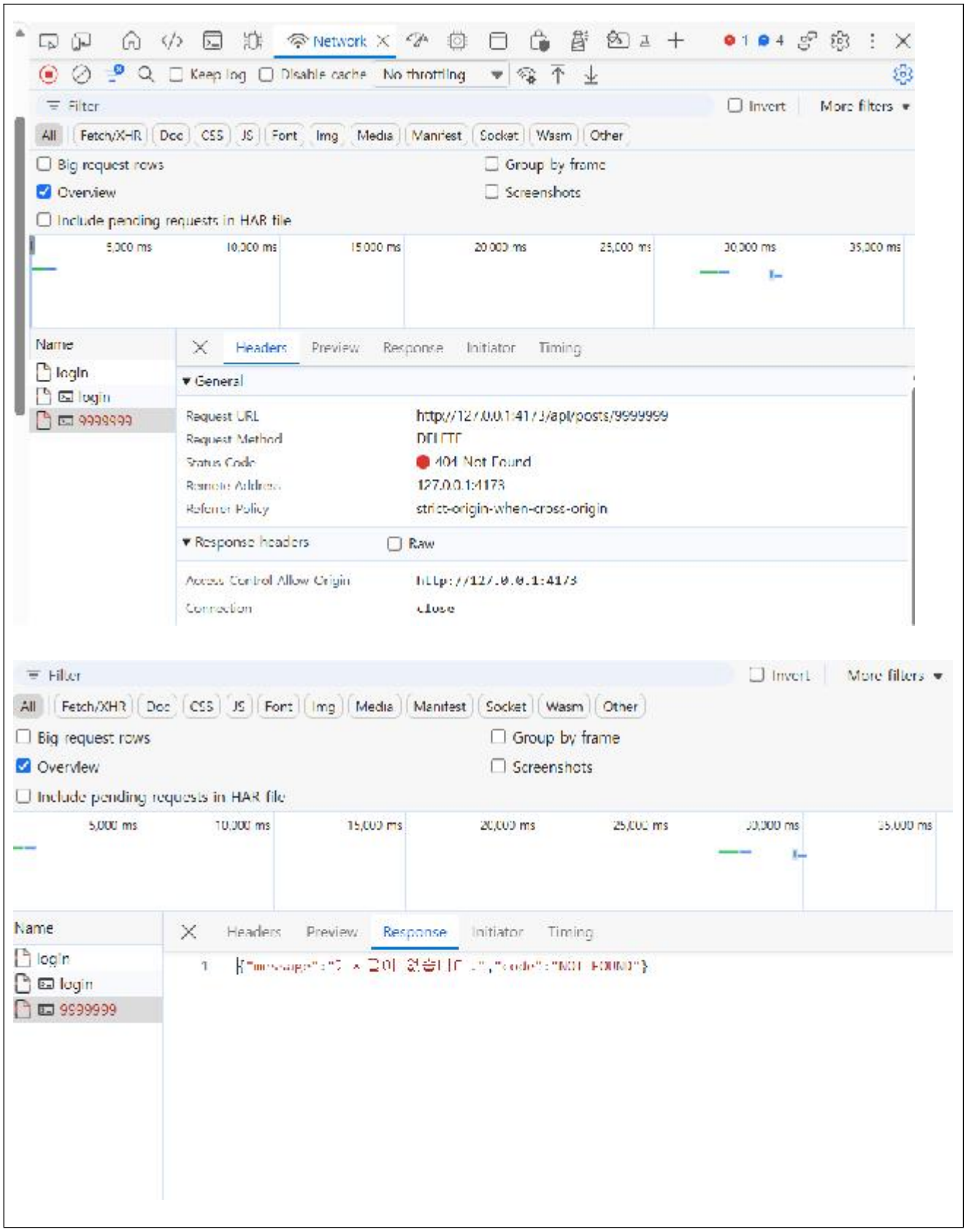
**[ T14 - 1 ] 없는 양의 정수 ID의 게시글 조회**



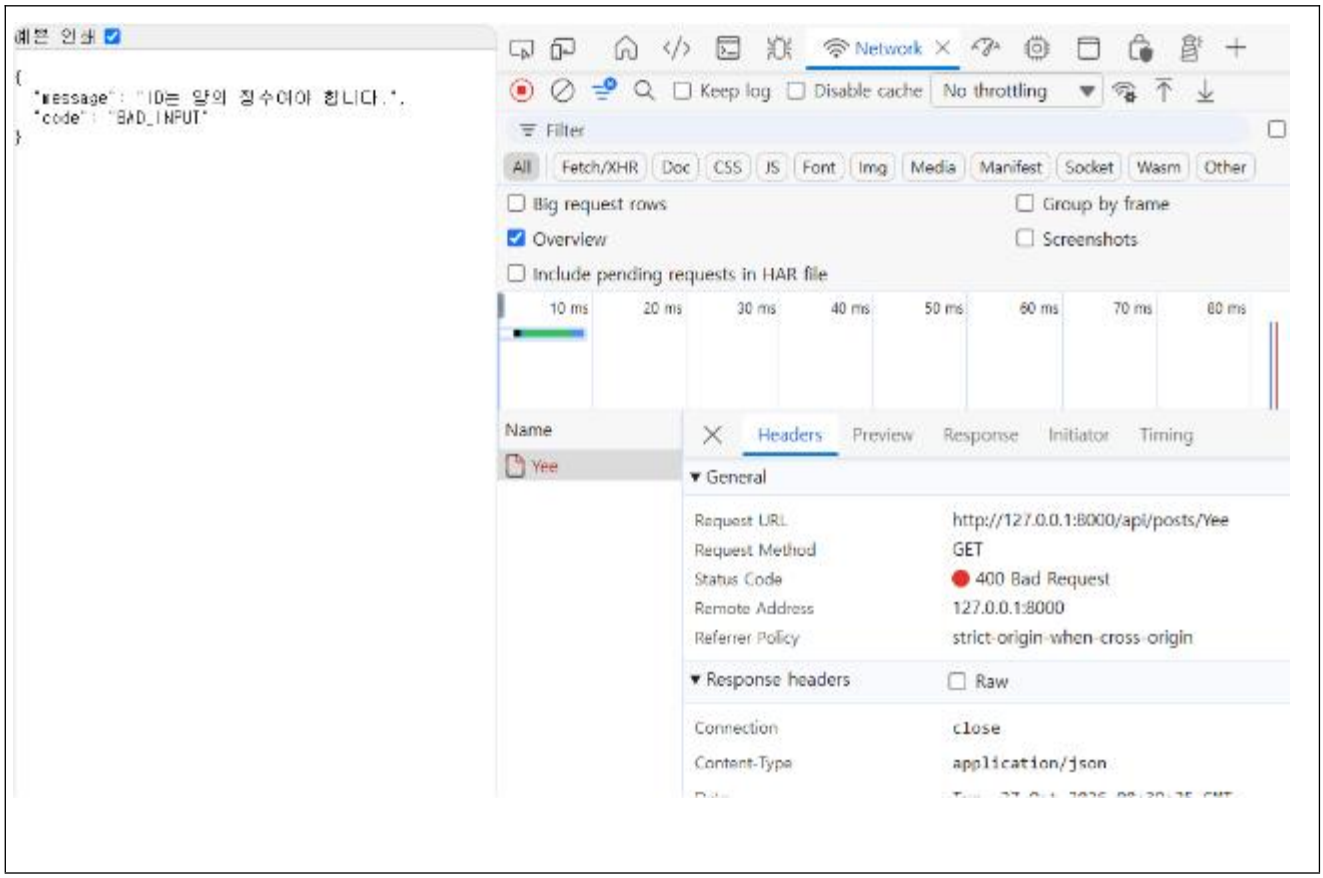

**[ T14 - 2 ] 없는 양의 정수 ID의 게시글 수정**



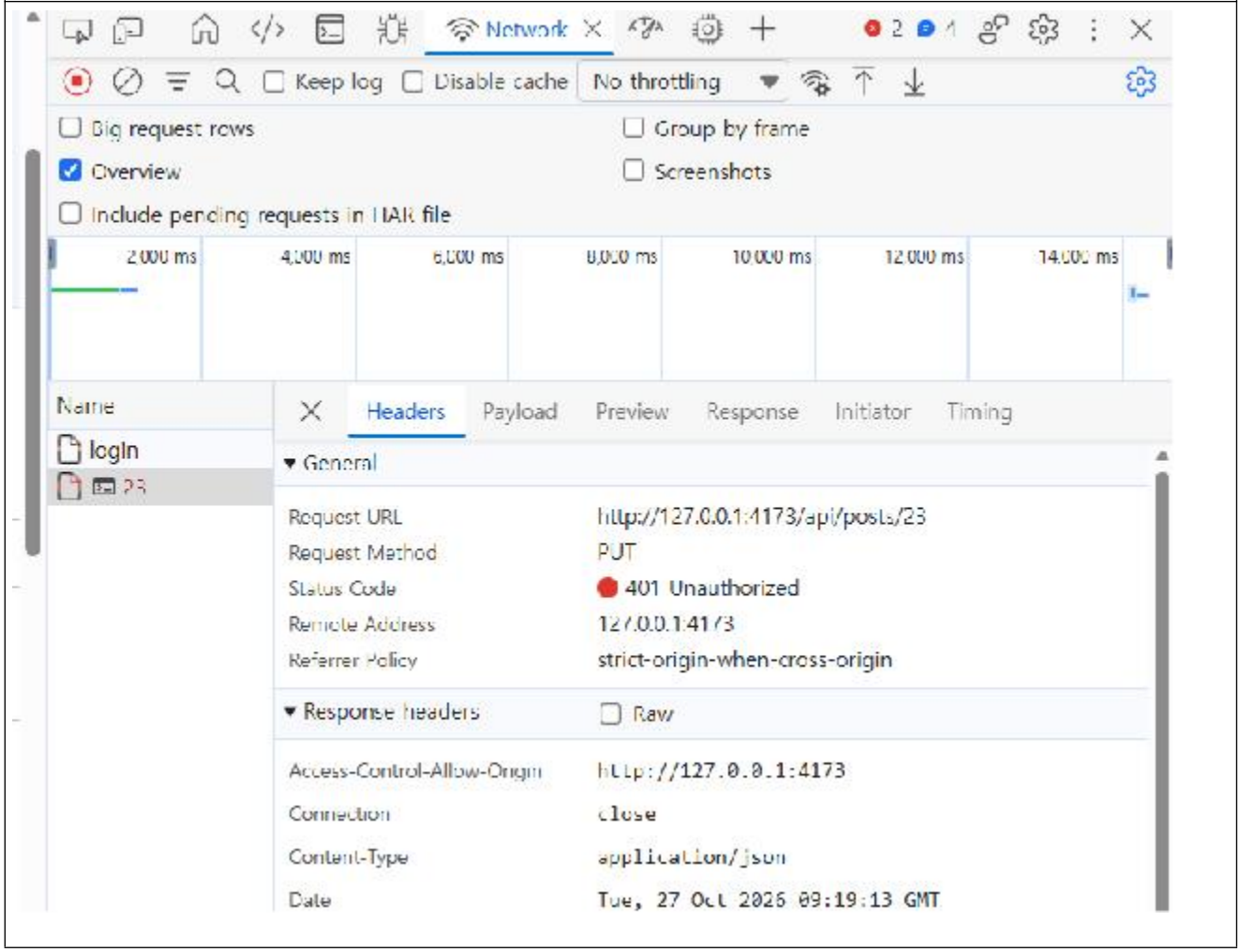

**[ T14 - 3 ] 없는 양의 정수 ID의 게시글 삭제 ( 헤더 및 응답 )**

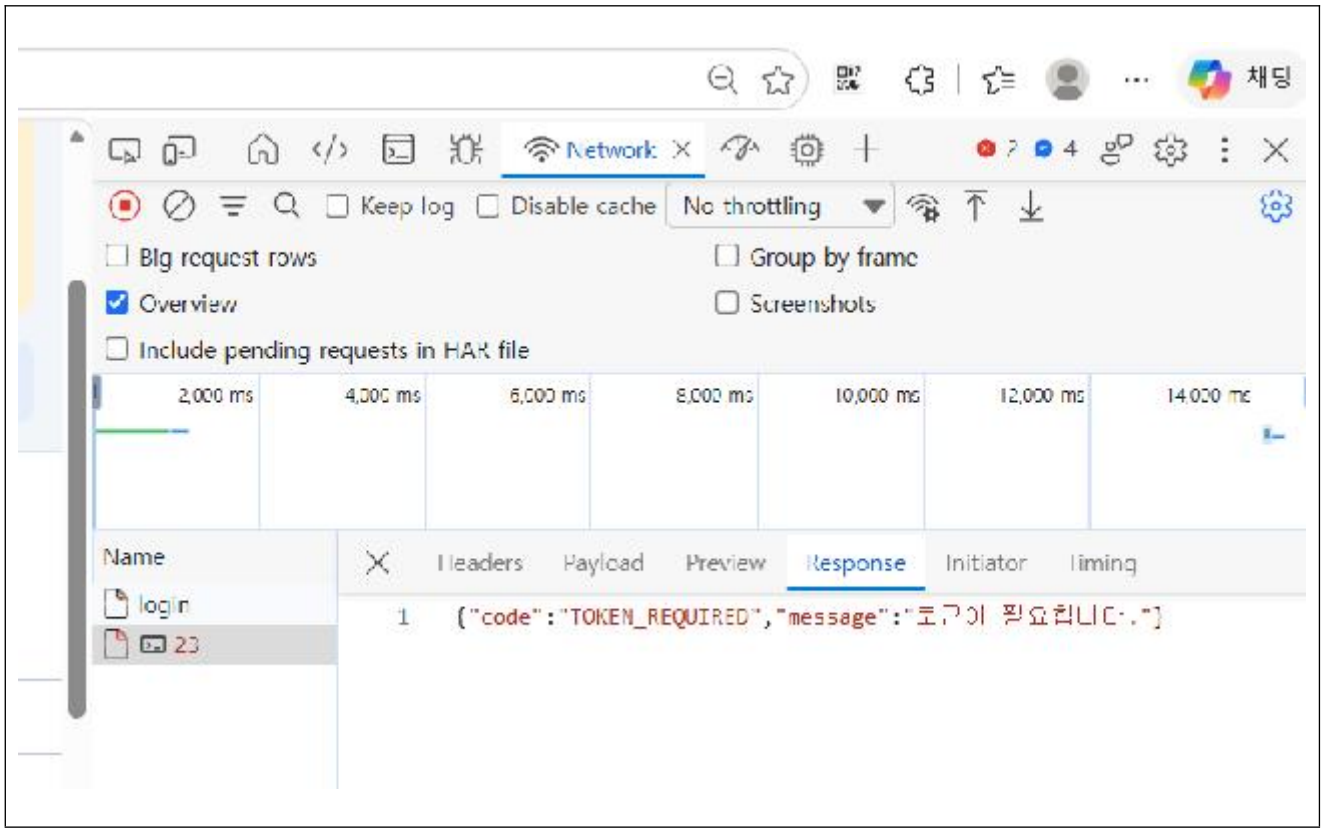



**[ T14 - 4 ] 잘못된 ID 조회**

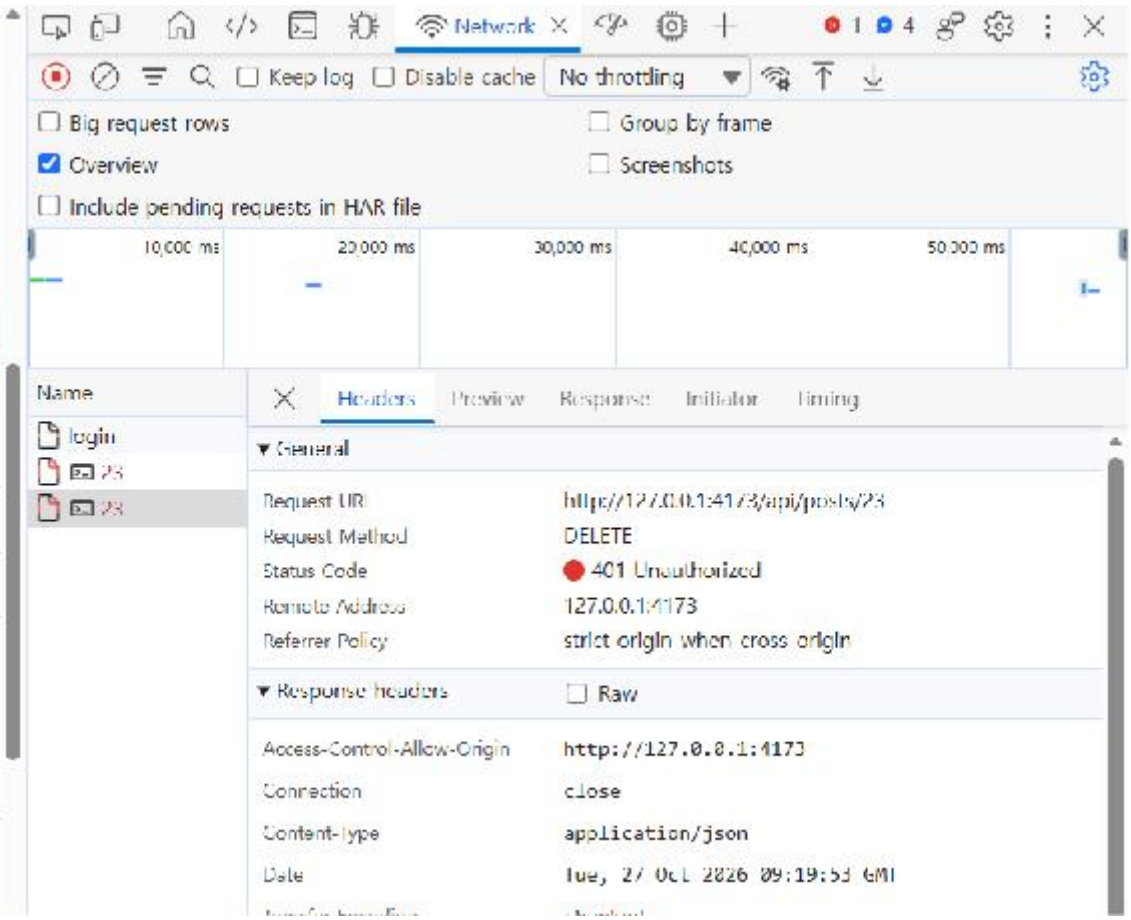



**[ T17 - 1 ] 토큰 없는 수정 요청 ( 헤더 및 응답 )**

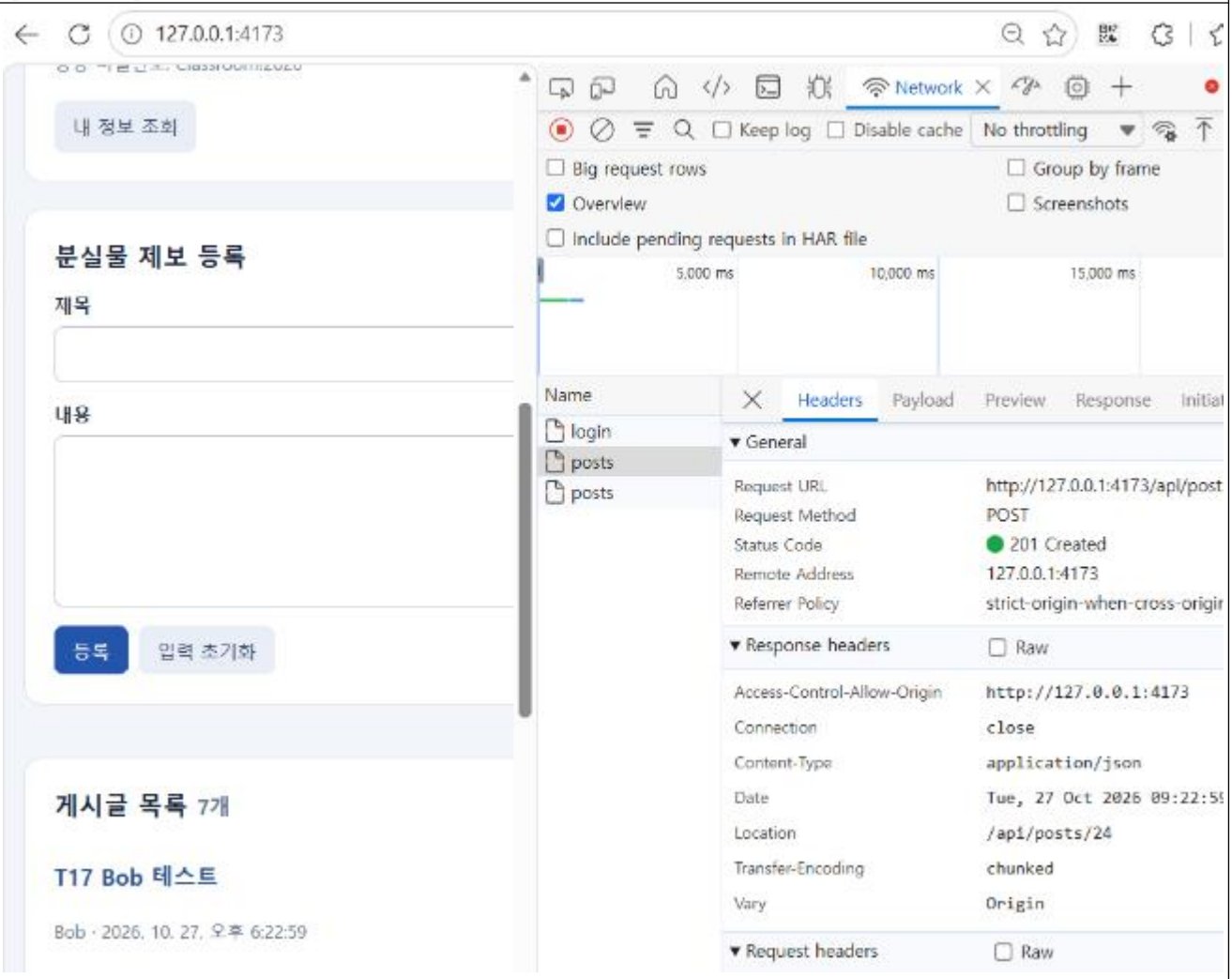




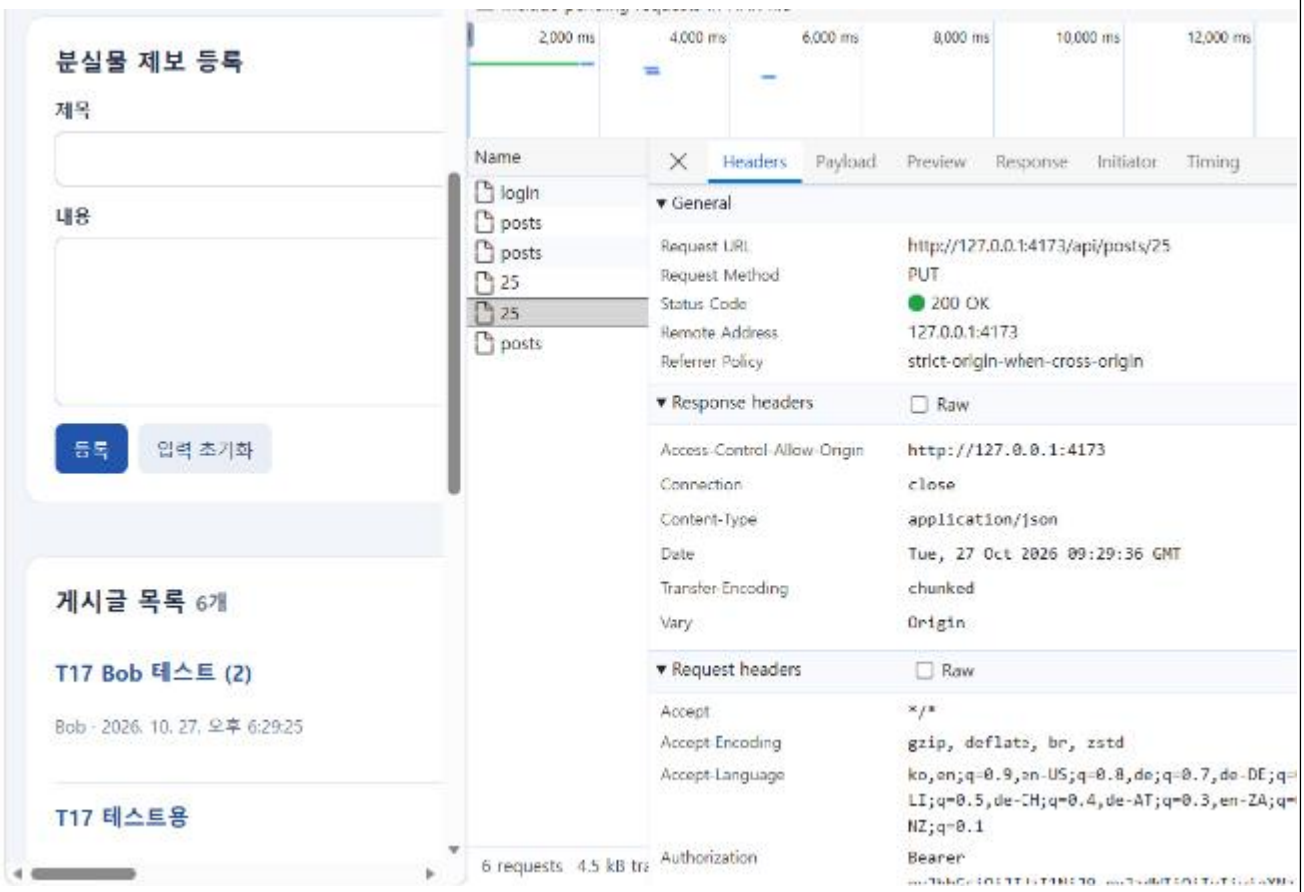

**[ T17 - 2 ] 토큰 없는 삭제 요청**

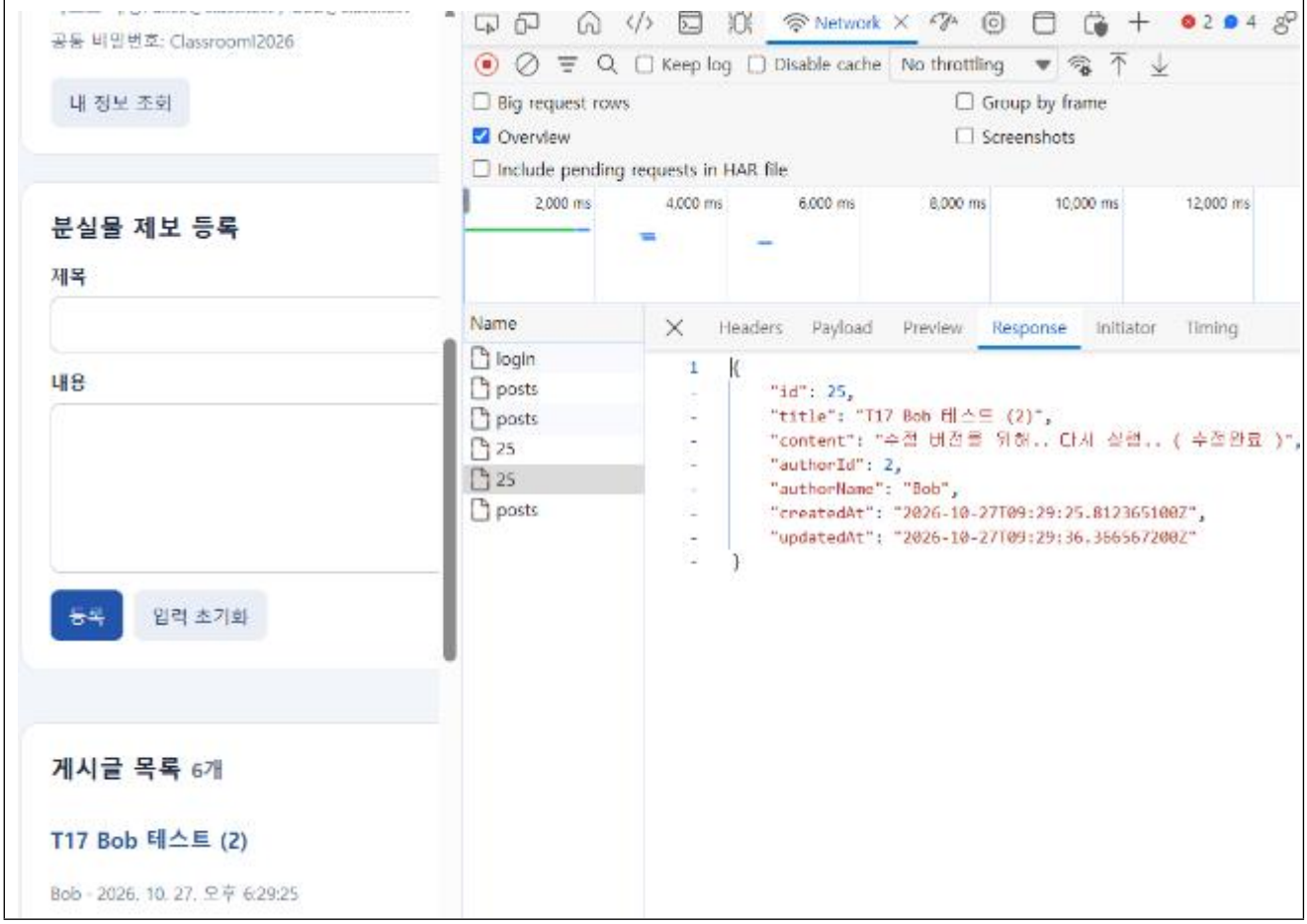


**[ T17 - 3 ] Bob 계정의 정상 작성 ( 헤더 및 응답 )**

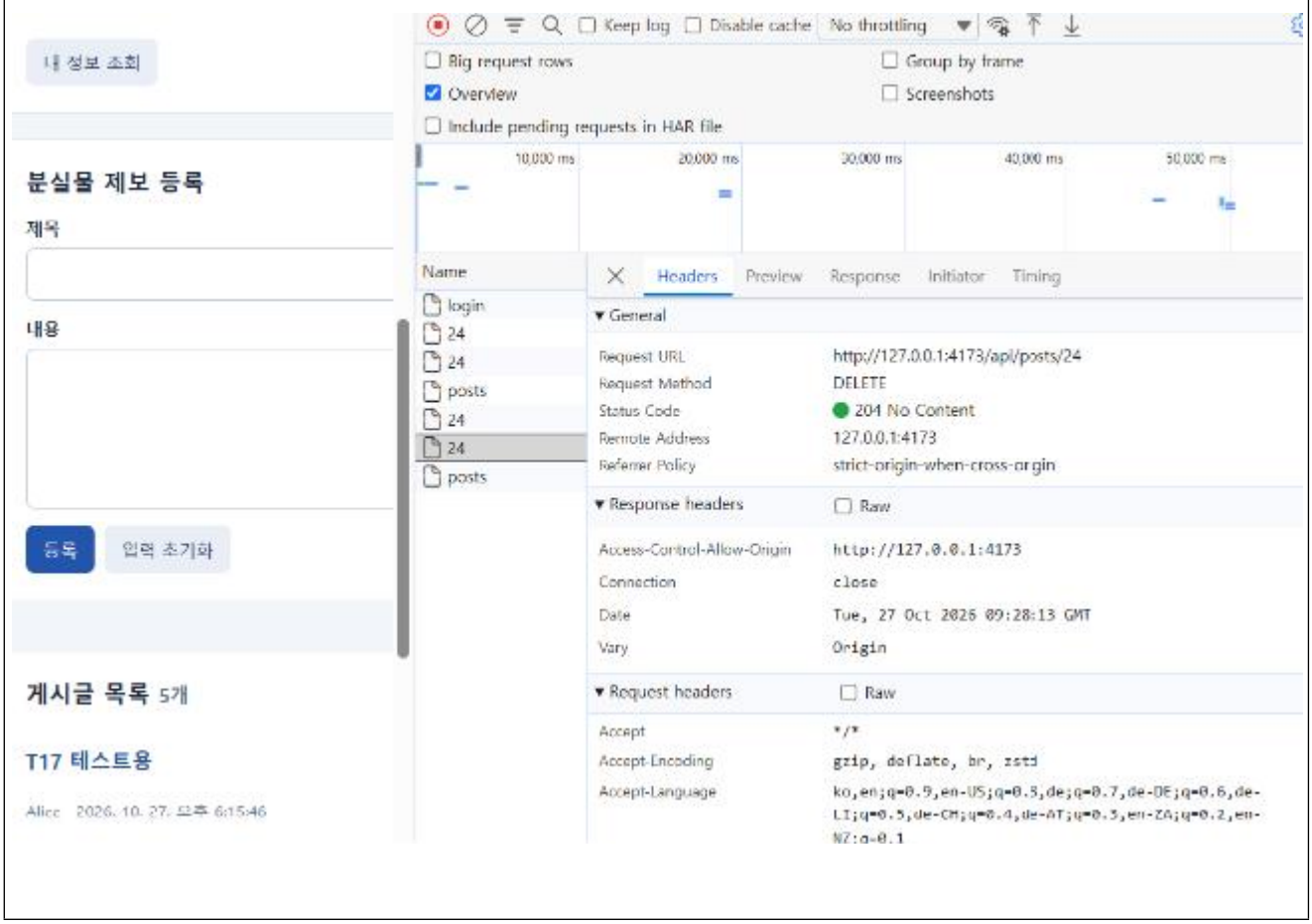


## [ T17 - 4 ] Bob 계정의 정상 게시글 수정 ( 헤더 및 응답 )



**[ T17 - 5 ] Bob 계정의 정상 게시글 삭제 ( 헤더 및 응답 )**




**웹서비스설계 05**

# AI 활용과 참고 자료

AI 사용 여부: 사용

사용하지 않았다면 미사용이라고 명시하고 참고한 수업 자료·공식 문서의 출처를 작성하세요. 사용했다면 전체 사용 범위를 요약하고 대표 사례 2~3건을 작성하세요. 사용이 1건이면 그 1건만 적습니다. 대화 전체 제출은 필요하지 않습니다.

| 도구와 모델 | 사용 목적 | 반영한 파일 또는 보고서 항목 | 반영 방식 |
|---|---|---|---|
| ChatGPT | [학습/설계/생성/디버깅/테스트] | AuthService.java 구현, api.js구현, 테스트 결과 과정 | AI의 설명과 코드 제안을 참고하여 구현했습니다. 실행 오류를 수정한 뒤 Postman과 Edge개발자 도구를 사용해 동작을 확인했습니다. |

## 1. JWT 발급 기능 구현

- 도구 / 날짜 : ChatGPT (GPT-6) / 2026.10.10

- 요청한 내용 : Spring Boot 환경에서 JWT 발급 기능을 구현하기 위해 기존 AuthService.java의 구조를 확인했는데, JWT 생성에 필요한 Java 문법과 import, JJWT 라이브러리의 사용 방법을 요청했습니다.

- AI가 제안한 내용 : JJWT 라이브러리의 Jwts.builder()를 사용해 JWT를 생성하는 방법과 Instant, HexFormat 등 필요한 Java 클래스의 역할을 설명해줬니다. 또한 @Value를 이용한 설정값 주입, HS256 서명 방법 등을 설명해줬습니다.

- 실제 반영 위치 : backend-spring/src/main/java/edu/classroom/board/AuthService.java

(1) AuthService() 생성자 = JWT 설정값과 비밀 키 검증

(2) issueToken() = JWT 클레임 설정, 만료 시간 계산 및 서명

- 내가 확인하고 수정한 내용 : 기존에 미구현 상태였던 JWT 발급 기능을 구현하면서 각 메서드의 역할과 필요한 import를 확인했습니다. 사용자 ID를 문자열로 변환하여 sub에 저장하고, 설정된 TTL을 이용해 만료시간을 계산하도록 적용했습니다.

- 검증 방법과 결과 : Spring Boot 서버를 실행한 뒤 PostMan에서 정상 계정으로 로그인 했을 때 HTTP 200 OK와 JWT 발급 여부를 확인했습니다. 이후 잘못된 비밀번호와 잘못된 입력 형식에 대해서도 각각 검증을 했을 때 401 INAVLID_CREDENTIALS, 400 BAD_INPUT 응답을 확인했습니다.

## 2. JWT 검증 및 사용자 인증 기능 구현

- 도구 / 날짜 : ChatGPT (GPT-6) / 2026.10.10

- 요청한 내용 : Spring Boot 환경에서 JWT 검증 기능 구현하기 위해 Authorization 헤더에서 Bearer 토큰을 추출하는 Java 문법과 JJWT 라이브러리를 이용한 서명 및 만료 시간 검증 방법을 질문했습니다. 또한 JWT에 저장된 사용자 정보를 확인하는 Claims 객체와 예외 처리 문법의 사용 방법에 대한 설명 요청을 했습니다.

- AI가 제안한 내용 : HttpServletRequest.getHeader()를 이용해 Authorization 헤더를 읽는 방법과 regionMatches(), substring()을 이용해 Bearer 토큰을 추출하는 방법을 설명해줬습니다. 또한 JJWT 라이브러리의 Jwts.parser(), verifyWith(), parseSignedClaims()를 이용한 JWT 검증하는 과정에 사용되며 해당 문법들이 각각 무슨 역할을 하고 어떨 때 사용해야 하는지에 대해 설명해줬습니다. 또한 Claims 객체에 대한 이해도가 부족하여 추가적인 설명을 해줬습니다. 마지막으로 ExpiredJwtException,JwtException을 이용해 토큰 만료 및 검증 실패 상황을 구분하는 방법을 설명해줬습니다.

- 실제 반영 위치 : backend - spring/src/main/java/edu/classroom/board/AuthService.java

(1) currentUserId() = Authorization 헤더에서 Bearer 토큰 추출 및 JWT 검증

(2) Claims = JWT의 발급자, 사용 대상, 만료 시각, 사용자 ID 등 필수 정보 확인

(3) try-catch = 토큰 만료 및 유효하지 않은 토큰에 대한 예외 처리

- 내가 확인하고 수정한 내용 : 처음에 Authorization 헤더에서 Bearer 토큰을 추출하고 ,JWT의 서명과 만료 여부를 검증하는 방식으로 코드를 구성했습니다. 이후 AI에게 Java의 문자열 처리 메서드와 JJWT 라이브러리의 검증 방법을 질문하면서 각 코드의 역할을 확인했습니다. 이 과정에서 서명과 만료 여부뿐만 아니라 발급자(iss), 사용 대상(aud), 사용자 ID(sub) 등의 필수 클레임도 확인해야 한다는 점을 이해하고 검증 조건을 보완했습니다. 또한 사용자 ID의 정규표현식에서 *가 누락된 부분과 INVALID_TOKEN 오류 코드의 오타를 수정하여 정상적인 사용자 ID를 처리하고 오류 응답을 일관되게 반환하도록 했습니다.

- 검증 방법과 결과: Spring Boot 서버를 실행한 뒤 Postman을 사용하여 GET/api/me 요청에 대한 JWT 인증 기능을 검증했습니다. 정상 계정으로 로그인하여 발급받은 JWT를 Authorization 헤더에 Bearer 형식으로 전달하고, 사용자 정보가 정상적으로 반환되는지 확인했습니다. 또한 토큰이 없는 경우, 잘못된

토큰을 전달한 경우, 만료된 토큰을 전달한 경우를 각각 테스트를 통해 HTTP 상태 코드와 JSON 오류 코드를 확인했습니다.

## 3 . 로그인 요청의 JSON 데이터 변환

- 도구 / 날짜 : ChatGPT / 2026.10.11

- 요청한 내용 : Vue에서 로그인 API를 구현하는 과정에서 이메일과 비밀번호는 JSON 형식으로 서버에 전달하는 과정에서 어떤 문법을 사용하여 구현해야하는지에 대해 질문했습니다.

- AI가 제안한 내용 : JSON.stringify()가 JavaScript 객체를 JSON 문자열로 변환하는 함수라는 것을 설명해줬습니다. 또한 fetch()의 POST 요청에서 body에 JavaScript 객체를 그대로 전달하는 것이 아닌 JSON.stringify({ email : email, password: password }) 형태로 작성하여 JSON 문자열을 전달하는 방법을 제안해줬습니다.

- 실제 반영 위치 : frontend-vue/src/api.js

(1) login() : 이메일과 비밀번호는 JSON.stringify()를 통해 JSON 문자로 변환해 POST 요청의 body에 전달

(2) request() : 로그인 요청을 서버에 전송, HTTP 응답 및 오류 처리

(3) login() 내 session 저장 : 로그인 성공 시 반환된 JWT와 사용자 정보를 JavaScript 메모리에 저장

- 내가 확인하고 수정한 내용 : AI가 설명한 JSON.stringify()의 사용 방법을 확인하고, request() 함수를 활용해 로그인 요청의 body 부분을 구현했습니다. 이메일과 비밀번호를 하나의 JavaScript 객체로 구성한 뒤 JSON 문자열로 변환하도록 작성했습니다.

- 검증 방법과 결과 : Vue 로그인 화면에서 정상 계정의 이메일과 비밀번호를 입력하여 로그인을 시도했으며, 서버로부터 로그인 성공 응답을 받아 사용자 이름이 화면에 표시되는 것은 확인했습니다. 또한 잘못된 비밀번호를 입력한 경우에는 로그인 되지 않고 "이메일 또는 비밀번호를 확인하세요" 라는 오류 메시지가 표시되는 것을 확인했습니다. 이를 통해 JSON 형식의 로그인 요청이 서버에 정상적으로 전달되고 처리되는 것을 확인했습니다.

## 4. JWT 서명 변조 및 토큰의 인증 오류 검증 (T09, T10)

- 도구 / 날짜 : ChatGPT / 2026.10.11

- 요청한 내용 : JWT 인증 오류를 검증하는 T09 테스트를 수행하는 과정에서, 유효한 JWT의 서명 부분 첫 글자를 다른 문자로 변경하는 구현 방법을 잘 알지 못해 질문했습니다. 실제 발급 받은 토큰을 어떻게 변조해야 하는지 단계별로 설명해 달라고 요청했습니다. 또한 변조된 토큰으로 /api/me 와 /api/posts를 수행할 때 어떤 식으로 구현해야 하는지에 대해 추가로 질문했습니다. 이후 T10에서는 JWT의 유효 시간이 지난 뒤 /api/me 요청과 게시글 변경 요청을 수행하는 방법을 질문했습니다. 더불어 Vue 화면에서 로그아웃 처리된 이후에도 만료된 토큰으로 게시글 등록 요청을 보내려면 어떻게 해야하는지, Console을 사용하지 않아도 Edge 개발자 도구에서 테스트 해볼 수 있는 방법이 있는지에 대해서 질문 했습니다.

- AI가 제안한 내용 : T09에서는 JWT가 점(.)으로 구분된 Header, Payload, Signature의 세 부분으로 구성되어 있으며, 마지막 Signature 부분의 첫 글자맞 변경하는 방법을 설명해줬습니다. JavaScript의 split('.')을 이용해 JWT를 세 부분으로 나눈 뒤, 세 번째 부분의 첫 글자를 다른 문자로 변경하고 join('.')으로 다시 결합하는 방법을 제안해줬습니다. 또한 Vue 화면의 '내 정보 조회' 버튼은 기존 정상 토큰을 사용할 수 있으므로 Edge 개발자의 도구에 들어가서 콘솔에 fetch()를 이용하여 변조된 JWT를 Authorization 헤더에 직접 설정하여 API 요청을 전송하는 방법을 안내해 줬습니다. T10에서는 JWT의 TTL이 지난 뒤 기존 토큰을 사용해 API 요청을 수행하는 방법을 안내해줬습니다. /api/me 요청에서 토큰 만료 오류가 발생하면

Vue 화면에서 로그아웃 상태와 만료 안내를 확인하도록 했으며, 변경 요청을 Edge 개발자 도구에서 기존 HTTP 요청을 재전송하는 방법을 제안했습니다. 또한 게시글 등록 요청의 body가 null로 설정되어 있는 것을 확인하여 JSON.stringify()를 이용해 제목과 내용을 포함한 JSON 데이터로 변경하는 방법을 안내해줬습니다.

- 실제 반영 위치 : Edge 개발자 도구의 Console 및 Network ( T09, T10 테스트 수행 )

(1) (T09) JWT 서명 변조 : 로그인 응답에서 발급받은 accessToken을 split('.')으로 분리한 뒤, 세 번째 부분인 Signature의 첫 글자만 다른 문자로 변경하도록 구현

(2) (T09) GET /api/me : fetch()의 Authorization 헤더에 변조된 JWT를 설정해 내 정보 조회 요청을 전송

(3) (T09) POST /api/posts : 동일한 변조 JWT를 사용해 게시글 등록 요청을 전송

(4) (T10) 변경 요청 : Edge 개발자 도구에서 기존 요청을 재전송하는 과정에서 POST 요청의 body를 null에서 JSON.stringify({title:"T10 테스트용", content:"만료된 JWT로 게시글 등록 시도"})로 변경. 이후 만료된 JWT를 이용해 POST /api/posts 요청을 전송

=> 해당 테스트는 브라우저 개발자 도구에서 테스트용 JavaScript 코드를 실행한 것입니다.

- 내가 확인하고 수정한 내용 : T09에서는 AI가 설명한 JWT의 구조를 확인하고, 정상적으로 발급받은 WJT에서 Header와 Payload는 유지한 상태로 Signature의 첫 글자만 변경했습니다. 이후 변조된 JWT를 modifiedToken 변수에 저장했으며 fetch() 요청의 Authorization 헤더에 Bearer ${modifiedToken} 형식으로 전달하도록 작성했습니다. T10에서는 정상 로그인 후 JWT의 유효시간이 지난 상태에서 '내 정보 조회' 버튼을 눌러 만료 오류가 발생하는 것을 확인했습니다. 이후 만료된 토큰으로 게시글 등록 요청을 수행하기 위해 Edge 개발자 도구의 요청 재전송 기능을 활용했습니다. 이 과정에서 처음 생성된 요청 코드의 body가 null로 되어 있어, AI의 안내를 참고하여 제목과 내용을 포함한 JSON 문자열로 직접 수정했습니다.

- 검증 방법과 결과 : T09에서는 Bob 계정으로 정상 로그인한 후 발급된 JWT의 서명 첫 글자를 변경하고, 변조된 토큰을 이용해 GET/api/me 과 POST/api/posts 요청을 전송해 수행했으며 두 상황 모두 상태 코드가 401 Unauthorized와 JSON 오류 코드로 INVALIE_TOKEN이 반환되는 것을 확인했습니다. T10에서는 정상적으로 발급된 JWT가 실제 만료된 이후 GET /api/me 요청을 수행했으며, HTTP 401과 JSON 오류 코드 TOKEN_EXPIRED가 반환되는 것을 확인했습니다. 이와 함께 Vue 화면에서 로그아웃 상태로 변경되고 "로그인 시간이 만료되었습니다. 다시 로그인하세요."라는 안내가 표시되는 것을 확인했습니다. 이후 만료된 JWT로 POST /api/posts 요청을 수행했으며, 동일하게 HTTP 401 / TOKEN_EXPIRED가 반환되는 것을 확인했습니다.

- 추가 응용 및 검증 (T1, T7, T8, T12, T13, T14)

: T09와 T10에서 JWT를 활용한 인증 오류 테스트를 진행하면서 HTTP 요청에 토큰을 포함하는 방법과 서버 응답을 확인하는 과정을 학습한 다음에 해당 테스트 방법을 T12에 해당하는 테스트인 작성자 위조 방지 검증에 응용하기 위해 AI에게 요청 본문에 다른 사용자의 authorId를 포함하는 방법을 질문하였고 AI가 제안한 방법을 참고해 Bob의 정상 JWT로 다른 사용자의 authorId를 포함한 게시글 작성 요청을 직접 전송했으며, 서버에서 400 BAD_REQUEST와 BAD_INPUT이 반환되는 것을 확인했습니다. 또한 T14에서는 T09와 T10에서 학습한 JWT인증 및 fetch() 요청 방법을 기반으로 PUT과 DELETE 요청에 필요한 변경 사항을 추가적으로 작성하여 실행해 두 요청이 모두 404 NOT_FOUND가 반환되는 것을 확인했습니다. 마지막으로 T01과 T07, T08에서도 기존에 학습한 JWT 인증 및 fetch() 요청 방법을 응용했습니다. 이를 통해 기존 테스트 방법을 새로운 보안 검증 상황에 응용하고, 실제 서버 응답을 통해 결과를 검증하는 과정과 기존에 미실행했던 서버 권한 검증을 수행할 수 있었습니다.

## 참고 자료

| 자료 제목과 URL | 사용한 위치 | 반영 내용 및 출처 표시 |
| --- | --- | --- |
| [수업 자료 포함] | [파일/함수] | [작성] |

# 이해도 확인과 회고

각 질문에 본인 코드의 동작을 근거로 답하세요.

1. 요청 본문의 authorId를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

: 요청 본문의 authorId를 그대로 신뢰하게 되면 사용자가 다른 사람의 ID를 임의로 입력하여 타인의 이름으로 게시글을 작성하는 작성자 위조 문제가 발생할 수 있습니다. 현재 작성한 코드에서는 App.vue의 save() 함수를 통해 게시글의 제목과 내용만 요청 본문에 포함하고, api.js의 authorizationHeader() 함수를 통해 JWT를 Authorization 헤더에 첨부하도록 구현한 상태입니다. 서버에서는 AuthService.java의 currentUserId() 함수가 JWT의 서명과 클레임을 검증해 실제 로그인 사용자의 ID를 반환하도록 합니다. 이후 BoardConrtroller.jvav에서 해당 사용자 ID를 게시글 작성자로 사용해, fields() 함수로 허용되지 않은 JSON 필드를 검사합니다. 실제로 T12에서 Bob의 정상 JWT를 사용해보면서 요청 본문에 Alice의 authorId:1을 추가해 게시글 등록을 시도해봤을 때 HTTP 400 Bad Request와 JSON오류 코드인 BAD_INPUT이 반환되어 작성자 위조 요청이 거부되는 것을 확인했습니다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

: 사용자가 브라우저 개발자 도구나 API 요청을 통해 화면의 버튼을 거치지 않고 서버에 직접 수정 및 삭제 요청을 보낼 수 있기 때문에 게시글의 수정 및 삭제 버튼을 숨기는 것만으로는 충분하지 않습니다. 이러한 문제를 방지하기 위해 AuthService.java의 assertOwner() 함수에서 인증된 사용자 ID(userId)와 게시글 작성자 ID(authorId)를 비교하도록 구현했습니다. 이 구현을 통해 두 값이 다르면 HTTP 403과 JSON 오류 코드로 FORBIDDEN을 반환해 요청을 거부하도록 했습니다. 실제로 T07과 T08에서 Bob의 정상 JWT로 Alice의 게시글을 수정 및 삭제하려고 시도한 결과, 두 요청 모두 HTTP 403과 오류 코드로 FORBIDDEN 응답이 반환되는 것을 직접 확인해볼 수 있었습니다.

3. JWT decode와 verify의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

: JWT의 decode는 토큰에 포함된 헤더와 페이로드(Payload)를 읽는 과정으로 이것만으로는 토큰의 위조 여부를 확인할 수 없습니다. 하지만 verify는 서버의 검증 키를 이용해 JWT 서명이 올바른지 확인하는 과정입니다. 현재 작성한 코드에서는 AuthService.java의 currentUserId() 함수에서 Jwts.parser().verifyWith(key).build().parseSignedClaims(token)을 사용해 JWT 서명을 검증합니다. 이후 HS256 알고리즘, 발급자(iss), 사용 대상(aud), 발급 시각(iat), 만료 시각(exp), 사용자 ID(sub)의 형식과 실제 사용자 존재 여부를 추가로 검사합니다. 검증에 실패하게 되면 HTTP 401을 반환하며 토큰이 변조되었거나 정보가 올바르지 않은 경우 INVALID_TOKEN으로, 만료된 경우에는 TOKEN_EXPIRED로 구분됩니다. T09와 T10에서 각각 변조 토큰과 만료 토큰에 대한 HTTP 401의 응답을 확인했습니다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

(1) 로그아웃은 App.vue의 logout() 함수에서 clearSession()을 호출해 브라우저 메모리에 저장된 JWT와 사용자 정보를 초기화하고, 화면을 로그아웃 상태로 변경합니다. 하지만 서버에서는 JWT를 별도로 폐기하는 기능은 구현되어 있지 않기 때문에 이미 복사된 토큰은 로그아웃만으로 무효화가 되지 않고 만료 전까지 사용할 수 있습니다.
(2) 토큰 만료는 AuthService.java에서 설정한 JWT의 만료 시간ㅇ니 지난 상태를 의미합니다. 만료된 토큰으로 요청하게 되면 서버가 HTTP 401과 TOKEN_EXPIRED를 반환합니다. 이후 api.js의 request() 함수는 401 응답을 받으면 clearSession()을 실행해 로그인 정보를 초기화하고, App.vue에서 로그인 시간이 만료되었다는 안내 메시지를 표시하도록 합니다.

(3) 새로고침은 프론트엔드의 JavaScript가 다시 실행되는 동작입니다. 현재 작성한 코드는 JWT를 별도의 저장소가 아닌 session 변수에서만 보관하므로 새로고침하면 로그인 상태가 초기화됩니다. 이때 서버에 저장된 게시글 데이터는 유지되며, 공개 게시글 목록은 다시 조회할 수 있습니다.
=> 따라서 이미 복사된 JWT는 로그아웃이나 새로고침만으로 무효화되지 않지만, 토큰의 만료 시간이 지나면 서버의 검증 과정에서 거부되는 것을 알 수 있습니다. 이러한 동작은 T10의 토큰 만료, T11의 로그아웃, T16의 새로고침 테스트를 통해 확인했습니다.

5. 가장 어려웠던 오류 한 가지와 원인 확인·수정·재검증 과정을 설명하라.

: 가장 어려웠던 오류 중 하나는 백엔드 구현 및 테스트 과정에서 발생한 오류이며, JWT 인증이 필요한 API 요청에서 HTTP 401 오류가 발생했을 때 원인을 확인하고 해결하는 과정이었습니다. 처음에는 존재하지 않는 게시글에 수정 요청을 보내 HTTP 404 응답을 확인하려고 했지만, 정상 JWT가 포함되지 않은 요청에서는 HTTP 401 오류가 먼저 발생했습니다. 원인을 확인하기 위해 AuthService.java의 currentUserId() 함수를 살펴보았고, 서버가 Authorization 헤더에서 Bearer 토큰을 추출한 뒤 서명과 만료 여부 등을 검증한다는 것을 확인했습니다. 이 문제를 해결하기 위해 서버의 인증 검사를 제거하는 대신, 테스트 요청의 Authorization 헤더에 정상 JWT를 포함하도록 fetch() 요청 코드를 수정했습니다. 이후 동일하게 존재하지 않는 게시글 ID에 PUT 요청을 다시 전송한 결과, HTTP 404 Not Found와 JSON 오류 코드 NOT_FOUND가 반환되는 것을 확인했습니다. 이러한 과정을 통해 보호 API에서는 게시글의 존재 여부를 확인하기 전에 사용자 인증이 수행될 수 있다는 것을 이해하게 되었으며, HTTP 오류가 발생했을 때 서버 코드와 요청 헤더를 함께 확인하는 것이 중요하다는 것을 배웠습니다.

* 현재 남은 문제와 다음에 개선할 점: 테스트 T13에서 실제 확인하지 못한 제목 길이 초과는 구현을 해봤지만 본문 길이 초과로 인해 서버 측 길이 초과 검증 결과를 직접 확인하지 못했습니다. 다음에 Edge개발자 도구를 활용해 제한 길이를 초과하는 요청을 서버에 직접 전송하고, HTTP 400 BAD_INPUT 응답 여부를 추가 검증할 계획입니다. 또한 분실물 검색 및 카테고리 분류 기능을 추가해 사용자의 편의성을 개선하고자 합니다.